



**федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)**

**Институт
информационных технологий**

**Кафедра
информационных систем**

Мустафаева Полина Махировна

**Тема: «Разработка информационной системы анализа предпочтений
пользователей при выборе кинопродукции для просмотра»**

**Выпускная квалификационная работа на присвоение квалификации
«бакалавр» по направлению 09.03.02 «Информационные системы и технологии»**

Регистрационный № 2026Б-_____

**Заведующий кафедрой
к.т.н., доцент**

Бильчук М.В.

подпись

**Руководитель
ст. преподаватель**

Быстрикова В.А.

подпись

**Обучающийся
группы ИДБ-22-06**

Мустафаева П.М.

подпись

Москва, 2026 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	4
ГЛАВА 1. ИССЛЕДОВАНИЕ ПРОЦЕССА ВЫБОРА КИНОПРОДУКЦИИ ДЛЯ ПРОСМОТРА И ОБОСНОВАНИЕ НЕОБХОДИМОСТИ СОЗДАНИЯ АНАЛИТИЧЕСКОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ ПРЕДПОЧТЕНИЙ ПОЛЬЗОВАТЕЛЕЙ.....	7
1.1. ИССЛЕДОВАНИЕ ПРОЦЕССА ВЫБОРА КИНОПРОДУКЦИИ ДЛЯ ПРОСМОТРА	7
1.2. АНАЛИЗ СУЩЕСТВУЮЩИХ ИНФОРМАЦИОННЫХ СИСТЕМ.....	8
1.3. ОБОСНОВАНИЕ НЕОБХОДИМОСТИ РАЗРАБОТКИ ИНФОРМАЦИОННОЙ СИСТЕМЫ.....	19
1.4. АНАЛИЗ ПРАВОВЫХ И НОРМАТИВНО-ТЕХНИЧЕСКИХ ДОКУМЕНТОВ.....	20
1.4.1. АНАЛИЗ ПРАВОВЫХ ДОКУМЕНТОВ.....	20
1.4.2. АНАЛИЗ НОРМАТИВНО-ТЕХНИЧЕСКИХ ДОКУМЕНТОВ.....	22
ГЛАВА 2. РАЗРАБОТКА ТРЕБОВАНИЙ И ФУНКЦИОНАЛЬНЫХ МОДЕЛЕЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ	25
2.1. РАЗРАБОТКА ТРЕБОВАНИЙ К ИНФОРМАЦИОННОЙ СИСТЕМЕ.....	25
2.1.1. РАЗРАБОТКА СИСТЕМНЫХ ТРЕБОВАНИЙ.....	25
2.1.2. РАЗРАБОТКА ТРЕБОВАНИЙ К ЦЕЛЕВОМУ ПРОГРАММНОМУ ПРОДУКТУ	28
2.2. РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ МОДЕЛЕЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ.....	32
ГЛАВА 3. РЕАЛИЗАЦИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ.....	40
3.1. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ	40
3.2. ПРОГРАММИРОВАНИЕ И ТЕСТИРОВАНИЕ КОМПОНЕНТ ПО И/ИЛИ ФУНКЦИЙ И ИНТЕРЕЙСА БД	51
3.2.1. ОБОСНОВАНИЕ ВЫБОРА СРЕДСТВ ПРОГРАММНОЙ РЕАЛИЗАЦИИ СИСТЕМЫ И/ИЛИ БД	51

3.2.2. ПРОГРАММИРОВАНИЕ, ТЕСТИРОВАНИЕ И ИНТЕГРАЦИЯ ЦЕЛЕВОГО ПО И/ИЛИ ФУНКЦИЙ И ИНТЕРФЕЙСА БД СИСТЕМЫ	54
3.3. ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ СИСТЕМЫ.....	58
ЗАКЛЮЧЕНИЕ	82
СПИСОК ЛИТЕРАТУРЫ.....	84
ПРИЛОЖЕНИЕ. ПРОГРАММНЫЙ КОД.....	88

ВВЕДЕНИЕ

Современные пользователи сталкиваются с существенными трудностями при выборе кинопродукции из-за непрерывного расширения баз на цифровых платформах, что приводит к информационной перезагрузке и усложняет процесс подбора подходящих фильмов и сериалов. Исследования подтверждают, что рекомендательные системы массовых сервисов основываются на усредненных алгоритмах, не учитывающих индивидуальные предпочтения зрителей, что снижает точность и качество рекомендаций [1].

В результате зрителю часто приходится осуществлять выбор кинопродукции интуитивно, опираясь на рейтинги, отзывы или не персонализированные рекомендации. Однако значительная часть аудитории отдает предпочтение выбору, который основан на доверии к людям с близкими вкусовыми и похожими предпочтениями. В научной литературе отмечается рост влияния социальных механизмов в принятии решения зрителями, а также рост влияния «сообществ интересов», которые способствуют обмену опытом среди пользователей с похожими вкусами [2]. Это подтверждает актуальность разработки систем, способных находить людей с совпадающими вкусами и использовать их опыт в качестве ориентиров для формирования рекомендаций.

Актуальность разработки возрастает благодаря развитию сервисов, которые используют сравнение пользовательских профилей для улучшения взаимодействия внутри цифровой среды. В отличие от таких систем, в которых идет направленность на автоматическую выдачу списка фильмов, более перспективным является подход, направленный на подбор аналогичных профилей. Такой механизм позволит не только просматривать рекомендации людей, которые смотрят схожий контент, но и инициировать коммуникацию для обмена впечатлениями или дополнительными материалами.

Цель работы – информационная поддержка процесса выбора пользователями фильмов и сериалов для просмотра на основе создания

аналитической информационной системы.

Для достижения этой цели поставлены следующие задачи:

- рассмотреть процессы предметной области, связанные с формированием пользовательских предпочтений и ведением списка кинопродукции;
- выполнить анализ существующих информационных систем, обладающих схожим функционалом;
- обосновать необходимость разработки новой системы, ориентированной на поиск пользователей с похожими интересами;
- изучить правовые и нормативно-технические документы, регулирующие обработку данных и функционирование информационных систем.

Объектом являются процессы выбора кинопродукции для просмотра и взаимодействия пользователей на платформах. Предметом процесс анализа предпочтений пользователей при выборе кинопродукции для просмотра.

В структуре данной работы можно выделить несколько основных частей. Во введении обоснована актуальность темы исследования, сформулированы цели и задачи работы, определены объект и предмет исследования. В первой части проводится исследование процессов предметной области, связанных с выбором кинопродукции и взаимодействием пользователей на цифровых платформах. Во второй части выполняется анализ существующих информационных систем и подходов к персонализированным рекомендациям. Выполняется сопоставление функционала этих систем, выявляются их сильные и слабые стороны. В третьей части обосновывается необходимость разработки новой информационной системы, ориентированной на поиск пользователей с похожими вкусами и формирование подборки профилей для взаимодействия. В четвертой части выполняется анализ правовых и нормативно-технических документов, регулирующих обработку данных и функционирование информационных систем.

В качестве методов исследования использовались системный анализ, сравнение и конкретизация. Совокупность этих методов позволила решить поставленные задачи и достичь цели работы. Итогом исследования является информационная система, которая обеспечивает добавление кинопродукции в коллекции, поиск пользователей с близкими вкусами и предоставляет возможность обмена рекомендациями и обсуждения контента между пользователями внутри платформы.

ГЛАВА 1. ИССЛЕДОВАНИЕ ПРОЦЕССА ВЫБОРА КИНОПРОДУКЦИИ ДЛЯ ПРОСМОТРА И ОБОСНОВАНИЕ НЕОБХОДИМОСТИ СОЗДАНИЯ АНАЛИТИЧЕСКОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ ПРЕДПОЧТЕНИЙ ПОЛЬЗОВАТЕЛЕЙ

1.1. ИССЛЕДОВАНИЕ ПРОЦЕССА ВЫБОРА КИНОПРОДУКЦИИ ДЛЯ ПРОСМОТРА

Процесс выбора кинопродукции для просмотра является повседневной задачей для широкой аудитории пользователей цифровых сервисов. В условиях развития стриминговых платформ и онлайн-кинотеатров объем доступного контента постоянно растет. Пользователь сталкивается с большим количеством предложений, отличающихся по жанру, тематике, рейтингу, году выпуска и другим характеристикам.

На первом этапе возникает потребность в просмотре – по настроению, жанру или рекомендации знакомых. Далее осуществляется поиск информации о фильме или сериале. Для этого используются поисковые системы, специализированные сайты с рейтингами и отзывами, а также каталоги онлайн-кинотеатров. Пользователь анализирует описание, актерский состав, трейлеры, отзывы и рейтинги, после чего принимает решение о просмотре.

Формирование интересов происходит постепенно. Просмотренные фильмы запоминаются, иногда фиксируются в заметках, социальных сетях или на сторонних сервисах учета просмотров. Однако единая система хранения информации о просмотренном контенте у большинства пользователей отсутствует. История просмотров распределена между разными платформами или хранится фрагментарно, что затрудняет анализ собственных предпочтений.

Отдельным процессом является обмен рекомендациями. В реальной практике рекомендации передаются через личное общение, мессенджеры или социальные сети. Оценка совпадения вкусов носит субъективный характер:

пользователь ориентируется на личное мнение о том, «похож ли вкус» другого человека. Формализованного механизма сопоставления интересов обычно не применяется.

В ходе исследования предметной области были выделены следующие критичные процессы:

- поиск кинопродукции;
- хранение информации о просмотренных, любимых и планируемых к просмотру фильмах/сериалах;
- сопоставление интересов разных пользователей;
- обмен рекомендациями и обсуждение контента.

Анализ существующей практики показал, что данные процессы реализуются разрозненно. Поиск выполняется на одних платформах, учет просмотров – на других, а сравнение вкусов фактически отсутствует как формализованный процесс. Пользователь вынужден самостоятельно сопоставлять списки фильмов и делать выводы о схожести интересов.

Критичными для автоматизации являются процессы структурированного хранения информации о просмотренном контенте и сопоставления пользовательских предпочтений. Без автоматизации анализ совпадений затруднен и требует значительных временных затрат. Вследствие этого снижается удобство выбора и качество рекомендаций.

В итоге проведенное исследование позволило определить основные процессы предметной области и выявить проблемы, связанные с отсутствием интегрированной информационной поддержки. Полученные результаты используются для обоснования необходимости разработки информационной системы и формирования требований к ее функционалу.

1.2. АНАЛИЗ СУЩЕСТВУЮЩИХ ИНФОРМАЦИОННЫХ СИСТЕМ

Для оценки перспективности разработки была проведена сравнительная характеристика существующих систем, частично или полностью выполняющих задачи, близкие к задачам разрабатываемой платформы.

Несмотря на отсутствие систем, которые одновременно совмещают функции хранения кинопродукции, управления пользовательскими коллекциями и анализа пересечений интересов на основе математических алгоритмов, некоторые решения предоставляют отдельные элементы необходимого функционала.

В качестве первого программного продукта рассмотрим сервис «Иви». «Иви» – российский онлайн-кинотеатр, один из ведущих в России, с развитой системой персональных рекомендаций [3]. На платформе применяется набор рекомендательных механизмов, которые учитывают явные и неявные пользовательские действия (просмотры, фичи фильмов, оценки) и комбинируют различные подходы для формирования подборок [4].

«Иви» предоставляет следующие основные возможности:

- Каталог фильмов и сериалов с возможностью применения фильтров (рис. 1.1).

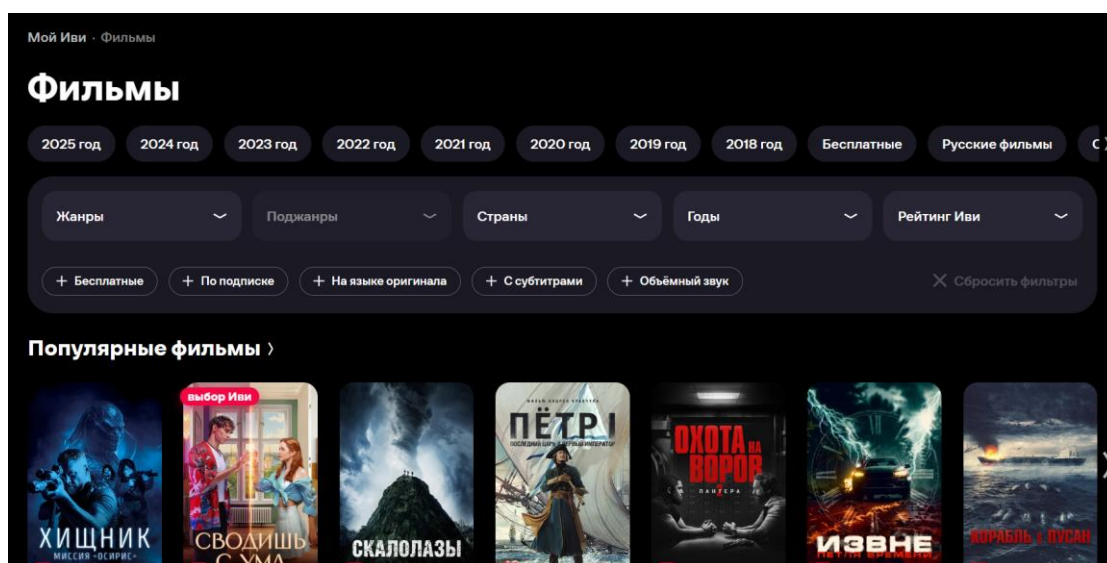


Рис. 1.1. Каталог фильмов и сериалов

- Личный кабинет с коллекциями «Буду смотреть» и «Просмотрено» (рис. 1.2, 1.3). В коллекцию «Буду смотреть» пользователь сам добавляет понравившиеся фильмы и сериалы, а в коллекцию «Просмотрено» они добавляются только после просмотра.

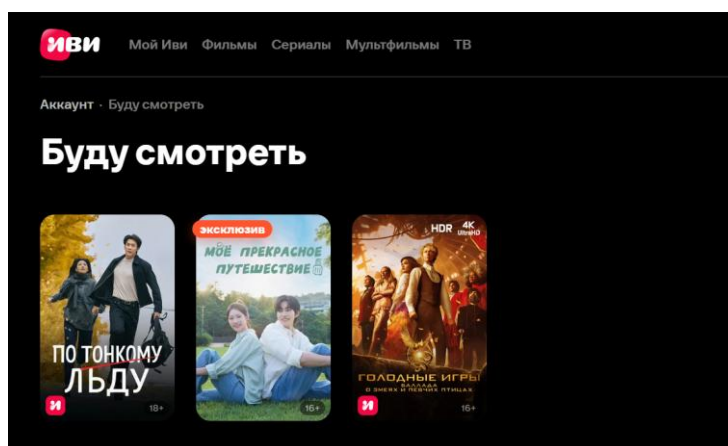


Рис. 1.2. Коллекция «Буду смотреть»

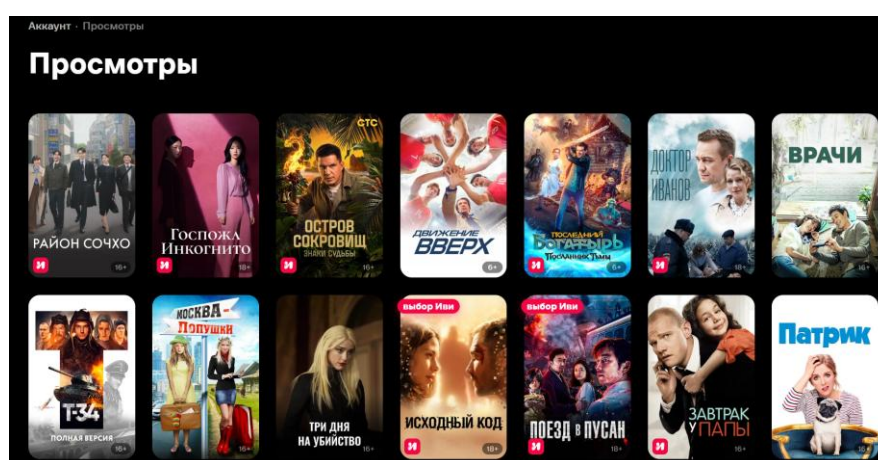


Рис. 1.3. Коллекция «Просмотры»

- Блок с рекомендациями «Подобрали только для тебя», который формируется после каждого просмотра или оценки (рис. 1.4).

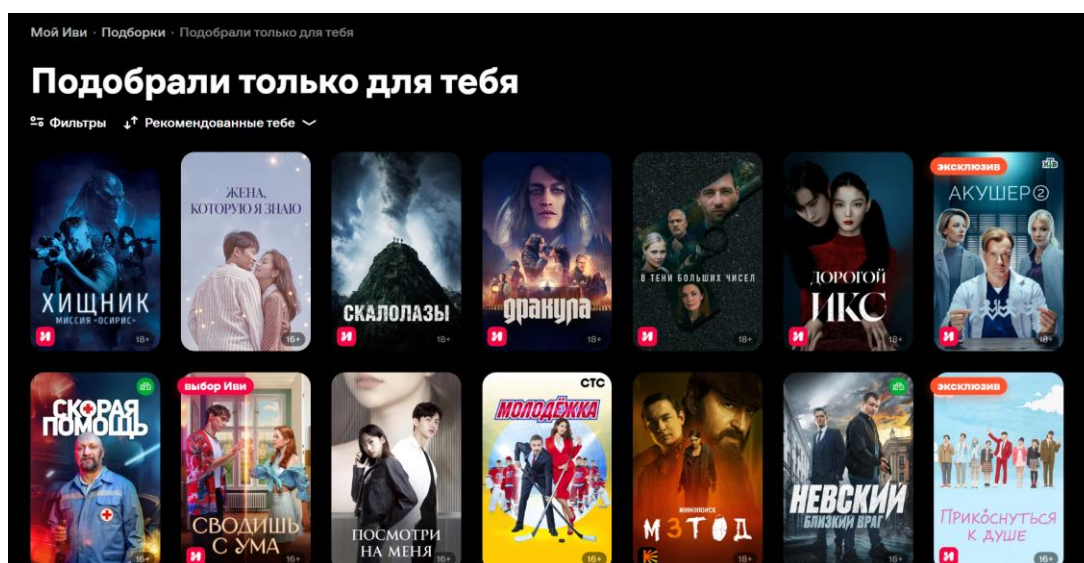


Рис. 1.4. Блок «Подобрали только для тебя»

- Подборка «Для меня» с процентом совпадения с моими интересами (рис. 1.5).

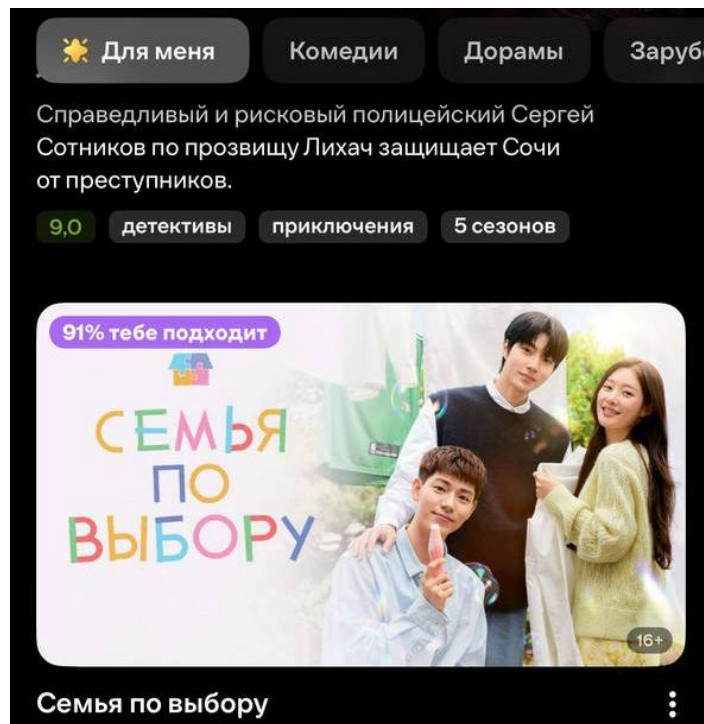


Рис. 1.5. Подборка «Для меня»

Сильные стороны платформы «Иви»:

- Алгоритмы рекомендаций, учитывающие историю просмотров внутри платформы.
- Простая и понятная система коллекций.
- Прямая интеграция каталога и плеера для просмотра.

Слабые стороны платформы «Иви»:

- Отсутствие функции автоматического сопоставления профилей по проценту совпадения коллекций.
- Нет возможности коммуникации между пользователями в виде чатов и личных сообщений.

Следующим программным продуктом рассмотрим «Кинопоиск». «Кинопоиск» – это крупнейший русскоязычный интернет-сервис и онлайн-кинотеатр, который совмещает функции каталога, рейтингов и рекомендательных лент [5]. Система предлагает обширный каталог

кинопродукции, рейтинги, рецензии и возможность составления личных коллекций.

Основные функции платформы «Кинопоиск» включают в себя:

- Поиск кинопродукции по названию, жанру, году и другим параметрам (рис. 1.6).

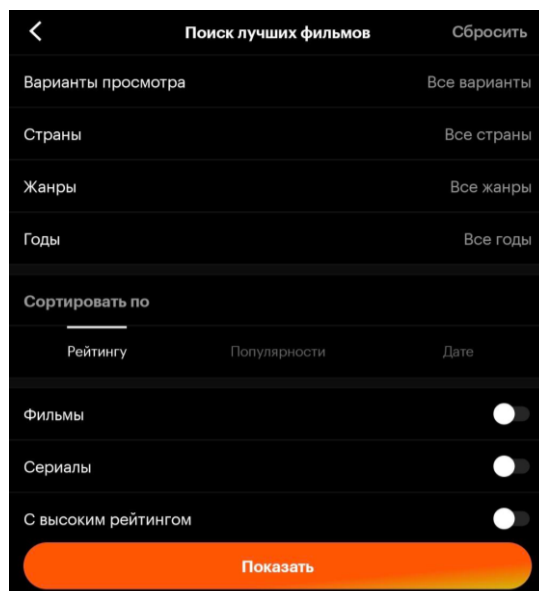


Рис. 1.6. Поиск с фильтрами

- Возможность добавления кинопродукции в папки «Буду смотреть», «Любимые фильмы», «Избранное», «Смотреть в кино», «Найти в интернете», «Купить на DVD» и «Смотреть на ТВ» (рис. 1.7).

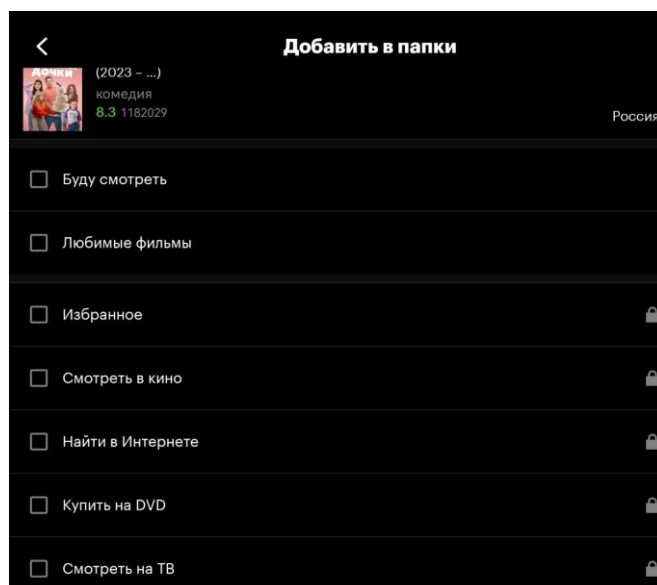


Рис. 1.7. Добавление в папки

- Рекомендации на основе просмотренного и оценок в разделах «Сериалы на основе ваших интересов» и «Фильмы для вас» (рис. 1.8).



Рис. 1.8. Рекомендации на основе просмотренного и оценок

- Интеграция с онлайн-кинотеатром для просмотра (рис. 1.9).

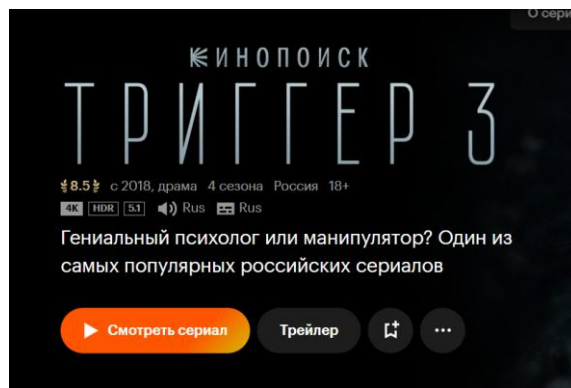


Рис. 1.9. Онлайн-кинотеатр

- Социальные функции в виде написания и чтения рецензий, просмотра коллекций других пользователей (рис. 1.10, 1.11).

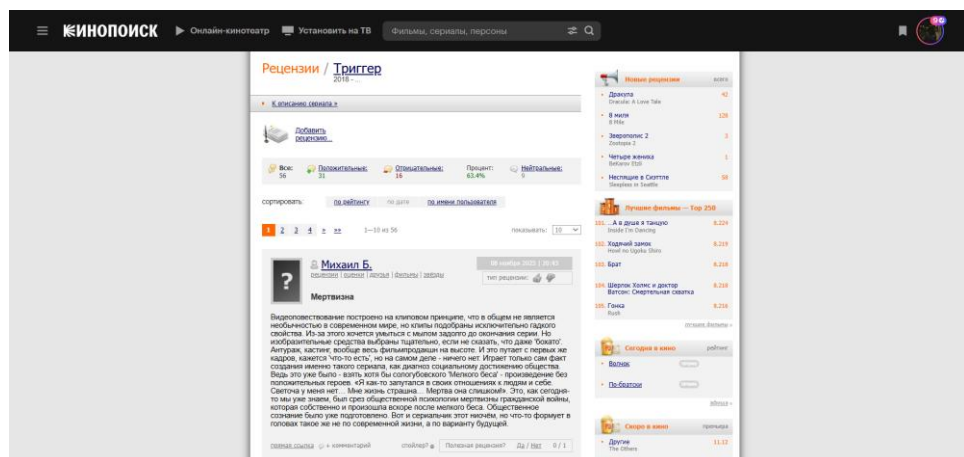


Рис. 1.10. Рецензии

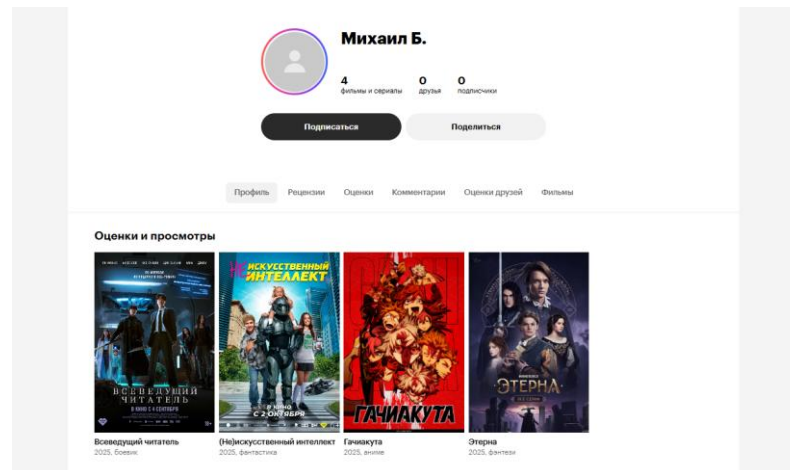


Рис. 1.11. Просмотр коллекций другого пользователя

Сильные стороны:

- Огромная и актуальная база данных.
- Удобный интерфейс.
- Наличие социальных функций.

Слабые стороны:

- Нет встроенного чата для непосредственного общения пользователей с похожими предпочтениями.
- Минимальный анализ пересечения коллекций для целенаправленного поиска единомышленников.

Также будет рассмотрена платформа «Letterboxd». «Letterboxd» – социальная сеть, посвященная обсуждению и поиску фильмов, в которой есть функции составления списков и подписки на других пользователей. Пользователи могут использовать ее как дневник – делиться мнениями о фильмах в процессе их просмотра или же вести учет уже просмотренных [6].

Основными функциями «Letterboxd» являются:

- Ведение личного «дневника просмотров». В сервисе можно отмечать фильмы как просмотренные, указывать дату, повторные просмотры, ставить оценки и писать короткие заметки (рис. 1.12, 1.13).

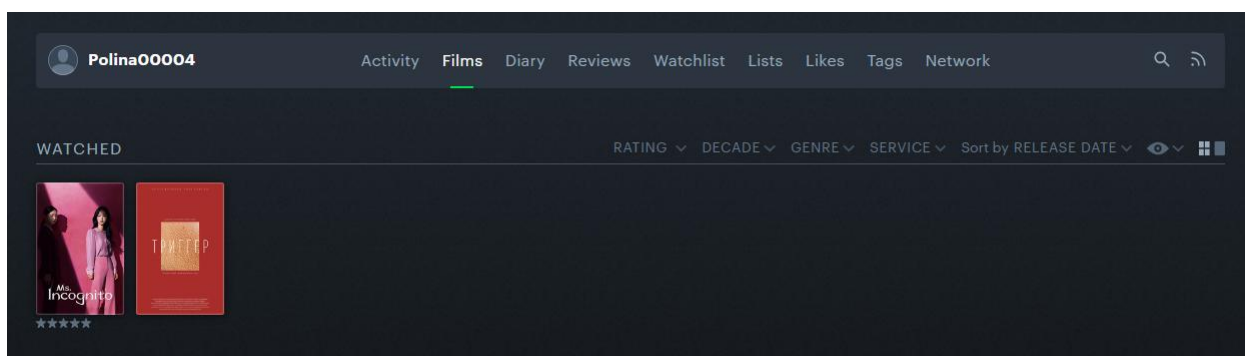


Рис. 1.12. Просмотренные фильмы

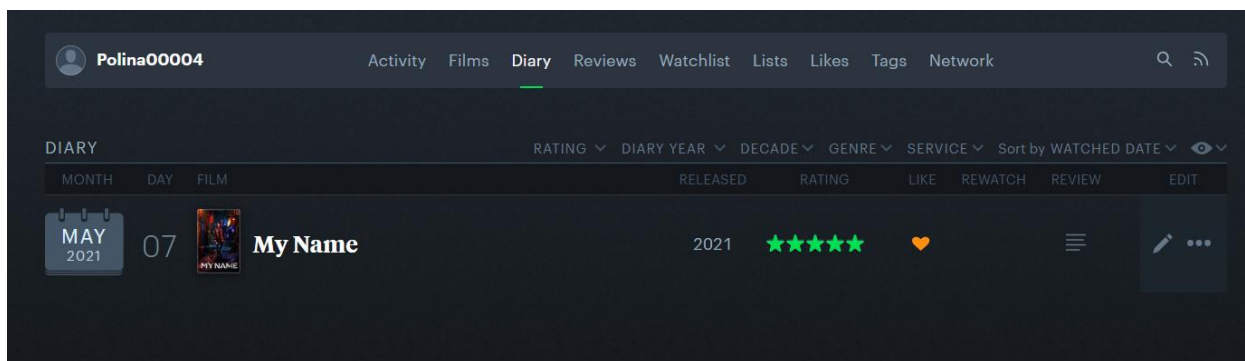


Рис. 1.13. Ведение личного «дневника просмотров»

- Создание коллекций и списков (рис. 1.14). Платформа позволяет собирать собственные подборки – от тематических списков (например, «лучшие фильмы для просмотра с друзьями») до личных (например, «посмотреть позже»). Списки могут быть открытыми или приватными.

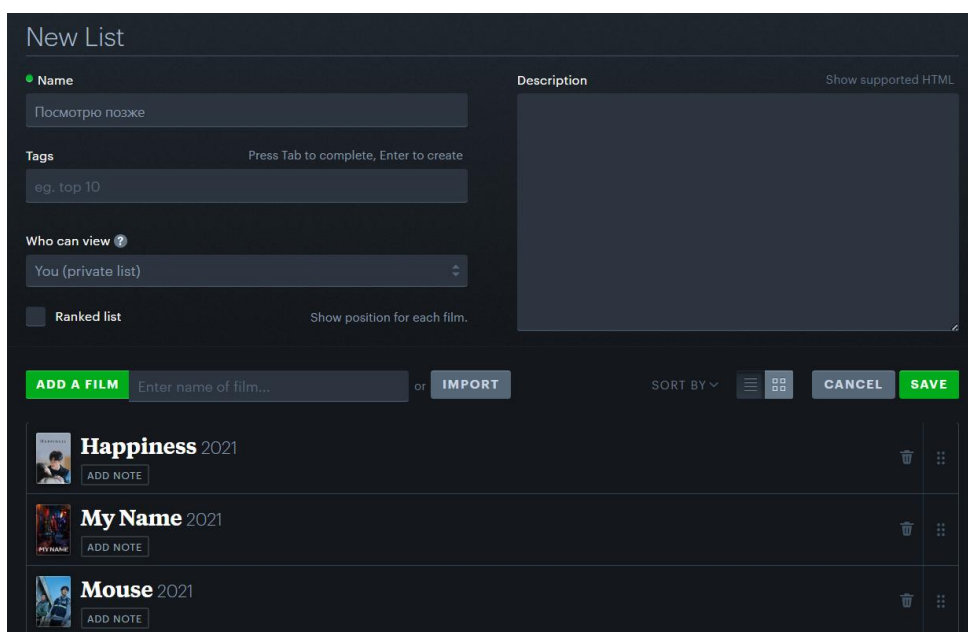


Рис. 1.14. Создание персональной коллекции

- Просмотр ленты активности и подписки на пользователей (рис. 1.15, 1.16). Пользователь видит, что смотрят люди, на которых он подписан: оценки, обзоры, составленные списки. Лента формируется по типу социальной сети и позволяет ориентироваться на предпочтения других людей.

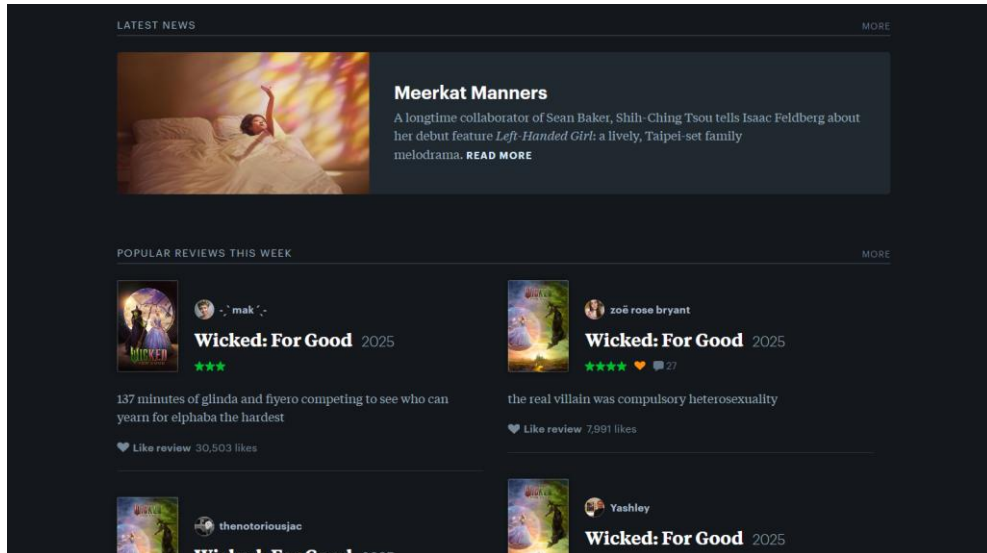


Рис. 1.15. Просмотр ленты активности

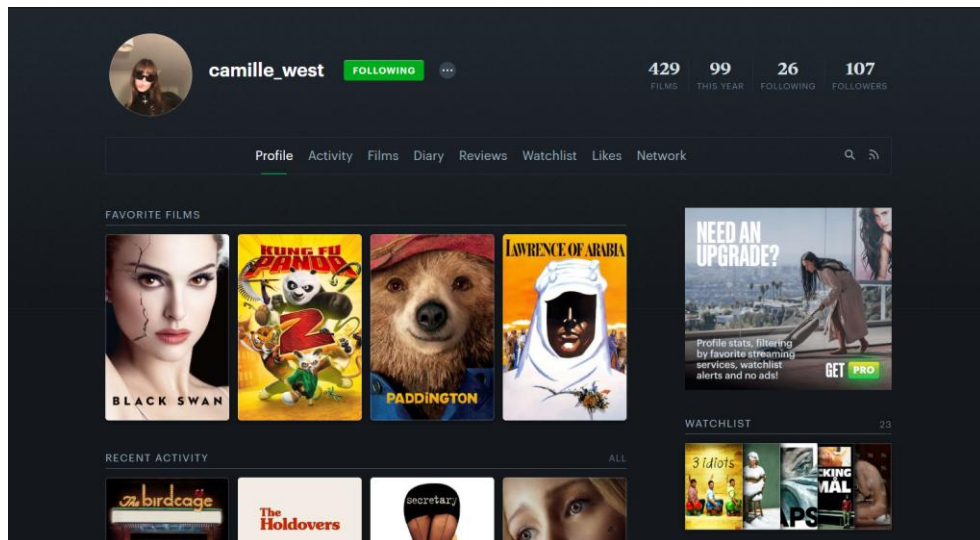


Рис. 1.16. Подписка на пользователя

- Написание обзоров и комментариев (рис. 1.17). На платформе можно публиковать как полноценные рецензии, так и просто оставлять комментарии под чужими обзорами.

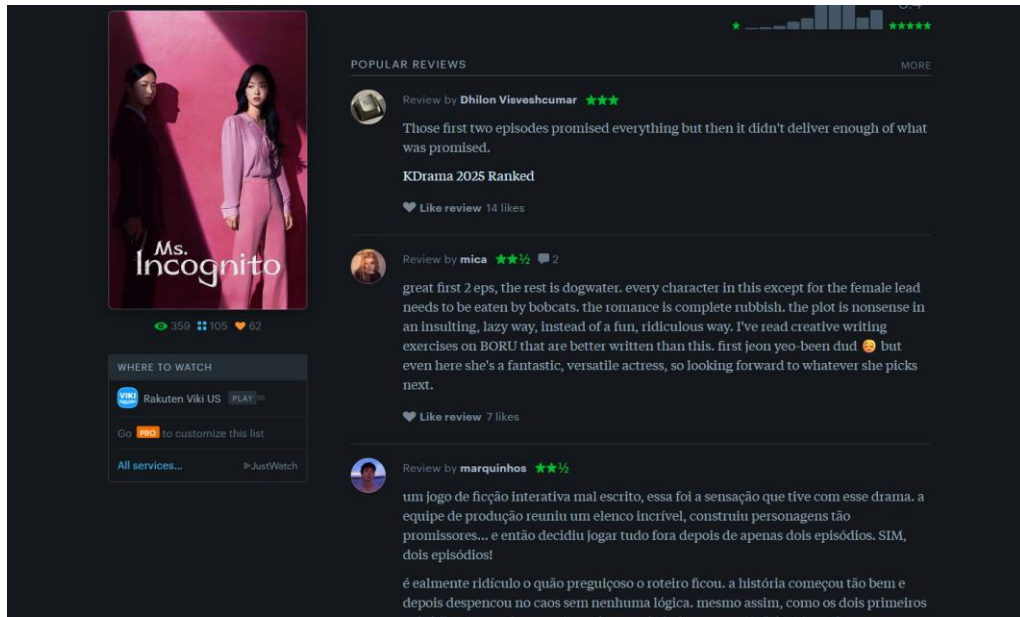


Рис. 1.17. Комментарии от пользователей

Сильные стороны «Letterboxd»:

- Коллекции и списки, которые можно настроить под себя.
- Подробный дневник просмотров. Функции отметки фильмов, просмотров, оценок и заметок помогают вести структурированный и понятный журнал просмотров.
- Развитое социальное сообщество. Большое количество активных пользователей формирует живую базу отзывов, списков и обсуждений. Также платформа построена вокруг общения и обмена впечатлениями, где пользователи могут обсуждать фильмы и ориентироваться на вкусы других.

Слабые стороны «Letterboxd»:

- Отсутствие автоматического подбора похожих пользователей. На данной платформе нет сравнения коллекций и списков других людей и не выдает процент совпадения, то есть поиск идет вручную, путем просмотра списков, обзоров и активности в ленте.
- Нет встроенных чатов. Общение ограничено комментариями и публичными обсуждениями.
- Ориентация только на фильмы. Функционал ориентирован преимущественно на кино. Работа с сериалами представлена в ограниченном

формате.

Чтобы было удобнее оценить отличие представленных сервисов и понять, каких возможностей не хватает пользователям, основные функции были сведены в сопоставительную таблицу (табл. 1.1).

Таблица 1.1.

Сопоставительная таблица целевых функций

Целевая функция	Иви	Кинопоиск	Letterboxd	Разрабатываемая система
Поиск кинопродукции	+	+	+	+
Хранение информации о просмотренных, любимых и планируемых фильмах	+	+	+	+
Формализованное сопоставление интересов пользователей (на основе коллекций)	—	—	—	+
Коммуникация между пользователями	—	+—	+	+

Сопоставление существующих информационных систем показало, что процессы, выделенные в разделе 1.1 как критичные для информационной поддержки (поиск кинопродукции, ведение пользовательских коллекций, сопоставление интересов и коммуникация), реализованы в рассмотренных сервисах частично. Поиск и хранение информации о просмотренном контенте поддерживаются во всех системах, однако формализованное сопоставление коллекций пользователей отсутствует. Коммуникационные возможности представлены преимущественно в формате комментариев и рецензий и не связаны с автоматическим подбором пользователей со схожими интересами.

В итоге проведенный анализ подтверждает отсутствие решений, объединяющих хранение коллекций, автоматическое вычисление процента совпадения интересов и целенаправленную коммуникацию между пользователями. Полученные результаты используются при формировании

требований к разрабатываемой системе, определении перечня обязательных функций и уточнении логики сопоставления пользовательских профилей в рамках дальнейшего проектирования.

1.3. ОБОСНОВАНИЕ НЕОБХОДИМОСТИ РАЗРАБОТКИ ИНФОРМАЦИОННОЙ СИСТЕМЫ

Материалы из предыдущих разделов выявили, что процессы выбора фильмов и сериалов, ведения личных коллекций и сопоставление предпочтений пользователей нуждаются в приоритетной автоматизации. В пункте 1.1 показано, как поиск кинопродукции, хранение списков, расчет процента совпадения профилей и общение напрямую влияют на удобство работы с платформами. Эти этапы определяют, насколько просто пользователю разбираться огромных каталогах и находить единомышленников по кинематографическим вкусам.

Сопоставление существующих систем в разделе 1.2 подтвердило, что сервисы покрывают лишь часть задач. Некоторые платформы отлично рекомендуют фильмы, но в них нет сравнения пользователей со схожими вкусами. Другие делают акцент на социальных связях, однако не предлагают автоматического анализа коллекций на предмет пересечений. Получается, что отдельные элементы нужного функционала существуют, но они распределены между разными системами и не объединены в единую логику.

По этой причине опираться на готовые платформы невозможно. Их структура и назначение ориентированы на другие задачи, а значит встроить туда механизм сравнения коллекций или подбор профилей не получится. Приобретение или заказ подобной системы также не решит задачу, потому что требуемый функционал является специфичным, ведь он должен учитывать способ хранения коллекций, принципы подсчета совпадений и возможность общения между пользователями. Готовых решений с такой комбинацией функций нет.

Вследствие этого требуется собственная информационная система, объединяющая поиск и добавление кинопродукции в коллекции, автоматическую оценку сходства предпочтений, рекомендации профилей и встроенное общение. Такой подход закрывает выделенные в анализе недостатки и создает удобную платформу для тех, кто ориентируется на предпочтения реальных людей, а не на алгоритмические рекомендации.

Цель данной работы заключается в создании мобильного приложения для выбора кинопродукции через анализ пользовательских предпочтений и подбор схожих профилей.

Для достижения поставленной цели выполняются следующие задачи:

- рассмотреть особенности выбора кинопродукции и обосновать необходимость создания системы, основанной на сравнении пользовательских предпочтений;
- разработать функциональные модели и требования к системе анализа предпочтений;
- реализовать мобильное приложение, которое объединяет подбор кинопродукции, анализ коллекций и коммуникацию между пользователями.

1.4. АНАЛИЗ ПРАВОВЫХ И НОРМАТИВНО-ТЕХНИЧЕСКИХ ДОКУМЕНТОВ

1.4.1. АНАЛИЗ ПРАВОВЫХ ДОКУМЕНТОВ

Проанализированы федеральные законы, регулирующие обработку информации и персональных данных в мобильных приложениях. Федеральный закон РФ от 27 июля 2006 года № 149-ФЗ «Об информации, информационных технологиях и о защите информации» устанавливает основные принципы работы с информацией и определяет обязанности разработчиков и пользователей [7]. Статья 3 закрепляет право на поиск, получение и передачу информации, что формирует основу функционала поиска контента и обмена рекомендациями внутри приложения. Статья 13

требует внедрения мер, защищающих информацию от несанкционированного доступа, включая резервное копирование, восстановление данных и мониторинг безопасности. Статья 16 обязывает использовать технические средства и методы, предотвращающие сбои в работе системы и нарушение ее функционирования, что важно для стабильной работы мобильного приложения.

Данный закон применяется при проектировании требований к безопасности приложения: реализуется защита коллекций пользователей и чатов от утечек, обеспечивается контроль доступа к профилям.

Федеральный закон РФ от 27 июля 2006 года № 152-ФЗ «О персональных данных» регулирует порядок обработки персональной информации пользователей информационных систем [8]. В рамках разрабатываемого программного продукта требования данного закона применяются на этапе проектирования структуры пользовательского профиля, механизмов хранения данных и организации пользовательского взаимодействия.

Положения статьи 5, устанавливающие законность и целевую направленность обработки персональных данных, учитываются при определении состава информации, хранимой в пользовательском профиле. В системе предусматривается хранение только тех данных, которые необходимы для формирования коллекций кинопродукции и расчета степени схожести предпочтений. Принципы минимизации, точности и актуальности данных, указанные в статье 9, реализуются за счет ограничения набора собираемых сведений и возможности их редактирования пользователем.

Требования статьи 18 учитываются при проектировании процесса регистрации пользователя. В интерфейсе приложения предусматривается получение согласия на обработку персональных данных до начала использования функциональных возможностей системы. Положения статьи 19 применяются при разработке механизмов защиты информации, включая разграничение доступа к пользовательским профилям, коллекциям и

переписке, а также ограничение доступа к данным только для авторизованных пользователей.

В результате нормы Федерального закона № 152-ФЗ учитываются при формировании требований к структуре данных, логике работы пользовательского интерфейса и мерам защиты информации в разрабатываемой информационной системе.

1.4.2. АНАЛИЗ НОРМАТИВНО-ТЕХНИЧЕСКИХ ДОКУМЕНТОВ

Изучены нормативно-технические документы, определяющие процессы разработки программного обеспечения. ГОСТ Р ИСО/МЭК 12207-2010 «Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств» используется в работе как основа для структурирования этапов разработки информационной системы [9].

Положения стандарта применяются при формировании логики разработки программного продукта. Этап анализа требований соотносится с процессами определения и уточнения требований к системе. Процессы проектирования и разработки учитываются при построении архитектуры и функциональных моделей информационной системы. Требования к верификации и валидации используются при описании проверки корректности реализованных функций и соответствия программного продукта заданным требованиям. Процессы интеграции и тестирования учитываются при описании проверки взаимодействия компонентов системы.

Стандарт ГОСТ Р ИСО/МЭК 12207-2010 применяется в качестве методической основы для описания жизненного цикла программного продукта, определения последовательности этапов его проектирования, реализации и проверки, а также для обоснования логики перехода между этапами разработки.

ГОСТ Р ИСО/МЭК 25010-2015 «Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и

программного обеспечения (SQuaRE). Модели качества систем и программных продуктов» вводит модель качества программных продуктов [10]. В ней выделяются восемь основных характеристик. Функциональная пригодность показывает, насколько приложение выполняет нужные функции правильно и полностью. Производительность оценивает скорость работы и нагрузку на ресурсы. Совместимость отражает способность работать с другими системами. Удобство пользования связано с простотой интерфейса и комфортом работы. Надежность показывает устойчивость к сбоям и ошибкам. Защищенность обеспечивает сохранность и безопасность данных. Сопровождаемость оценивает удобство внесения изменений и исправления ошибок. Переносимость проверяет работу приложения на разных устройствах и платформах. Подхарактеристики, такие как доступность и целостность данных, применимы к коллекциям и чатам.

Документ применяется для оценки качества приложения: проверяется удобство поиска, надежность расчета сходства и мобильная переносимость. Модель качества, предложенная в нормативном документе, будет использована при разработке требований к характеристикам качества создаваемой информационной системы.

ГОСТ 34.602-2020 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы» определяет структуру ТЗ [11]. Обязательные разделы: цели создания, требования к функциям, режимам работы, диагностике и документированию. Требования к функциям включают точность, время выполнения и критерии отказов.

Стандарт будет использоваться при составлении технического задания на приложение.

Рассмотренные правовые и нормативно-технические документы определяют ограничения и требования, которые учитываются при формировании требований к информационной системе и целевому программному продукту. Учет данных положений используется в

последующих разделах при проектировании структуры системы, формулировании функциональных и качественных требований, а также при проверке соответствия разрабатываемого решения установленным нормам.

ГЛАВА 2. РАЗРАБОТКА ТРЕБОВАНИЙ И ФУНКЦИОНАЛЬНЫХ МОДЕЛЕЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ

2.1. РАЗРАБОТКА ТРЕБОВАНИЙ К ИНФОРМАЦИОННОЙ СИСТЕМЕ

2.1.1. РАЗРАБОТКА СИСТЕМНЫХ ТРЕБОВАНИЙ

Разработка требований к информационной системе выполняется на основе результатов исследования предметной области, анализа существующих решений и изучения нормативно-технических документов. В ходе анализа было показано, что для эффективной поддержки процессов выбора кинопродукции и взаимодействия пользователей требуется формализация как системных требований, так и требований к целевому программному продукту.

В соответствии с положениями ГОСТ Р ИСО/МЭК 25010-2015 информационная система рассматривается как совокупность взаимосвязанных компонентов, обеспечивающих реализацию целевых функций и заданный уровень качества [10]. Требования формируются по каждому компоненту системы, что позволяет заранее определить условия ее функционирования и ограничения, в рамках которых будет осуществляться разработка.

В рамках данной работы информационная система анализа предпочтений пользователей при выборе кинопродукции включает минимальный набор компонентов компьютерной системы, определенный стандартом ГОСТ Р ИСО/МЭК 25010-2015 [10]. К таким компонентам относится вычислительная техника, нецелевое программное обеспечение, нецелевые данные и целевой программный продукт.

Требования к вычислительной технике.

Архитектура разрабатываемой системы строится на взаимосвязи мобильного приложения, облачной платформы «Firebase» и внешнего API Кинопоиска. Такой подход позволяет минимизировать затраты на серверную инфраструктуру и сосредоточиться на реализации целевой логики

приложения.

Информационная система ориентирована на использование мобильных устройств пользователей. Серверная часть реализуется с использованием облачных сервисов «Firebase», что исключает необходимость в поддержке собственного серверного оборудования.

Минимальные требования к пользовательским мобильным устройствам включают:

- операционная система Android версии не ниже 8.0. Данная версия выбрана, так как она обеспечивает совместимость в Firebase SDK и охватывает большинство актуальных устройств;
- объем оперативной памяти не менее 3 ГБ;
- объем свободного внутреннего хранилища не менее 250 МБ. Дополнительное место требуется для кэширования результатов запросов к API и изображений постеров фильмов;
- разрешение экрана не ниже 1280x720 пикселей для корректного отображения интерфейса;
- стабильное подключение к сети Интернет, так как приложение работает с облачными данными в режиме реального времени.

Требования к нецелевому программному обеспечению.

К нецелевому программному обеспечению относятся операционные системы и прикладные программные средства, обеспечивающие функционирование целевого программного продукта, но напрямую не реализующие его целевые функции. Для разрабатываемой системы определен следующий перечень взаимодействующего нецелевого ПО:

1. Клиентское нецелевое ПО (Мобильная платформа): Целевое мобильное приложение функционирует под управлением операционной системы Android. Требуется обеспечение корректного взаимодействия с системными сервисами «Android» (менеджер уведомлений, службы синхронизации). Приложение должно использовать Android SDK версии 26 (Android 8.0) в качестве минимальной поддерживаемой версии.

Бесконфликтная работа с системой достигается за счет использования стандартных компонентов «Android» (Activity, Fragment, Service) и соблюдения их жизненного цикла.

2. Облачные сервисы Firebase (как платформа):

- **Firebase Authentication:** Сервис аутентификации, обеспечивающий регистрацию и вход пользователей. Поддерживается аутентификация по электронной почте и паролю.
- **Cloud Firestore:** NoSQL база данных для хранения коллекций фильмов пользователей и профилей. Используется для долговременного хранения информации о том, какие фильмы добавлены в «Просмотрено», «В планах» и «Любимое» Firestore обеспечивает синхронизацию данных в реальном времени и кэширование на устройстве.

3. Внешние источники данных (нецелевые БД):

- **Kinopoisk API:** Внешний сервис, предоставляющий доступ к базе данных фильмов и сериалов.
- **Ключевое ограничение:** Ключ API, используемый в проекте, имеет ограничение – не более 200 запросов в сутки.

Требования к нецелевым данным.

Под нецелевыми данными в данной архитектуре понимаются данные, хранящиеся во внешних источниках, с которыми взаимодействует приложение. Разрабатываемая система не управляет этими данными, а только применяет их.

Основным источником нецелевых данных является Kinopoisk API. К данному источнику предъявляются следующие требования:

- **Состав предоставляемых данных:** API должно предоставлять информацию о фильмах и сериалах, необходимую для реализации целевых функций: название, описание, постер, рейтинг, жанр, год выпуска и уникальный идентификатор фильма.

- **Формат данных:** Обмен данными должен осуществляться в формате JSON, что обеспечит совместимость с мобильным приложением на платформе Android.

- **Доступность:** API должно быть доступно не менее 95% времени для обеспечения возможности поиска и добавления контента.

- **Доступ через API-ключ:** Доступ к данным должен предоставляться по уникальному ключу, что позволит контролировать количество обращений со стороны приложения.

Данные требования относятся исключительно к характеристикам внешнего источника и не затрагивают логику обработки информации внутри разрабатываемого приложения.

2.1.2. РАЗРАБОТКА ТРЕБОВАНИЙ К ЦЕЛЕВОМУ ПРОГРАММНОМУ ПРОДУКТУ

Требования к целевому программному продукту включают требования к целевым функциям, определяющим функциональные возможности системы, а также требования к нецелевым функциям, обеспечивающим качество работы программного продукта в соответствии с моделью качества ГОСТ Р ИСО/МЭК 25010-2015 [10].

Требования к целевым функциям программного продукта.

Целевые функции программного продукта направлены на поддержку процессов хранения информации о кинопродукции, анализа пользовательских предпочтений и взаимодействия между пользователями. Целевая информационная система должна обеспечивать следующие функциональные возможности:

- **Поиск и получение информации о кинопродукции:** отправка поисковых запросов к Kinopoisk API и отображение результатов.

- **Управление персональными коллекциями:** добавление и удаление фильмов/сериалов в коллекции «Просмотрено», «В планах» и «Любимое» с сохранением данных в «Cloud Firestore».

- Просмотр коллекций других пользователей: доступ к открытым коллекциям других пользователей (если профиль не является приватным).
- Анализ пересечений и расчет схожести: вычисление процента совпадения предпочтений между пользователями путем сравнения содержимого коллекций.
- Формирование списка пользователей с близкими вкусами: ранжированный список пользователей по убыванию процента схожести с возможностью перехода в их профили.
- Обмен сообщениями: реализация чата между пользователями через «Cloud Firebase».

Требования к нецелевым функциям программного продукта.

Нецелевые функции обеспечивают качество реализации целевых функций и формируются в соответствии с характеристиками модели качества программного обеспечения, установленной ГОСТ Р ИСО/МЭК 25010-2015. В рамках разрабатываемой информационной системы устанавливаются следующие основные характеристики качества:

- Функциональная пригодность: программный продукт должен корректно реализовывать все заявленные целевые функции. Результаты анализа пользовательских предпочтений и расчета степени схожести должны быть логически обоснованными и воспроизводимыми при одинаковых входных данных.
- Уровень производительности: для комфортной работы пользователей устанавливаются следующие временные показатели: время запуска приложения не должно превышать 3 секунд, время отклика интерфейса при навигации по коллекциям – не более 1 секунды, время расчета схожести для 50 пользователей – не более 2 секунд. Достижение указанных показателей обеспечивается за счет асинхронной загрузки данных с использованием параллельных блоков кода Kotlin, кэширования данных Firestore на устройстве и оптимизации запросов к базе данных через составные

индексы.

- **Совместимость:** Система должна корректно взаимодействовать с внешними сервисами и компонентами: Kinopoisk API (через библиотеку Retrofit с обработкой ошибок сети и таймаутов), Firebase SDK актуальных версий, а также с операционной системой «Android» (API level 26 и выше). Взаимодействие с платформой «Android» должно быть бесконфликтным.

- **Удобство использования:** Пользовательский интерфейс должен быть интуитивно понятным и адаптированным под мобильные устройства. Основные пользовательские сценарии, такие как поиск контента и добавление в коллекции, должны выполняться за 3-4 касания экрана. Навигационная модель приложения строится на стандартном для «Android» паттерне Bottom Navigation View, обеспечивающем быстрый доступ к ключевым разделам. Формы ввода должны содержать валидацию с понятными сообщениями об ошибках: некорректные поля подсвечиваются, под ними отображается текст с пояснением причины ошибки, кнопка отправки формы блокируется до устранения всех несоответствий. Данная функция реализуется с использованием компонента TextInputLayout из библиотеки Material Design, который предоставляет встроенные механизмы отображения ошибок и подсветки полей. При первом запуске приложения отображаются экраны онбординга, знакомящие пользователя с основным функционалом: поиском фильмов, формированием коллекций и поиском людей со схожими вкусами.

- **Надежность:** Вероятность безотказной работы приложения в течение одной пользовательской сессии (30 минут) должна составлять не менее 0,99. Надежность обеспечивается всесторонней обработкой исключительных ситуаций с использованием блоков try-catch, механизмами повторных подключений при сбоях сети, сохранением данных в кэше Firestore при отсутствии соединения и применением транзакций Firestore для гарантии целостности данных при конкурентном доступе.

- **Защищенность:** Доступ к персональным данным пользователей должен предоставляться только после успешной аутентификации, время

блокировки неактивной сессии не должно превышать 5 минут. Аутентификация пользователей реализуется через «Firebase Authentication» с использованием адреса электронной почты и пароля. После входа в систему пользователю предоставляется доступ только к его собственным данным и к открытым данным других пользователей в соответствии с настройками приватности. Весь трафик между приложением и внешними сервисами («Firebase», Kinopoisk API) шифруется по протоколу HTTPS. Пароли пользователей не хранятся на устройстве – аутентификация выполняется через Firebase с использованием токенов. API-ключ Kinopoisk хранится с применением механизмов безопасного хранения Android (NDK или EncryptedSharedPreferences). На уровне базы данных настраиваются правила безопасности «Firebase» (Security Rules), запрещающие чтение и запись данных, к которым пользователь не имеет отношения.

- Сопровождаемость: Структура программного продукта должна обеспечивать возможность внесения изменений, исправления ошибок и расширения функциональности без необходимости полной переработки системы. Сопровождаемость достигается за счет модульной архитектуры (Clean Architecture, MVVM), покрытия ключевых алгоритмов (расчет схожести коллекций) модульными тестами и соблюдения единого стиля кода Kotlin.

- Переносимость: Приложение должно корректно функционировать на различных устройствах под управлением «Android» (API level 26 и выше). Корректное отображение интерфейса на разных экранах обеспечивается использованием ConstraintLayout и альтернативных ресурсов. Установка, обновление и удаление приложения осуществляются стандартными средствами операционной системы «Android».

В итоге сформированные требования к целевому программному продукту определяют функциональные и качественные характеристики разрабатываемой информационной системы и используются в дальнейшем при построении функциональных моделей.

2.2. РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ МОДЕЛЕЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ

Функциональное моделирование является важным этапом проектирования информационной системы, позволяющим наглядно представить процессы ее функционирования, выявить взаимосвязи между отдельными функциями и определить потоки информации и ресурсов. В данной работе для построения функциональных моделей используется методология IDEF0 [12]. Основой для построения моделей послужили требования к целевому программному продукту, сформулированные в разделе 2.1.2.

Согласно методологии IDEF0, модель образует иерархическую систему диаграмм, где каждая диаграмма раскрывает процесс на заданном уровне детализации. Ключевыми элементами служат функциональные блоки, обозначающие именованные процессы или функции, а также интерфейсные дуги (стрелки), представляющие взаимодействия блоков между собой и с внешней средой.

Каждая стрелка в IDEF0 относится к одному из четырех типов:

- Вход – информация или ресурсы, преобразуемые работой в результат. Изображается стрелкой, входящей в левую грань блока.
- Управление – определяются условия, необходимые функции, чтобы произвести правильный выход (правила, ограничения, условия). Изображается стрелкой, входящей в верхнюю грань блока.
- Выход – данные или материальные объекты, произведенные функцией. Изображается стрелкой, исходящей из правой грани блока.
- Механизм – средства, поддерживающие выполнение функций (люди, оборудование, другие программы, внешние системы и т.п.). Изображается стрелкой, входящей в нижнюю грань блока.

Перед началом моделирования были определены цель и точка зрения модели. Цель моделирования – описать процессы функционирования

информационной системы анализа предпочтений пользователей при выборе кинопродукции для последующей разработки и документирования требований. Точка зрения – разработчик информационной системы.

Контекстная диаграмма представляет собой общее описание системы и ее взаимодействия с внешней средой и является вершиной древовидной структуры диаграмм. В данной модели контекстная диаграмма содержит один функциональный блок А0 под названием «Поддержка процессов выбора кинопродукции и взаимодействия пользователей» (рис. 2.1).



Рис. 2.1. Контекстная диаграмма

Входы:

- Поисковые запросы пользователя – текстовые запросы, вводимые пользователем для поиска кинопродукции.
- Учетные данные пользователя - информация (email/пароль), используемая для регистрации и входа в систему.

Управление:

- Политика конфиденциальности и правила обработки данных – нормативные требования, включая 152-ФЗ «О персональных данных», определяющие порядок работы с пользовательской информацией.
- Ограничения Kinopoisk API – лимиты и правила использования внешнего API для получения информации о кинопродукции.
- Правила безопасности «Firebase» – механизмы разграничения доступа к данным и проверки прав пользователя.

Механизмы:

- Мобильное устройство пользователя – техническое средство (смартфон или планшет на базе «Android»), через которое осуществляется взаимодействие пользователя с системой.
- Сервисы «Firebase» – облачная инфраструктура, обеспечивающая аутентификацию, хранение данных и выполнение серверной логики.
- Внешний API Кинопоиска – программный интерфейс доступа к информации о фильмах и сериалах, через который осуществляется получение данных о кинопродукции по поисковым запросам пользователей.

Выходы:

- Сформированные коллекции – структурированные списки фильмов и сериалов, сохраненные пользователем в личном профиле.
- Рейтинг схожести пользователей – числовой показатель процента совпадения коллекций двух пользователей.
- Список пользователей с близкими предпочтениями – перечень профилей, у которых зафиксирован наибольший процент совпадения коллекций.
- Сообщения чата – текстовые сообщения, которыми обмениваются пользователи внутри системы.
- Информация об ошибках – системные сообщения о результатах операций, ошибках авторизации, сбоях запроса и иных событиях.

Диаграмма декомпозиции первого уровня детализирует контекстную диаграмму и показывает основные подпроцессы, из которых состоит функционирование системы, а также взаимосвязи между ними. В соответствии с принципом ограничения сложности, количество функциональных блоков на диаграмме составляет от трех до шести. Функциональный блок А0 (Поддержка процессов выбора кинопродукции и взаимодействия пользователей) декомпозирован на четыре основных блока (рис. 2.2):

- А1 «Управление доступом пользователей» – выполняется регистрация и вход в систему, обрабатываются учетные данные, формируется идентификатор пользователя.
- А2 «Поиск и получение данных о фильмах/сериалах» – осуществляется поиск кинопродукции через внешний API, обрабатываются поисковые запросы пользователя, применяется кэширование для оптимизации обращений.
- А3 «Управление пользовательскими коллекциями» – выполняются операции добавления, удаления и просмотра фильмов в коллекциях «Просмотрено», «В планах» и «Любимое», обеспечивается сохранность и доступность данных.
- А4 «Сравнение коллекций и обеспечение взаимодействия пользователей» – выполняется расчет процента схожести коллекций пользователей, формируется список профилей с близкими вкусами, обеспечивается обмен сообщениями через встроенный чат.

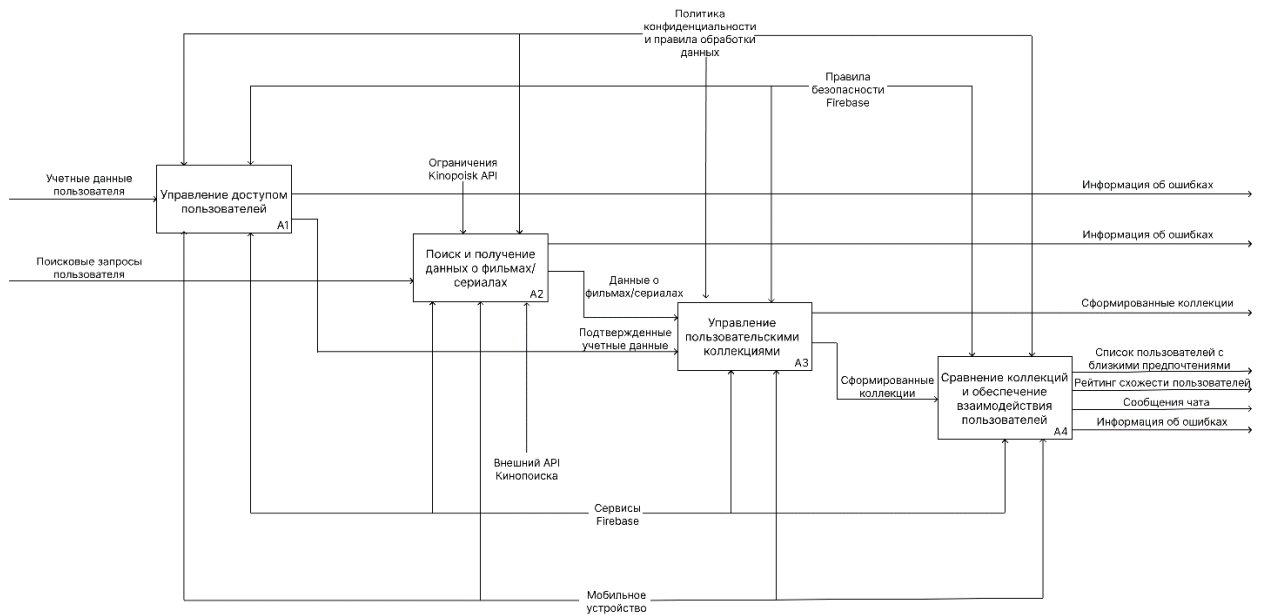


Рис. 2.2. Декомпозиция верхнего уровня

При декомпозиции блока A1 «Управление доступом пользователей» (рис. 2.3) выполняется регистрация и аутентификация пользователей. Результатом являются подтвержденные учетные данные.

Блок A1 включает 3 подпроцесса:

- A11: Регистрация нового пользователя.
- A12: Аутентификация пользователя.
- A13: Формирование идентификатора пользователя.

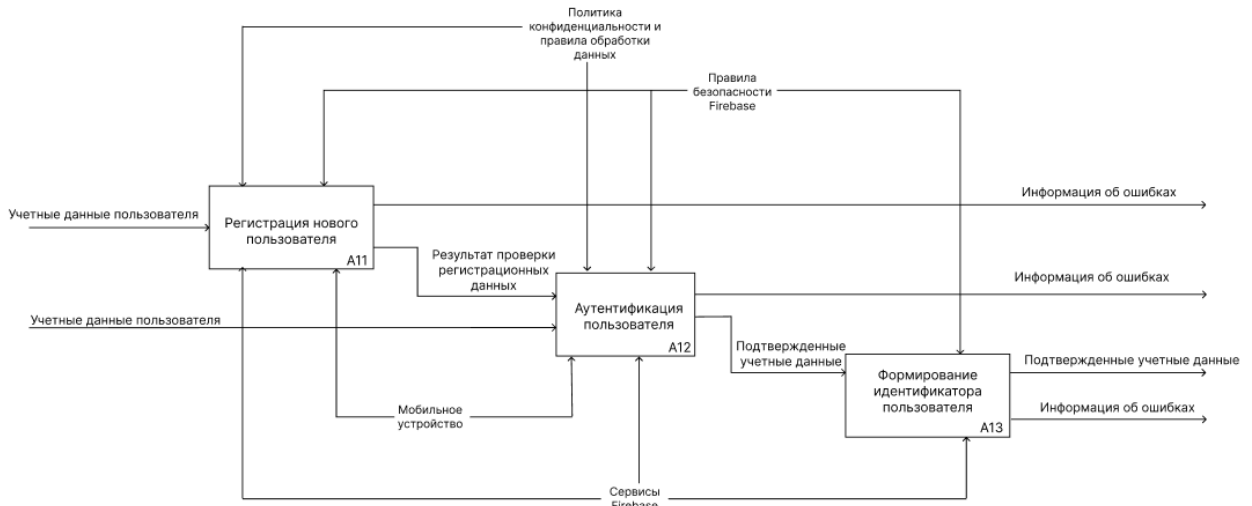


Рис. 2.3. Декомпозиция блока A1 «Управление доступом пользователей»

При декомпозиции блока A2 «Поиск и получение данных о фильмах и сериалах» (рис. 2.4) выполняется обработка поисковых запросов и получение информации из внешнего API. Результатом являются структурированные данные о фильмах.

Блок A2 включает 3 подпроцесса:

- A21: Обработка поискового запроса.
- A22: Взаимодействие с внешним API.
- A23: Кэширование полученных данных.

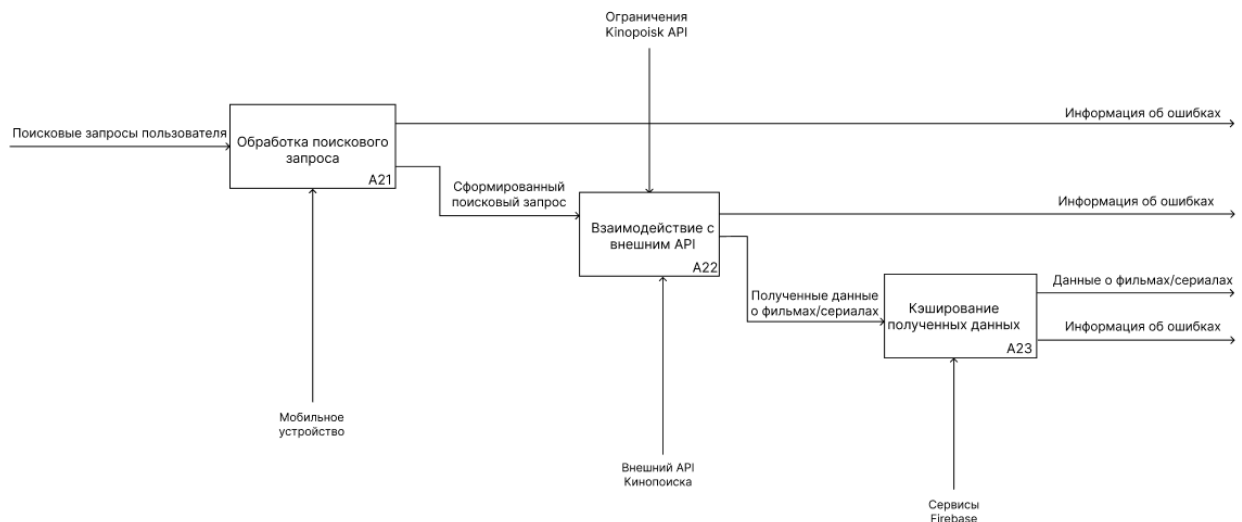


Рис. 2.4. Декомпозиция блока A2 «Поиск и получение данных о фильмах/сериалах»

При декомпозиции блока А3 «Управление пользовательскими коллекциями» (рис. 2.5) выполняются операции по ведению персональных коллекций. Результатом являются сформированные коллекции.

Блок А3 включает 3 подпроцесса:

- А31: Просмотр коллекций.
- А32: Добавление контента в коллекцию.
- А33: Удаление контента из коллекции.

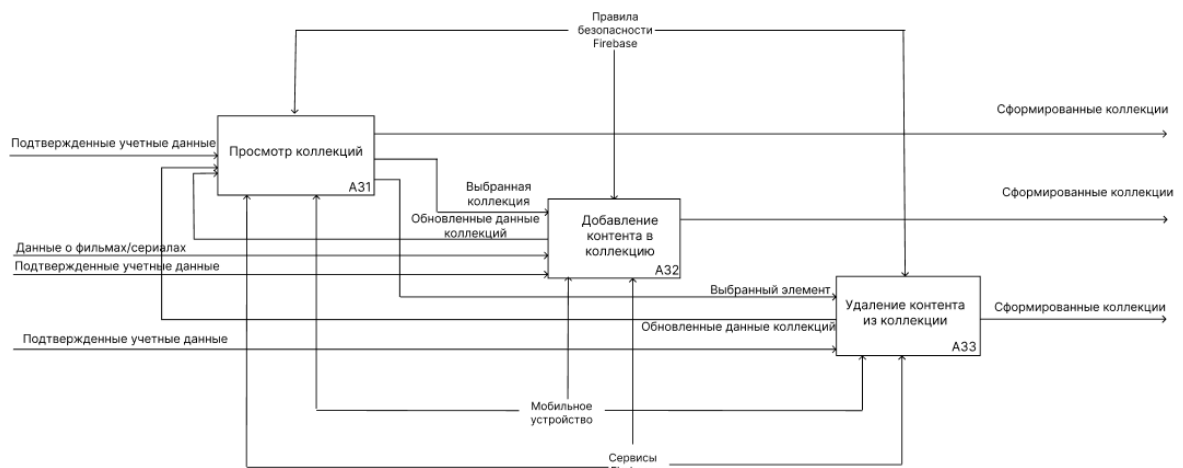


Рис. 2.5. Декомпозиция блока А3 «Управление пользовательскими коллекциями»

При декомпозиции блока А4 «Сравнение коллекций и обеспечение взаимодействия» (рис. 2.6) выполняется расчет процента совпадения коллекций и общение между пользователями. Результатом являются список пользователей с близкими вкусами, рейтинг схожести и сообщения чата.

Блок А4 включает 3 подпроцесса:

- А41: Расчет схожести предпочтений.
- А42: Формирование списка пользователей.
- А43: Обмен сообщениями.

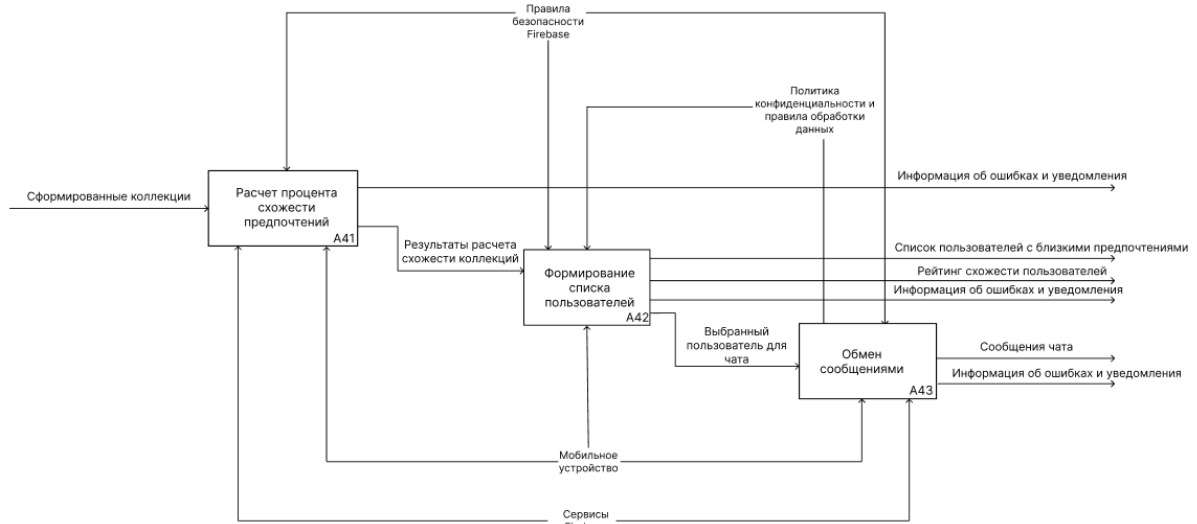


Рис. 2.6. Декомпозиция блока A4 «Сравнение коллекций и обеспечение взаимодействия пользователей»

Построенные функциональные модели в нотации IDEF0 отражают логику работы информационной системы и последовательность обработки данных при взаимодействии пользователя с приложением. На контекстной диаграмме показаны основные внешние сущности и потоки данных, а декомпозиция блоков A1-A4 детализирует процессы регистрации, поиска контента, управления коллекциями и их сравнение. Моделирование позволило зафиксировать требования к входным и выходным данным, определить необходимые механизмы (мобильное устройство, сервисы «Firebase», внешний API) и учесть управляющие ограничения – политику конфиденциальности, правила безопасности и лимиты API. Полученные диаграммы служат основой для разработки архитектуры приложения и последующей реализации программного кода.

ГЛАВА 3. РЕАЛИЗАЦИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

Проектирование информационной системы выполнено на основе требований, сформулированных в главе 2, и включает разработку архитектуры компонентов системы, структуры целевого программного обеспечения и модели целевой базы данных.

Архитектура компонентов системы.

Разрабатываемая информационная система анализа предпочтений пользователей при выборе кинопродукции относится к классу клиент-серверных приложений с использованием облачных сервисов. Архитектура системы включает три основных компонента:

- Клиентская часть (целевое ПО) – мобильное приложение для платформы «Android», реализованное на языке Kotlin с использованием Jetpack Compose для построения пользовательского интерфейса. Приложение обеспечивает все функции взаимодействия с пользователем: аутентификацию, поиск кинопродукции, управление коллекциями, обмен сообщениями, просмотр и редактирование профиля.

- Серверная часть (нецелевое ПО) – реализована на платформе «Firebase» и включает два сервиса: «Firebase Authentication» для управления учетными записями пользователей (регистрация, вход, восстановление пароля) и «Cloud Firestore» – документо-ориентированную NoSQL базу данных для хранения профилей пользователей, коллекций фильмов, чатов и сообщений.

- Внешний источник данных (нецелевые данные) – Kinopoisk.dev API (неофициальный API Кинопоиска), предоставляющий информацию о фильмах и сериалах. Взаимодействие с API осуществляется по протоколу HTTPS с передачей API-ключа. Используется ограничение – не более 200 запросов в сутки.

Архитектура компонентов системы представлена на рисунке 3.1 и

реализована по клиент-серверной модели с использованием облачной платформы «Firebase» и внешнего API кинопродукции.

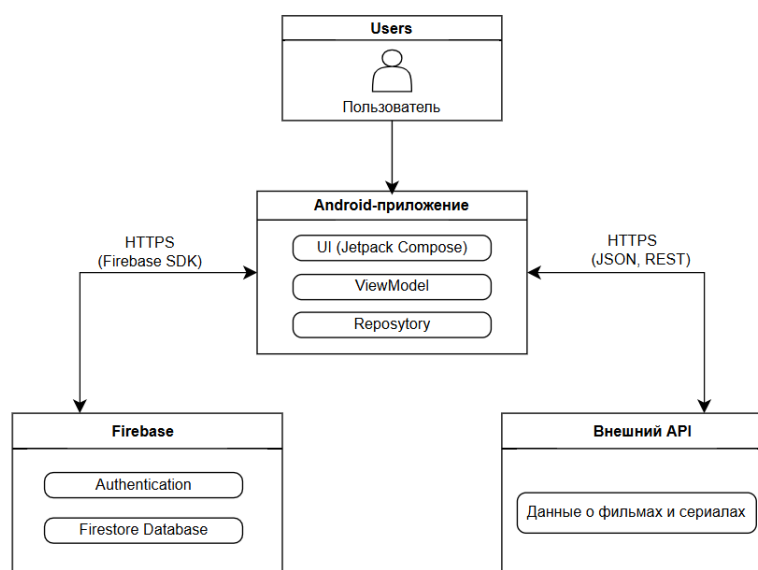


Рисунок 3.1. Архитектура компонентов информационной системы

На верхнем уровне архитектуры выделяется пользователь, взаимодействующий с системой через мобильное приложение.

Клиентская часть реализована в виде Android-приложения, включающего уровень представления (UI), уровень бизнес-логики (ViewModel) и уровень доступа к данным (Repository).

Серверная часть представлена облачными сервисами «Firebase», обеспечивающими аутентификацию пользователей и хранение данных в базе «Firestore».

Внешним источником данных является «Kinopoisk API», предоставляющий информацию о фильмах и сериалах.

Взаимодействие между компонентами осуществляется по протоколу HTTPS с использованием формата JSON.

Структура целевого программного обеспечения.

Целевое ПО (мобильное приложение) построено на основе архитектурного паттерна MVVM (Model-View-ViewModel) с использованием Jetpack Compose для декларативного построения интерфейса. Такая структура

обеспечивает разделение ответственности между компонентами, упрощает тестирование и сопровождение кода.

На верхнем уровне декомпозиции выделяются следующие модули:

1. Модуль аутентификации – реализован в `AuthViewModel.kt` и `AuthScreen.kt`. Отвечает за регистрацию новых пользователей, вход по email и паролю, выход из системы, а также проверку уникальности username. Взаимодействует с «Firebase Authentication» для создания и проверки учетных записей. Поддерживается переключение между режимами «Вход» и «Регистрация» на одном экране.

2. Модуль навигации – реализован в `NavGraph.kt` и `MainScreen.kt`. `MainScreen.kt` выступает контейнером с нижней панелью навигации (`Bottom Navigation Bar`), обеспечивающей переключение между тремя основными разделами: «Главная», «Чаты», «Люди». Навигация между экранами внутри каждого раздела осуществляется через `NavController`.

3. Модуль поиска кинопродукции – реализован в `HomeScreen.kt` и `MovieRepository.kt`. Обеспечивает поиск фильмов и сериалов по названию через `Kinopoisk.dev API` с автоматическим выполнением запроса при вводе текста. Отображает результаты в виде списка с постерами, названиями, годом выпуска и рейтингом. Для загрузки постеров используется библиотека `Coil`.

4. Модуль управления коллекциями – реализован в `CollectionsViewModel.kt` и вкладках на `HomeScreen.kt`. Поддерживает три типа коллекций: «В планах», «Просмотрено», «Любимое». Для коллекции «Просмотрено» предусмотрена возможность установки пользовательского рейтинга (от 1 до 5) и текстового комментария. Добавление фильма в коллекцию выполняется через кнопки на карточке фильма. Удаление из коллекции – по долгому нажатию с подтверждением через диалоговое окно. Данные хранятся в `Firestore` в коллекциях `plan_to_watch`, `watched` и `favorites`.

5. Модуль профиля – реализован в `ProfileScreen.kt` и `ProfileViewModel.kt`. Отображает информацию о пользователе: `username`, `email`, настройки приватности. Поддерживает редактирование имени

пользователя, смену уникального идентификатора (userId) и включение/отключение приватного режима профиля. Приватный профиль скрывает коллекции пользователя от других.

6. Модуль обмена сообщениями – реализован в ChatsViewModel.kt, ChatsListScreen.kt и ChatScreen.kt. Обеспечивает следующие функции: создание нового чата по userId другого пользователя; отображение списка чатов с последним сообщением, временем и счетчиком непрочитанных сообщений; отправка и получение сообщений в реальном времени; статусы сообщений (одна галочка – доставлено, две – прочитано); группировка сообщений по датам; счетчик непрочитанных сообщений отображается как в списке чатов, так и на иконке «Чаты» в нижней навигации.

7. Модуль подбора пользователей – реализован в PeopleScreen.kt и PeopleViewModel.kt. Обеспечивает формирование списка пользователей с близкими вкусами на основе сравнения коллекции «Любимое». Расчет процента совпадения выполняется по формуле: $\text{процент} = (\text{количество общих фильмов} * 100) / \text{максимальное количество фильмов в одной из коллекций}$. В список попадают пользователи с совпадением 50% и выше. Реализован поиск по уникальному идентификатору userId.

8. Общий модуль – содержит модели данных (User.kt, Chat.kt, Message.kt, Movie.kt), репозиторий для работы с API (MovieRepository.kt) и вспомогательные утилиты.

На нижнем уровне декомпозиции модули разбиваются на следующие компоненты в соответствии с паттерном MVVM:

- Model – модели данных и репозитории. MovieRepository.kt инкапсулирует логику обращения к «Kinopoisk API». Данные из «Firestore» загружаются непосредственно во ViewModel через Firebase SDK.

- ViewModel – AuthViewModel.kt, CollectionsViewModel.kt, ChatsViewModel.kt, ProfileViewModel.kt, PeopleViewModel.kt. Содержат бизнес-логику, управляют состоянием интерфейса (StateFlow/MutableStateFlow) и взаимодействуют с Firebase SDK.

- View – composable-функции в AuthScreen.kt, HomeScreen.kt, ChatsListScreen.kt, ChatScreen.kt, ProfileScreen.kt, PeopleScreen.kt. Отображают интерфейс и реагируют на изменения состояния в ViewModel.

Проектирование целевой базы данных.

Концептуальное проектирование заключается в формализованном описании предметной области с помощью которого можно проанализировать корректность созданной схемы. Целью такого проектирования является определение содержания базы данных анализа предпочтений пользователей при выборе кинопродукции. Определение возможных хранимых данных происходит после анализа задач и объектов предметной области. Анализ задач осуществляется с помощью создания диаграммы вариантов использования на унифицированном языке моделирования (UML).

Диаграмма вариантов использования (ДВИ) нужна для формализации функциональных требований к системе через действующих лиц и варианты использования. Суть данной диаграммы состоит в следующем: проектируемая система представляется в виде множества действующих лиц, взаимодействующих с системой через варианты использования. Действующим лицом называется любой субъект или внешняя система, взаимодействующая с разрабатываемой системой извне. Вариантом использования является спецификация функций, которые система предоставляет действующему лицу. Каждый вариант использования определяет набор действий, совершаемых системой при взаимодействии с действующим лицом, без детализации способа реализации этих действий.

В данной предметной области концептуальное проектирование начинается с анализа работы мобильного приложения для анализа предпочтений пользователей и поиска единомышленников по кинематографическим вкусам. Пользователь может авторизоваться в приложении через ввод email и пароля, либо зарегистрироваться при отсутствии учетной записи. После входа на главном экране доступны вкладки «В планах», «Просмотрено», «Любимое» с поисковой строкой для нахождения

фильмов и сериалов, которые берутся из внешнего источника данных – «API Кинопоиска». Результаты поиска позволяют добавлять контент в коллекции с возможностью установки рейтинга и комментария для коллекции «Просмотрено». Через нижние вкладки пользователь переходит в чаты для общения, где доступны функции отправки сообщений со статусами доставки/прочтения (галочки), блокировки собеседника и push-уведомления о новых сообщениях. Третий экран показывает предложенных пользователей, отсортированных по проценту схожести коллекций «Любимое», с поиском по уникальному пользовательскому ID. Сравнение предпочтений происходит автоматически по пересечениям в коллекции «Любимое». Личный кабинет доступен из любой вкладки по нажатию на имя пользователя и отображает его информацию. При просмотре профиля другого пользователя (из чатов или рекомендаций) можно увидеть коллекции, если профиль не приватный; из чужого «Любимого» фильтруются непросмотренные фильмы/сериалы для добавления в «В планах» через чекбоксы. «API Кинопоиска» обеспечивает получение данных о кинопродукции по поисковым запросам.

С проектируемой системой взаимодействуют следующие действующие лица:

- Пользователь.
- «API Кинопоиска».

Рассмотрим варианты использования каждого из действующих лиц.

Пользователю доступны следующие функции:

- Авторизация/регистрация в приложении.
- Поиск кинопродукции.
- Просмотр результатов поиска.
- Добавление фильма/сериала в коллекции «Просмотрено», «В планах» или «Любимое».
- Удаление фильма/сериала из коллекции.
- Установка рейтинга и комментариев для коллекции «Просмотрено».

- Управление личным профилем.
- Переход в чаты.
- Отправка/получение сообщений.
- Поиск других пользователей по ID.
- Просмотр предложенных пользователей.
- Просмотр профиля/коллекций другого пользователя.
- Добавление из коллекции «Любимое» другого пользователя в свою коллекцию «В планах».
- Блокировка собеседника в чате.

«API Кинопоиска» доступны следующие функции:

- Предоставить данные о кинопродукции по поисковому запросу.

Исходя из возможных вариантов использования, диаграмма вариантов использования имеет следующий вид (рис. 3.2).

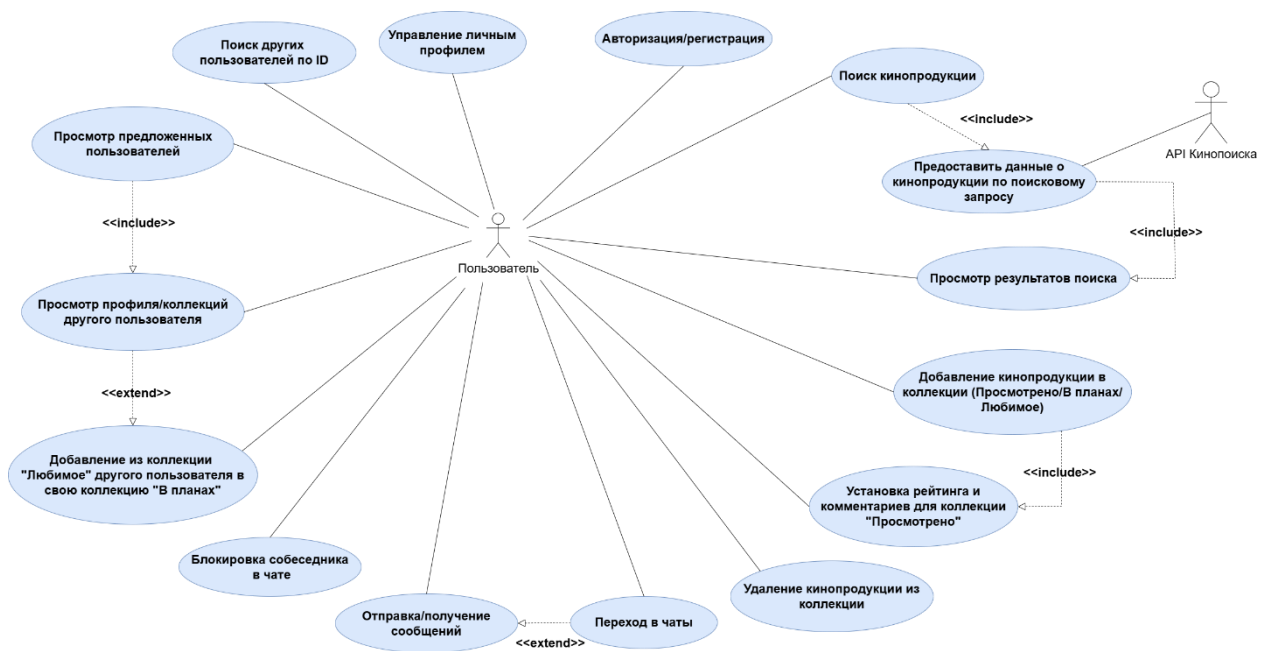


Рис. 3.2. Диаграмма вариантов использования

В первом приближении для решения выделенных задач необходимо хранение данных о следующих объектах:

- Пользователь.
- Кинопродукция.

- Коллекция.
- Чат.
- Сообщение.

Необходимые данные можно классифицировать по частоте их изменения:

- условно-постоянные данные (пользователь, кинопродукция);
- данные, которые оперативно обновляются при каждом решении задачи (сообщения в чатах, статусы, коллекции).

На этапе концептуального проектирования выделены следующие сущности предметной области:

- Пользователь (идентификатор, имя пользователя, электронная почта, уникальный код пользователя, признак приватности, дата создания).
- Кинопродукция (идентификатор, название, адрес постера, год выпуска, рейтинг, жанр, оценка пользователя, комментарий пользователя, дата добавления).
- Коллекция (тип: «В планах», «Просмотрено», «Любимое»).
- Чат (идентификатор, участники, последнее сообщение, время последнего сообщения, статус набора текста участниками).
- Сообщение (идентификатор, идентификатор отправителя, текст, временная метка, признак доставки, признак прочтения).

Для формализации структуры данных и взаимосвязей между сущностями предметной области была разработана диаграмма классов, представленная на рисунке 3.3.

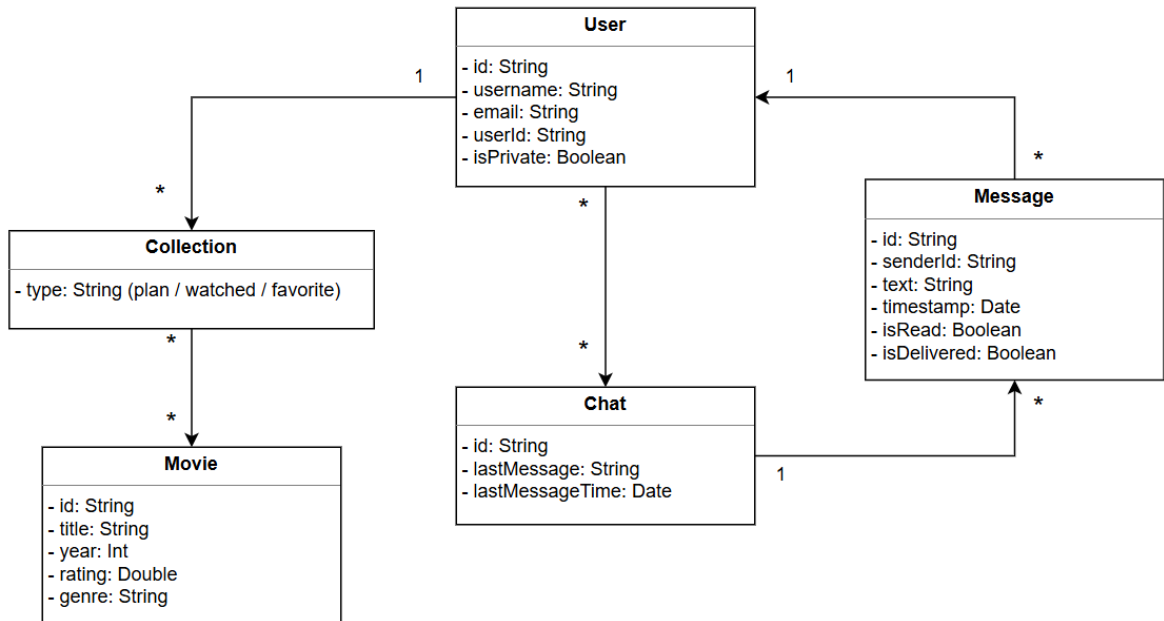


Рис. 3.3. Диаграмма классов

Диаграмма классов отражает основные сущности системы анализа предпочтений пользователей при выборе кинопродукции, а также связи между ними. В качестве ключевых классов выделены: пользователь (User), кинопродукция (Movie), коллекция (Collection), чат (Chat) и сообщение (Message).

Класс User содержит данные о пользователе, включая идентификатор, имя пользователя, адрес электронной почты, уникальный пользовательский идентификатор и признак приватности профиля. Пользователь может иметь несколько коллекций, что отражает связь «один ко многим» между классами User и Collection.

Класс Collection описывает пользовательские коллекции фильмов и сериалов (например, «В планах», «Просмотрено», «Любимое») и связан с классом Movie отношением «многие ко многим», поскольку одна коллекция может содержать множество фильмов, а один фильм может находиться в коллекциях разных пользователей.

Класс Movie содержит информацию о кинопродукции, включая название, год выпуска, рейтинг и жанр.

Для реализации взаимодействия между пользователями используются классы Chat и Message. Класс Chat описывает диалог между пользователями и связан с классом Message отношением «один ко многим», так как один чат содержит множество сообщений. Класс Message хранит текст сообщения, временную метку, а также признаки доставки и прочтения. Каждое сообщение связано с одним пользователем-отправителем, что отражает связь «многие к одному» между классами Message и User.

Разработанная диаграмма классов позволяет наглядно представить структуру данных системы и служит основой для реализации базы данных и логики взаимодействия между компонентами приложения.

Для хранения пользовательских данных используется «Cloud Firestore» – документо-ориентированная NoSQL база данных. В отличие от реляционных баз, данные в «Firestore» организованы в виде коллекций документов, каждый из которых содержит набор полей в формате ключ-значение.

На этапе логического проектирования определена структура коллекций и документов в «Firestore»:

Коллекция users – содержит документы пользователей. Идентификатором документа является уникальный uid, генерируемый «Firebase Authentication». Поля документа:

- id (string) – идентификатор пользователя (дублируется для удобства);
- username (string) – отображаемое имя пользователя;
- email (string) – адрес электронной почты;
- userId (string) – уникальный идентификатор для поиска других пользователей;
- isPrivate (boolean) – признак приватности профиля;
- createdAt (timestamp) – дата регистрации.

Подколлекции внутри документа пользователя:

- plan_to_watch – содержит документы фильмов/сериалов, добавленных в коллекцию «В планах»;

- watched – содержит документы фильмов/сериалов, добавленных в коллекцию «Просмотрено» (включает поля userRating и userComment);
- favorites – содержит документы фильмов/сериалов, добавленных в коллекцию «Любимое».

Каждый документ фильма содержит поля: movieId (int) – идентификатор фильма из «Kinopoisk API»; title (string) – название; posterUrl (string) – URL постера; year (int) – год выпуска; rating (double) – рейтинг из API; genres (string) – жанры; addedAt (timestamp) – дата добавления; для коллекции «Просмотрено» дополнительно: userRating (int) – оценка пользователя; userComment (string) – комментарий.

Коллекция chats – содержит документы чатов. Идентификатор документа генерируется автоматически. Поля документа:

- participants (array of strings) – список участников чата (два userId);
- lastMessage (string) – текст последнего сообщения;
- lastMessageTime (timestamp) – время последнего сообщения;
- participantTyping (map) – статус набора текста для каждого участника.

Подколлекция messages – содержит документы сообщений внутри каждого чата. Поля документа:

- senderId (string) – идентификатор отправителя;
- text (string) – текст сообщения;
- timestamp (timestamp) – время отправки;
- isDelivered (boolean) – статус доставки;
- isRead (boolean) – статус прочтения.

На этапе физического проектирования созданы необходимые индексы «Firestore» для оптимизации запросов:

- Индекс для коллекции chats по полю participants с сортировкой по lastMessageTime – для быстрой загрузки чатов текущего пользователя в хронологическом порядке.

- Индекс для подколлекции messages по полям isRead и senderId – для подсчета непрочитанных сообщений.

Правила безопасности «Firestore» (Security Rules) настроены следующим образом:

- Аутентификация обязательна для всех запросов.
- Пользователи могут читать и записывать только свои документы в коллекции users и подколлекциях plan_to_watch, watched, favorites.
- В коллекции chats пользователи могут читать только те чаты, в которых они участвуют.
- В подколлекции messages пользователи могут читать и отправлять сообщения только в чатах, где они являются участниками.

В итоге спроектированная архитектура компонентов системы, структура целевого ПО и модель целевой базы данных полностью соответствуют требованиям, сформулированным в главе 2, и являются основой для последующего программирования и тестирования.

3.2. ПРОГРАММИРОВАНИЕ И ТЕСТИРОВАНИЕ КОМПОНЕНТ ПО И/ИЛИ ФУНКЦИЙ И ИНТЕРЕЙСА БД

3.2.1. ОБОСНОВАНИЕ ВЫБОРА СРЕДСТВ ПРОГРАММНОЙ РЕАЛИЗАЦИИ СИСТЕМЫ И/ИЛИ БД

Выбор средств программной реализации выполнялся на основе требований, предъявляемых к мобильному приложению, а также анализа альтернативных технологий. Основными критериями сравнения являлись производительность, безопасность, поддержка асинхронных операций, объем шаблонного кода, наличие актуальной документации и степень интеграции с облачными сервисами.

Мобильное приложение разрабатывается в среде Android Studio Panda 2. Android Studio – это официальная среда разработки для платформы Android, предоставляемая компанией Google [13]. Она включает в себя все

необходимое для создания, сборки, тестирования и отладки приложений: редактор кода с подсветкой синтаксиса и автодополнением, встроенный эмулятор устройств Android, систему сборки Gradle, инструменты профилирования производительности и отладчик с точками останова. При использовании обычного текстового редактора пришлось бы вручную настраивать сборку проекта, запускать компиляцию из командной строки и использовать отдельные инструменты для отладки, что значительно замедлило бы разработку. Также Android Studio имеет встроенную поддержку Jetpack Compose: визуальная проверка интерфейса обновляется в реальном времени при изменении кода, а инструменты отладки показывают дерево компоновки. Без этого пришлось бы каждый раз собирать и запускать приложение на устройстве или эмуляторе, чтобы увидеть изменения интерфейса. С учетом того, что разрабатываемое приложение использует современный стек технологий (Kotlin, Jetpack Compose, Firebase), Android Studio является наиболее удобным выбором.

В качестве основного языка разработки клиентской части выбран Kotlin. Основной альтернативой является Java – традиционный язык для платформы Android. Ключевое преимущество Kotlin заключается во встроенной защите от пустых указателей за счет явного разделения типов, которые могут принимать пустое значение, и тех, которые не могут. Это позволяет выявлять потенциальные ошибки на этапе компиляции, тогда как в Java ошибка, связанная с обращением к пустому указателю, возникает только во время выполнения программы, что увеличивает риск сбоев. Кроме того, Kotlin поддерживает сопрограммы – легкие асинхронные компоненты, которые позволяют писать код в последовательном стиле без сложных цепочек вложенных вызовов. В Java для аналогичной функциональности потребовалось бы использовать тяжеловесные потоки или дополнительную библиотеку, что усложняет разработку и отладку. Согласно официальной документации Google, начиная с 2019 года компания объявила о стратегии «Kotlin first», то есть Kotlin стал рекомендуемым языком для разработки под

Android. Еще одно преимущество – лаконичность: классы-хранилища данных, умные приведения и функции-расширения сокращают объем шаблонного кода по сравнению с Java. Поскольку разрабатываемая система активно использует асинхронные вызовы к Firestore, внешнему API и требует автоматического обновления интерфейса, сопрограммы и безопасная работа с типами, допускающими пустое значение, являются критически важными [14].

Для построения пользовательского интерфейса выбран декларативный фреймворк Jetpack Compose. Альтернативой является традиционная система на основе разметки в формате XML и компонентах View, где изменения интерфейса приходится прописывать вручную. Основное отличие Compose заключается в том, что интерфейс описывается непосредственно на языке Kotlin в виде функций, которые автоматически перерисовываются при изменении состояния. В классическом подходе требуется вручную обновлять элементы интерфейса через адаптеры, что приводит к увеличению объема связующего кода и повышает вероятность ошибок, связанных с несогласованностью данных. Compose также позволяет повторно использовать компоненты проще, чем собственные View. В разрабатываемом приложении – результаты поиска кинопродукции, три типа коллекций, список чатов, список пользователей с близкими вкусами. Compose обеспечивает автоматическое обновление только изменившихся элементов, что упрощает реализацию и повышает предсказуемость поведения [15].

В качестве серверной части выбрана платформа Firebase [16]. Альтернативой является разработка собственного серверного приложения с использованием реляционной базы данных. Главное преимущество «Firebase» – предоставление готовых сервисов аутентификации («Firebase Authentication») и документно-ориентированной базы данных с синхронизацией в реальном времени («Cloud Firestore») [17,18]. «Firestore» поддерживает механизм отслеживания изменений, который автоматически уведомляет клиентское приложение об изменениях в коллекциях и сообщениях, что критически важно для чатов и обновления списков

кинопродукции. При создании собственного сервера аналогичную функциональность пришлось бы реализовывать через веб-сокеты, что увеличило бы сроки разработки. Firebase также обеспечивает автоматическое сохранение данных на устройстве и масштабирование без участия разработчика. На начальном этапе доступен бесплатный тарифный план, полностью покрывающий потребности тестирования и демонстрации. Правила Firebase настраиваются посредством описания желаемого результата и позволяют гибко ограничивать доступ к профилям и коллекциям, что соответствует требованиям Федерального закона №152-ФЗ «О персональных данных».

Для доступа к информации о фильмах и сериалах выбран Kinopoisk.dev API – неофициальный, но документированный интерфейс с передачей данных в формате JSON [19]. Данный API предоставляет все необходимые поля – название, постер, год выпуска, рейтинг, жанры и уникальный идентификатор фильма, а бесплатный тариф с ограничением 200 запросов в сутки является достаточным для тестирования и демонстрации разработанной системы.

Для выполнения сетевых запросов к внешнему интерфейсу используется библиотека Retrofit, которая упрощает преобразование ответов в формате JSON в объектах языка Kotlin и поддерживает сопрограммы [20]. Для загрузки постеров фильмов применяется библиотека Coli, интегрирующаяся с Jetpack Compose и обеспечивающая асинхронную загрузку, сохранение в кэш и автоматическую отмену запросов при повторном использовании элементов списка [21]. Обе библиотеки имеют открытый исходный код и широко используются в разработке Android приложений.

3.2.2. ПРОГРАММИРОВАНИЕ, ТЕСТИРОВАНИЕ И ИНТЕГРАЦИЯ ЦЕЛЕВОГО ПО И/ИЛИ ФУНКЦИЙ И ИНТЕРФЕЙСА БД СИСТЕМЫ

Процесс программирования включал в себя реализацию всех целевых функций, описанных в требованиях к программному продукту. Разработка велась в среде Android Studio на языке Kotlin с использованием фреймворка

Jetpack Compose для построения пользовательского интерфейса. Проект организован по модульному принципу: модели данных вынесены в отдельный пакет `models`, классы `ViewModel`, содержащие бизнес-логику, в пакет `viewmodel`, экраны приложения – в пакет `screens`. Такой подход обеспечивает разделение ответственности между компонентами, что упрощает тестирование и последующее сопровождение кода.

Программирование целевых функций выполнялось в следующей последовательности. Первым был реализован модуль аутентификации, включающий экран входа и регистрации (`AuthScreen`) и соответствующий класс `AuthViewModel`. Для управления учетными записями использовался «Firebase Authentication». При регистрации нового пользователя дополнительно выполнялась проверка уникальности выбранного имени пользователя через отдельный запрос к «Firestore», так как «Firebase Authentication» не предоставляет такой возможности автоматически.

Далее был разработан модуль поиска кинопродукции. Для взаимодействия с внешним API Кинопоиска был создан класс `MovieRepository`, который через библиотеку `Retrofit` отправляет HTTP-запросы и преобразует ответы в формате JSON в объекты Kotlin. Поиск выполняется автоматически при вводе текста в строку поиска – для этого используются сопрограммы Kotlin, которые позволяют отменять предыдущий запрос при вводе нового символа, чтобы не создавать лишнюю нагрузку на API.

Модуль управления коллекциями реализован в `CollectionsViewModel`. Данные из «Cloud Firestore» загружаются асинхронно с помощью `snapshotListener`, что обеспечивает автоматическое обновление интерфейса при изменении содержимого коллекций на сервере. Добавление и удаление фильмов реализовано через вызов методов добавления и удаления документов в соответствующие подколлекции: `plan_to_watch`, `watched` и `favorites`. Для коллекции «Просмотрено» дополнительно реализована возможность установки пользовательской оценки от 1 до 5 и текстового комментария.

Модуль подбора пользователей с близкими предпочтениями реализован в `PeopleViewModel`. Расчет процента совпадения выполняется по формуле: $\text{процент} = (\text{количество общих фильмов в коллекции «Любимое»} * 100) / \text{максимальное количество фильмов в коллекции одного из двух пользователей}$. Поскольку «Firestore» не поддерживает выполнение подобных вычислений на стороне сервера, вся логика расчета реализована на стороне клиента: загружается коллекция «Любимое» текущего пользователя и всех остальных пользователей, после чего вычисляется пересечение и процент совпадения. Для оптимизации производительности загрузка данных выполняется асинхронно с использованием сопрограмм, а результат фильтруется так, что в итоговый список попадают только пользователи с совпадением 50 процентов и выше.

Модуль обмена сообщениями реализован в `ChatsViewModel` и включает экраны списка чатов и самого чата. При открытии экрана чата устанавливается `snapshotListener` на подколлекцию `messages`, благодаря чему новые сообщения появляются в интерфейсе без необходимости обновлять экран вручную. Статусы сообщений (доставлено, прочитано) реализованы следующим образом: при отправке сообщение сохраняется в «Firestore» с полем `isDelivered = false` и `isRead = false`. Когда сообщение попадает на устройство получателя, оно автоматически помечается как доставленное. Когда получатель открывает чат, вызывается функция `markMessagesAsRead`, которая обновляет статус `isRead` на `true` для всех неп прочитанных сообщений, отправленных текущему пользователю. Счетчик неп прочитанных сообщений отображается как в списке чатов, так и на иконке «Чаты» в нижней панели навигации.

Тестирование проводилось по двум направлениям: модульное тестирование ключевых алгоритмов и функциональное тестирование пользовательских сценариев на физических устройствах.

Модульное тестирование выполнялось с использованием фреймворка `JUnit`. Проверялась корректность работы функции расчета процента совпадения коллекций на различных наборах входных данных. Например, при

пустой коллекции у одного из пользователей результат должен быть равен нулю; при полном совпадении фильмов – 100 процентов; при частичном совпадении – рассчитано по указанной формуле. Тестирование показало, что во всех проверенных случаях функция возвращает ожидаемый результат.

Функциональное тестирование проводилось на двух физических устройствах под управлением «Android 10» и «Android 13». Проверены следующие пользовательские сценарии:

- регистрация нового пользователя и вход зарегистрированного пользователя;
- поиск фильмов и сериалов по названию;
- добавление кинопродукции в коллекции «В планах», «Просмотрено» и «Любимое»;
- установка оценки и комментария для просмотренных фильмов;
- удаление кинопродукции из коллекций по долгому нажатию с подтверждением;
- создание чата с другим пользователем по его уникальному коду;
- отправка и получение сообщений в реальном времени;
- проверка работы статусов сообщений (доставлено, прочитано);
- просмотр коллекций других пользователей (если профиль не приватный);
- переключение приватности профиля и проверка ограничения доступа;
- поиск пользователей по уникальному коду и отображение списка рекомендованных пользователей, отсортированного по проценту совпадения.

Интеграция всех модулей выполнена в едином объекте навигации NavGraph, где определены маршруты для всех экранов приложения. Нижняя панель навигации (Bottom Navigation Bar) реализована в MainScreen, который выступает контейнером для трех основных разделов: главный экран с коллекциями и поиском, экран чатов и экран поиска людей. Такая организация

обеспечивает единую точку управления навигацией и сохраняет нижнюю панель при переключении между вкладками.

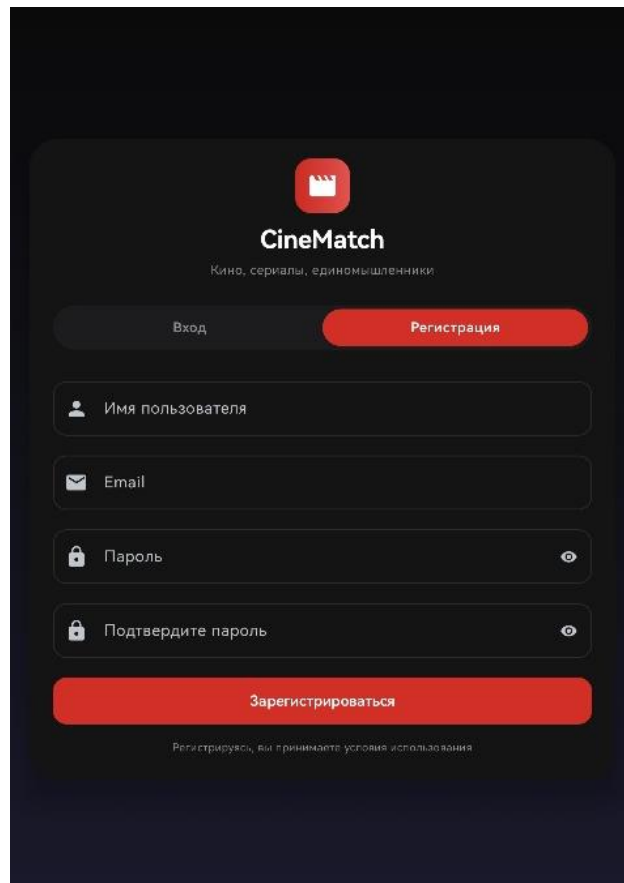
По завершении отладки и интеграции было проведено итоговое тестирование системы в целом. Проверялась работа всех целевых функций в условиях, приближенных к реальным: одновременная работа двух пользователей (на двух устройствах) с отправкой сообщений, добавлением фильмов в коллекции и проверкой отображения списка рекомендованных пользователей. Во время тестирования не было выявлено критических ошибок, влияющих на работоспособность приложения. Время расчета списка пользователей с близкими вкусами для ста профилей не превысило установленного в требованиях значения в две секунды, а время отклика интерфейса при навигации по коллекциям составило менее одной секунды.

3.3. ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ СИСТЕМЫ

Разработанное мобильное приложение «MARK&MATCH» представляет собой информационную систему для анализа предпочтений пользователей при выборе кинопродукции. Система обеспечивает возможность поиска фильмов и сериалов, ведения пользовательских коллекций, поиска людей со схожими вкусами и обмена сообщениями.

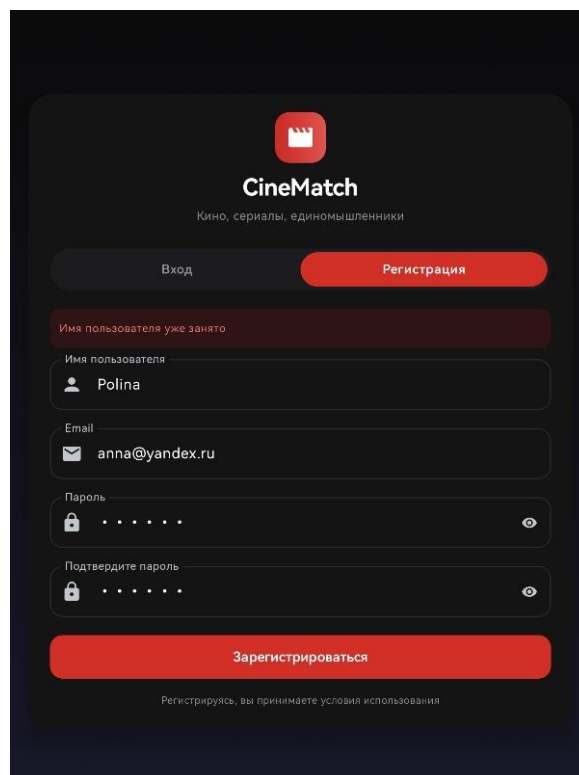
При первом запуске приложения пользователь попадает на экран аутентификации. На данном экране реализовано переключение между режимами «Вход» и «Регистрация».

На экране регистрации необходимо указать имя пользователя, адрес электронной почты и пароль (рис. 3.4). Система проверяет уникальность имени пользователя: если указанное имя уже занято, отображается соответствующее сообщение об ошибке (рис. 3.5). Пароль должен содержать не менее 6 символов, при меньшем количестве символов выводится соответствующее предупреждение (рис. 3.6).



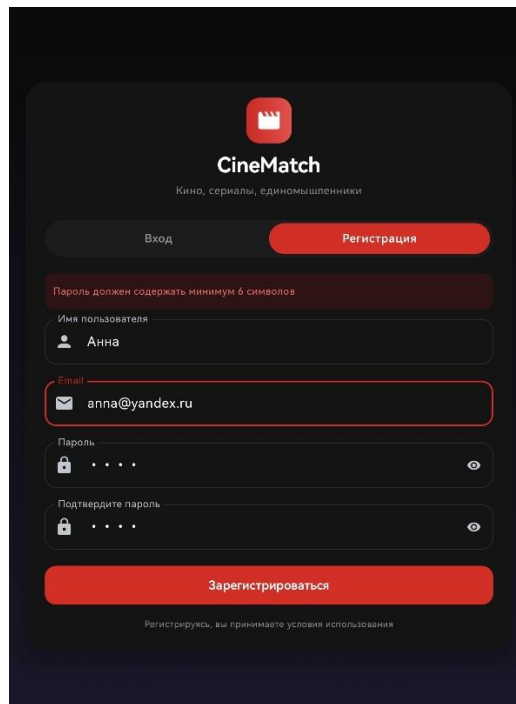
The image shows the registration screen of the CineMatch application. At the top, there is a red icon of a clapperboard and the text "CineMatch" with the tagline "Кино, сериалы, единомышленники." Below this, there are two buttons: "Вход" (Login) and "Регистрация" (Registration). The registration form consists of four input fields: "Имя пользователя" (Username), "Email", "Пароль" (Password), and "Подтвердите пароль" (Confirm Password). Each field has a corresponding icon (person, envelope, and padlocks respectively). There are also eye icons to toggle password visibility. Below the form is a red button labeled "Зарегистрироваться" (Register). At the bottom, there is a small text link: "Регистрируясь, вы принимаете условия использования" (By registering, you accept the terms of use).

Рис. 3.4. Экран регистрации



The image shows the registration screen of the CineMatch application with an error message. The layout is identical to the previous screen, but the "Имя пользователя" (Username) field now contains the text "Polina". Above this field, a red error message is displayed: "Имя пользователя уже занято" (Username is already taken). The "Email" field contains "anna@yandex.ru", and the "Пароль" (Password) and "Подтвердите пароль" (Confirm Password) fields are filled with dots. The "Зарегистрироваться" (Register) button is still visible at the bottom, along with the terms of use link.

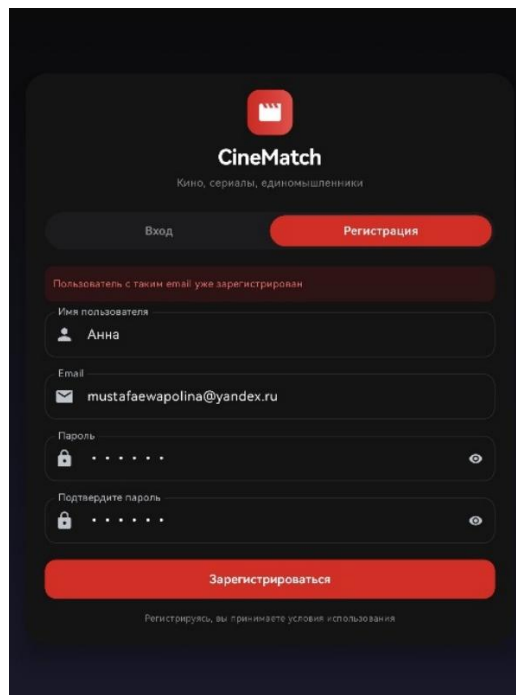
Рис. 3.5. Сообщение об уже занятом имени



The image shows the CineMatch registration interface. At the top is the CineMatch logo with the tagline "Кино, сериалы, единомышленники". Below the logo are two buttons: "Вход" (Login) and "Регистрация" (Registration). The registration form includes a red error message at the top: "Пароль должен содержать минимум 6 символов". The form fields are: "Имя пользователя" (Username) with the value "Анна", "Email" with the value "anna@yandex.ru", "Пароль" (Password) with masked characters, and "Подтвердите пароль" (Confirm password) with masked characters. A red "Зарегистрироваться" (Register) button is at the bottom, followed by a small link: "Регистрируясь, вы принимаете условия использования".

Рис. 3.6. Сообщение о длине пароля

Если при регистрации пользователь указывает электронную почту, с которой уже был вход в систему, выводится сообщение о том, что пользователь с таким email уже зарегистрирован (рис. 3.7).



The image shows the CineMatch registration interface with a different error message. The red error message at the top reads: "Пользователь с таким email уже зарегистрирован". The form fields are: "Имя пользователя" (Username) with the value "Анна", "Email" with the value "mustafaewapolina@yandex.ru", "Пароль" (Password) with masked characters, and "Подтвердите пароль" (Confirm password) with masked characters. A red "Зарегистрироваться" (Register) button is at the bottom, followed by a small link: "Регистрируясь, вы принимаете условия использования".

Рис. 3.7. Сообщение «Пользователь с таким email уже зарегистрирован»

Для входа в приложение необходимо указать email и пароль, зарегистрированные ранее (рис. 3.8). При успешной аутентификации пользователь перенаправляется на главный экран приложения.

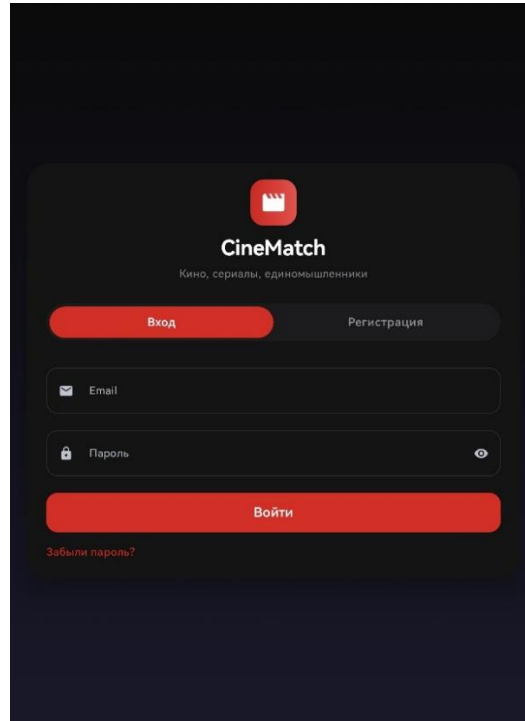


Рис. 3.8. Экран входа

При неверном вводе email или пароля выводится предупреждение «Неверный email или пароль» (рис. 3.9).

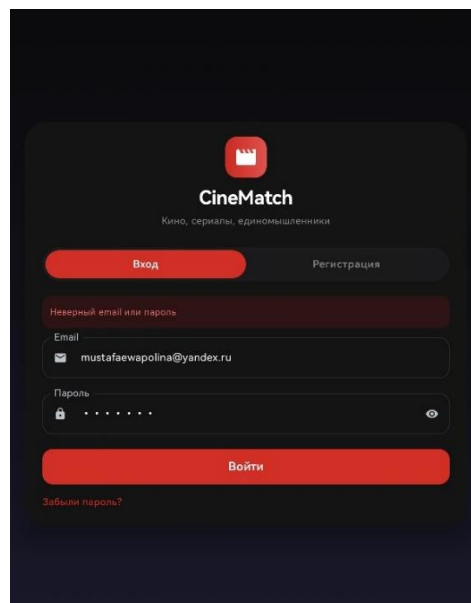


Рис. 3.9. Сообщение «Неверный email или пароль»

На экране аутентификации также предусмотрена функция восстановления пароль. При нажатии на ссылку «Забыли пароль?» открывается диалоговое окно, в котором пользователь может ввести свой email, с помощью которого осуществлялась регистрация (рис. 3.10). После отправки запроса «Firebase Authentication» отправляет письмо со ссылкой для сброса пароль (рис. 3.11, 3.12).

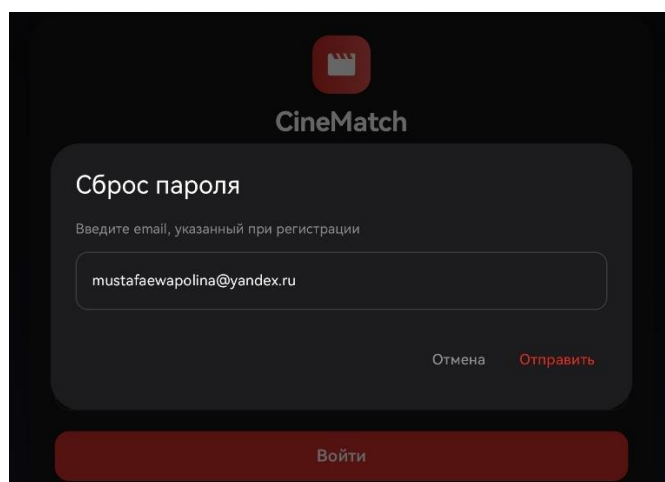


Рис. 3.10. Диалоговое окно сброса пароля

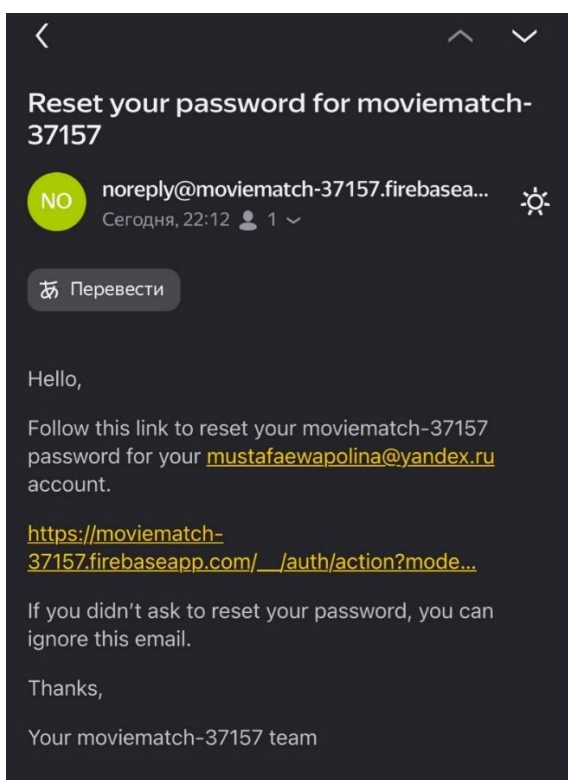


Рис. 3.11. Сообщение на электронную почту о сбросе пароля

Сбросьте свой пароль

для **mustafaewapolina@yandex.ru**

Новый пароль



СОХРАНИТЬ

Рис. 3.12. Сброс пароля

После успешного входа пользователь попадает на главный экран приложения (рис. 3.13). В верхней панели отображается имя пользователя и аватар с первой буквой имени. Нажатие на имя или аватар открывает экран профиля.

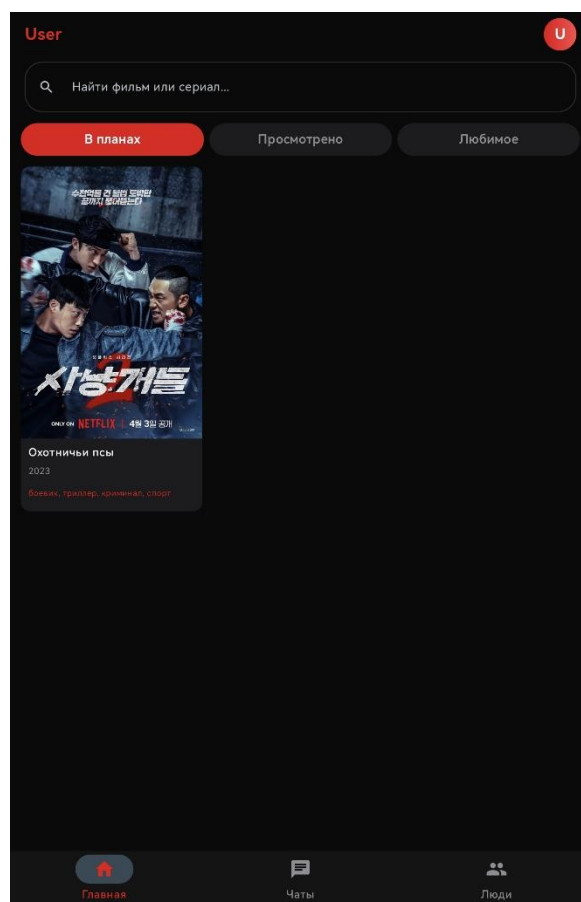


Рис. 3.13. Главный экран приложения

Нижняя панель навигации (рис. 3.14) обеспечивает переключение между тремя основными разделами приложения:

- «Главная» – управление коллекциями фильмов.
- «Чаты» – список диалогов и обмен сообщениями.
- «Люди» – поиск пользователей со схожими вкусами.

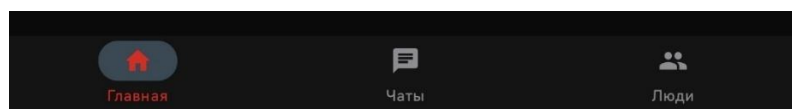


Рис. 3.14. Нижняя панель навигации

На главном экране сверху расположена поисковая строка. При вводе текста автоматически выполняется поиск фильмов и сериалов через внешний API. Запрос отправляется при каждом изменении текста в поле поиска, что обеспечивает мгновенное отображение результатов.

Результаты поиска отображаются в виде горизонтальных карточек (рис. 3.15). каждая карточка содержит постер, название, год выпуска, рейтинг и три кнопки для добавления в коллекции «В планах», «Просмотрено» и «Любимое».

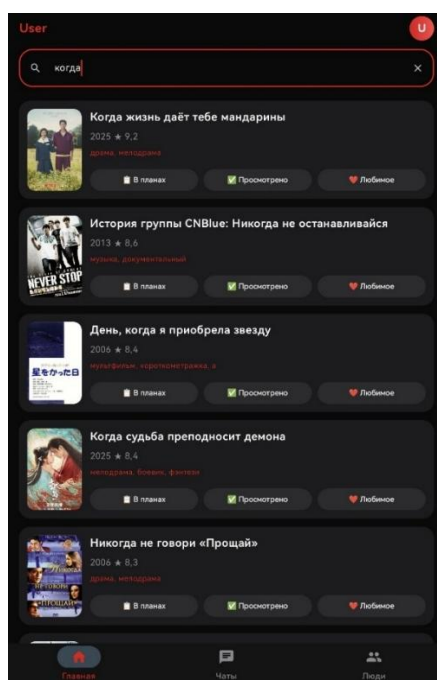


Рис. 3.15. Результаты поиска

Пользователь может добавлять фильмы в три типа коллекций: «В планах», «Просмотрено», «Любимое». Переключение между коллекциями осуществляется с помощью вкладок, расположенных под поисковой строкой.

При добавлении фильма в коллекцию «Просмотрено» открывается диалоговое окно, в котором пользователь может оценить фильм по шкале от 1 до 5 звезд и оставить текстовый комментарий, которые будут видны при просмотре его профиля другим человеком (рис. 3.16).

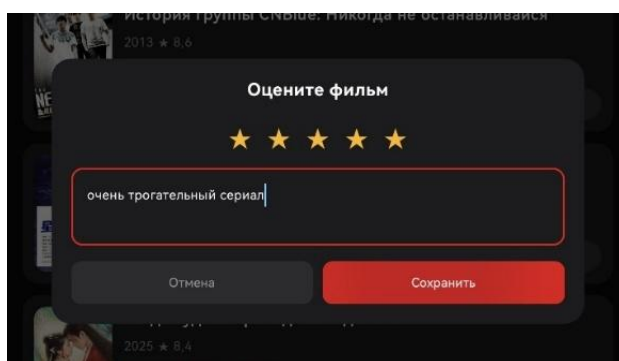


Рис. 3.16. Диалоговое окно оценки кинопродукции

Фильмы в коллекциях отображаются в виде сетки (рис. 3.17). При долгом нажатии на карточку фильма открывается контекстное меню, позволяющее добавить фильм в другую коллекцию или удалить из текущей (рис. 3.18).

При добавлении фильма в коллекцию «Просмотрено» из коллекции «В планах» фильм автоматически удаляется из исходной коллекции.

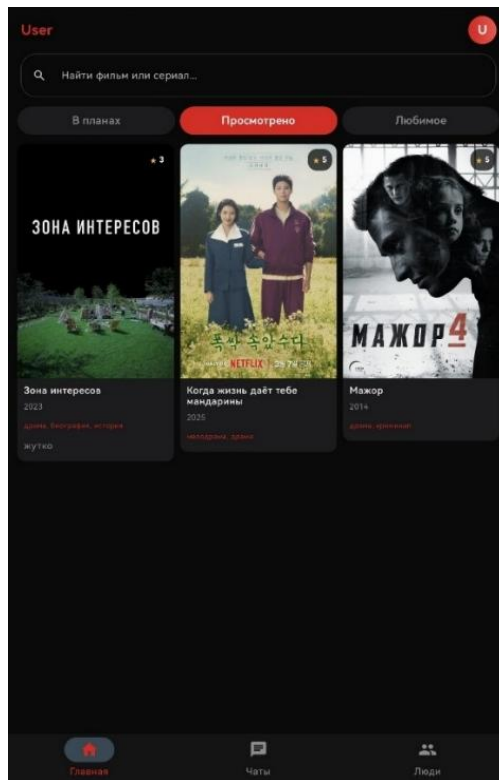


Рис. 3.17. Сетка кинопродукции коллекции

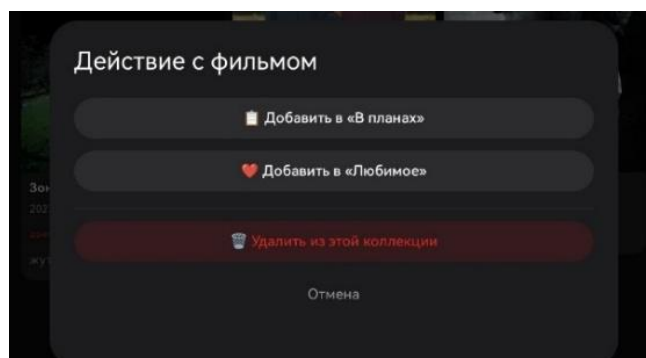


Рис. 3.18. Контекстное меню

Экран профиля открывается при нажатии на имя пользователя или аватар в верхней панели главного экрана над поисковой строкой (рис. 3.19).

Данный экран содержит:

- аватар (первая буква имени);
- имя пользователя;
- уникальный идентификатор (userID) с возможностью его редактирования;
- описание (bio), добавляемое пользователем;

- статистику количества кинопродукции в каждой коллекции;
- настройки приватности.

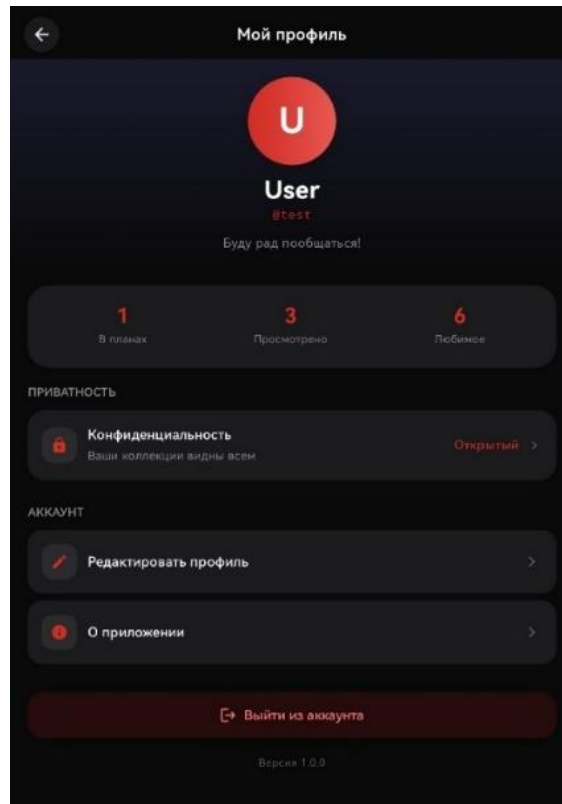


Рис. 3.19. Экран профиля

При нажатии на кнопку «Редактировать профиль» открывается диалоговое окно, в котором пользователь может изменить имя и описание (рис. 3.20).

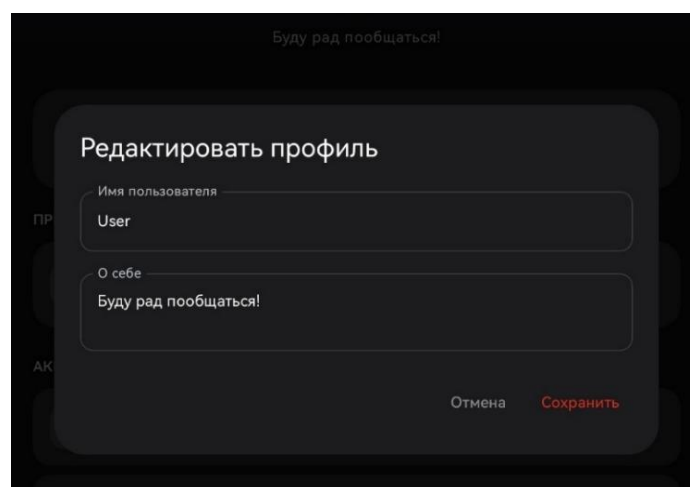


Рис. 3.20. Редактирование профиля

Изменение уникального идентификатора осуществляется через отдельное диалоговое окно (рис. 3.21). Система проверяет уникальность нового идентификатора и, если указанный ID уже занят другим пользователем, отображается сообщение об ошибке (рис. 3.22).

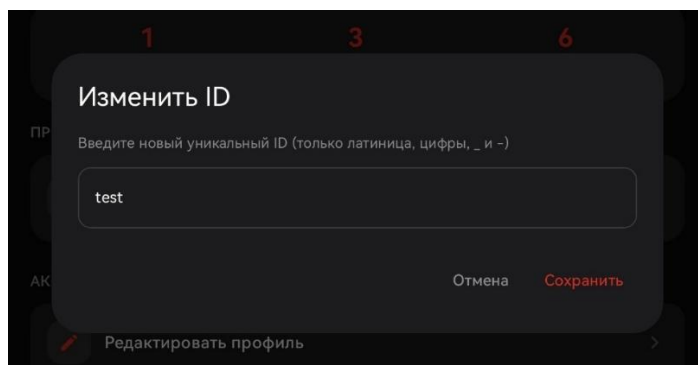


Рис. 3.21. Изменение уникального идентификатора

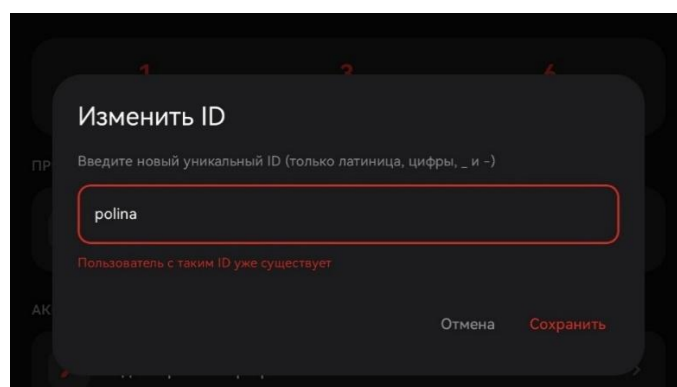


Рис. 3.22. Сообщение об ошибке «Пользователь с таким ID уже существует»

Настройка приватности реализована в виде карточки с выбором режима (рис. 3.23). При нажатии на карточку открывается модальное окно, позволяющее выбрать между открытым и приватным профилем. При включении приватного режима коллекции пользователя становятся недоступны для просмотра другими пользователями, а кнопка «Написать сообщение» на странице профиля становится неактивной.

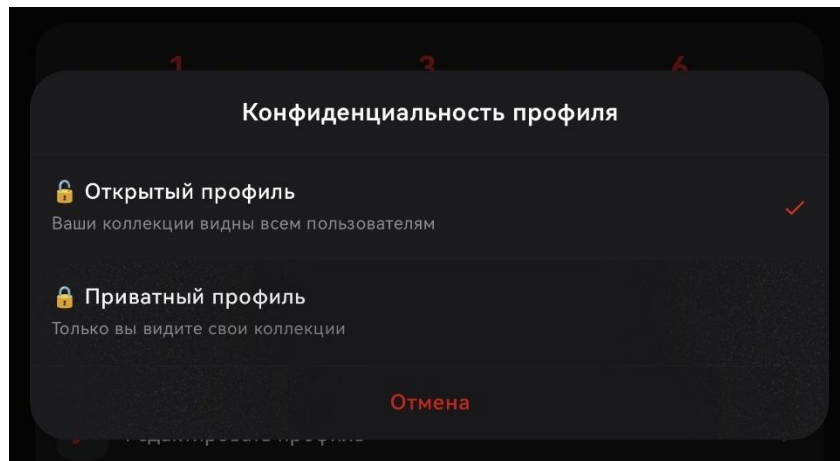


Рис. 3.23. Настройка приватности

При переходе на профиль другого пользователя (например, из раздела «Люди») отображается экран, содержащий:

- аватар, имя, описание и идентификатор пользователя;
- кнопку «Написать» сообщение»;
- статистику коллекций.

Если профиль является приватным, коллекции не отображаются (рис. 3.24). Вместо них показывается сообщение «Профиль приватный. Пользователь скрыл свои коллекции». Кнопка «написать сообщение» становится неактивной, отображается подпись «У пользователя приватный профиль, вы не можете писать ему сообщения».

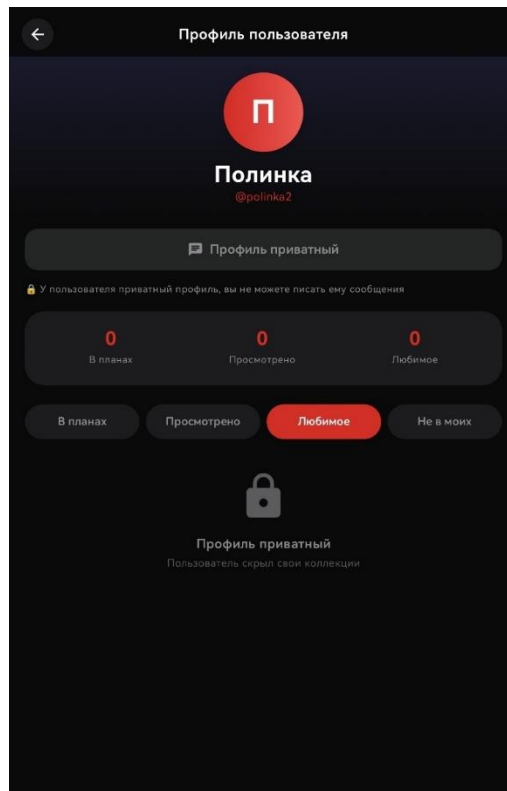


Рис. 3.24. Приватный профиль просматриваемого пользователя

Если профиль открыт, пользователь может просматривать коллекции другого пользователя (рис. 3.25-3.27). В коллекции «Любимое» кинопродукция разделена на следующие категории:

- фильмы/сериалы, которых нет у текущего пользователя (доступны для добавления, отображаются с чекбоксами);
- фильмы/сериалы, уже находящиеся у текущего пользователя в коллекции «Любимое» (отмечены как «Уже в вашем любимом»);
- фильмы/сериалы, уже находящиеся в планах у текущего пользователя (отмечены как «Уже в ваших планах»);
- фильмы/сериалы, уже просмотренные текущим пользователем (отмечены как «Уже просмотрено»).

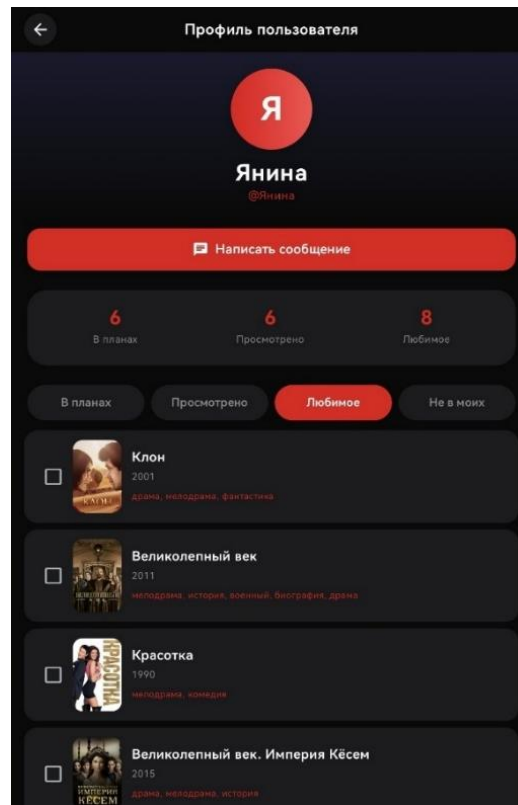


Рис. 3.25. Открытый профиль просматриваемого пользователя

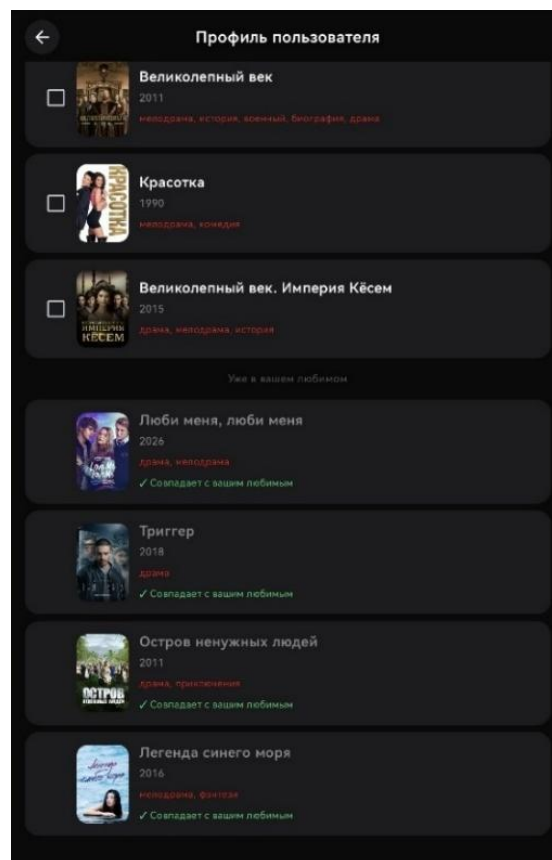


Рис. 3.26. Кинопродукция, уже имеющаяся в моих коллекциях

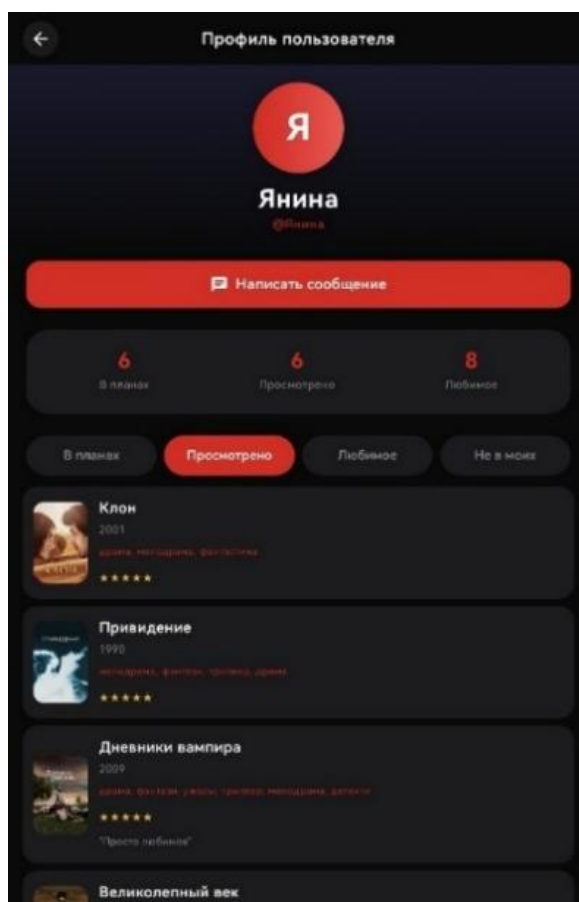


Рис. 3.27. Просмотр коллекции «Просмотрено» другого пользователя

Для добавления кинопродукции из чужой коллекции «Любимое» пользователь может отметить нужные фильмы/сериалы с помощью чекбоксов и нажать кнопку «Добавить выбранное в «В планах»» (рис. 3.28). После добавления выбранные фильмы исключаются из списка и больше не предлагаются для добавления.

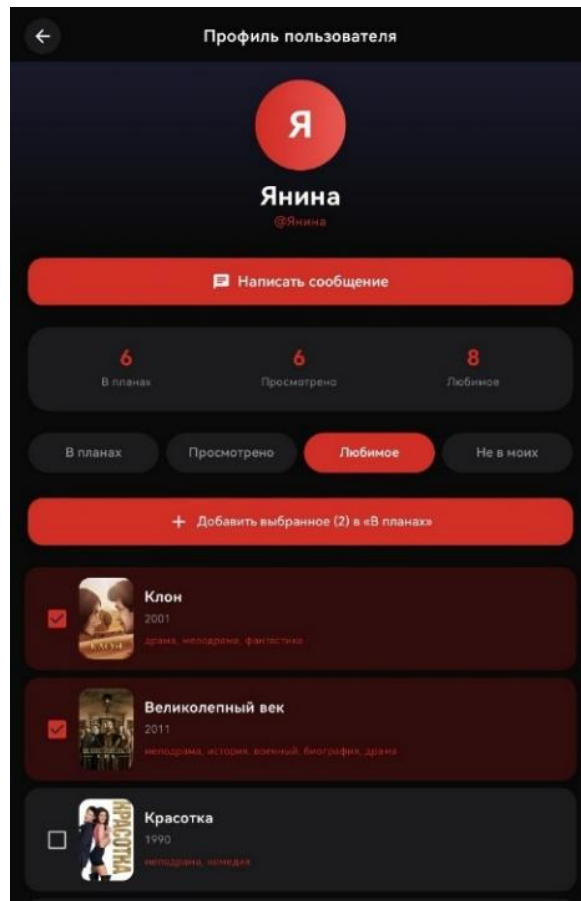


Рис. 3.28. Добавление выбранной кинопродукции в свою коллекцию «В планах»

Также в просмотре коллекций другого пользователя есть коллекция «Не в моих» (рис. 3.29). Данная вкладка содержит кинопродукцию, которая отсутствует во всех трех коллекциях текущего пользователя – то есть фильмы и сериалы, которые пользователь еще не добавил ни в «В планах», ни в «Просмотрено», ни в «Любимое».

Для удобства массового добавления каждый элемент коллекции имеет чекбокс. Пользователь может отметить несколько фильмов или сериалов, после чего в верхней части экрана появляется кнопка «Добавить выбранное в «В планах»». При нажатии на кнопку все выбранные позиции автоматически добавляются в личную коллекцию «В планах» текущего пользователя. После добавления, эти фильмы или сериалы исчезают из данной коллекции.

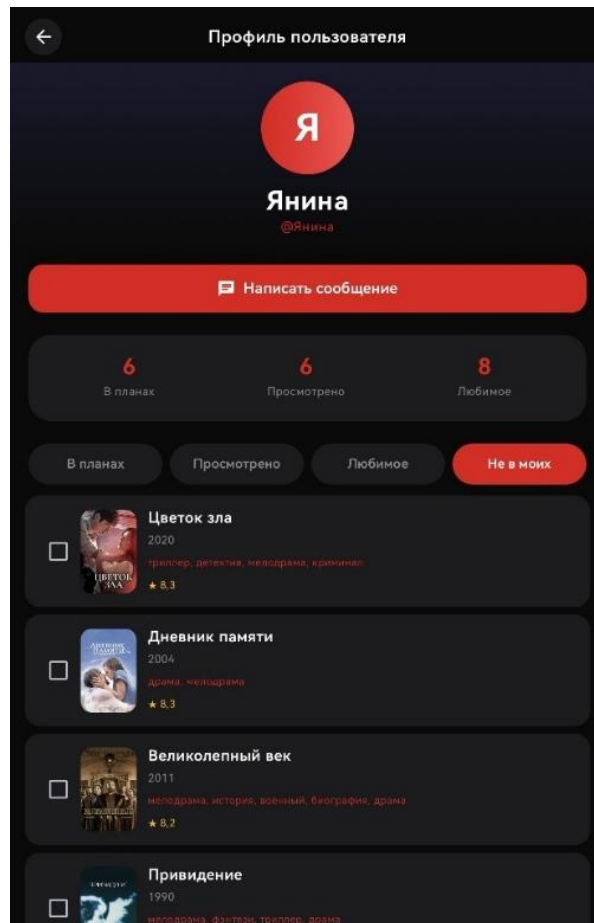


Рис. 3.29. Коллекция «Не в моих»

Раздел «Люди» предназначен для поиска пользователей с похожими вкусами. Алгоритм расчета схожести основан на сравнении коллекции «Любимое» текущего пользователя с коллекциями «Любимое» других пользователей.

В список попадают только пользователи с совпадением 50% и выше (рис. 3.30). Результаты сортируются по убыванию процента совпадения. Карточка пользователя содержит имя, идентификатор, процент совпадения и список общих фильмов/сериалов (первые три).

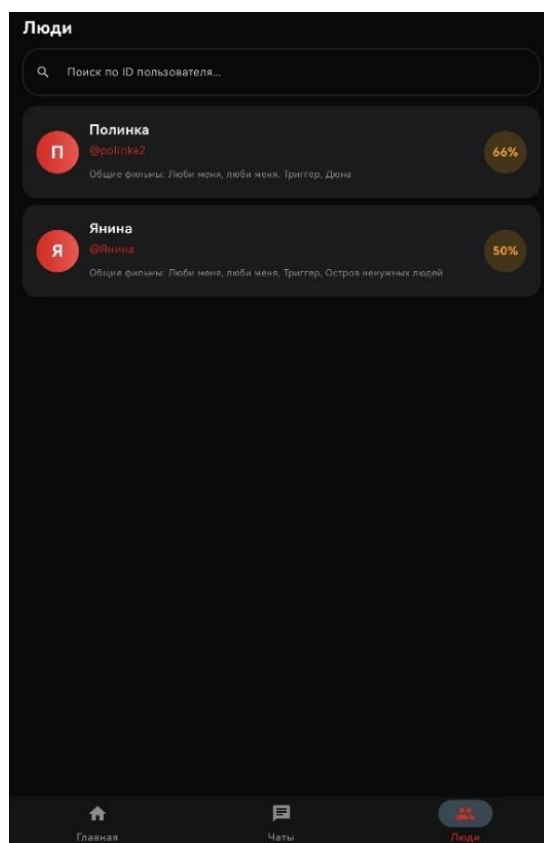


Рис. 3.30. Список по схожести коллекций

Реализован поиск пользователей по идентификатору userID (рис. 3.31). Поиск выполняется автоматически при вводе текста в поисковую строку.

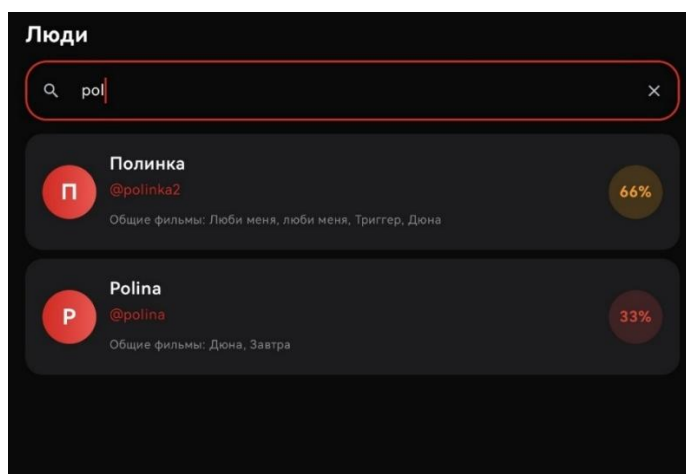


Рис. 3.31. Поиск пользователя по ID

Раздел «Чаты» отображает список всех диалогов пользователя (рис. 3.32). Каждый элемент списка содержит аватар собеседника, его имя (или заданное пользователем отображаемое только для него имя), последнее

сообщение, время отправки и счетчик непрочитанных сообщений. Красный кружок с числом непрочитанных сообщений также отображается на иконке «Чаты» в нижней панели навигации (рис. 3.33).

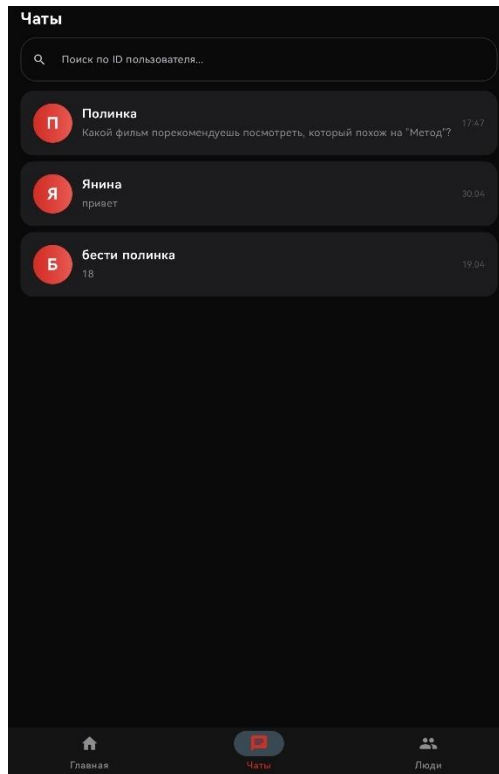


Рис. 3.32. Раздел «Чаты»

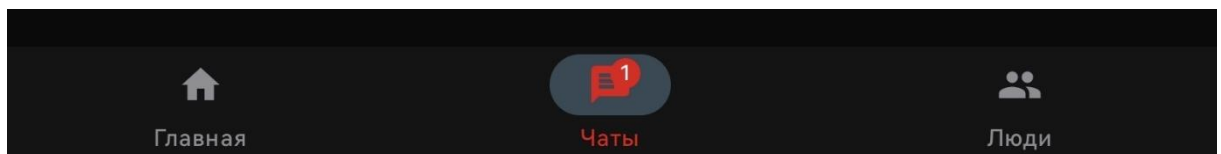


Рис. 3.33. Счетчик непрочитанных сообщений в нижней панели навигации

Поиск пользователя по ID позволяет начать новый чат. При вводе ID выполняется поиск пользователей, содержащих введенную подстроку. При нажатии на найденного пользователя создается новый чат (если его не еще не существует) и выполняется переход в экран переписки.

Экран переписки (рис. 3.34) содержит верхнюю панель с аватаром, именем собеседника, значком редактирования имени собеседника и кнопкой блокировки, список сообщений, сгруппированных по датам («Сегодня», «Вчера», полная дата), и поле ввода сообщения.

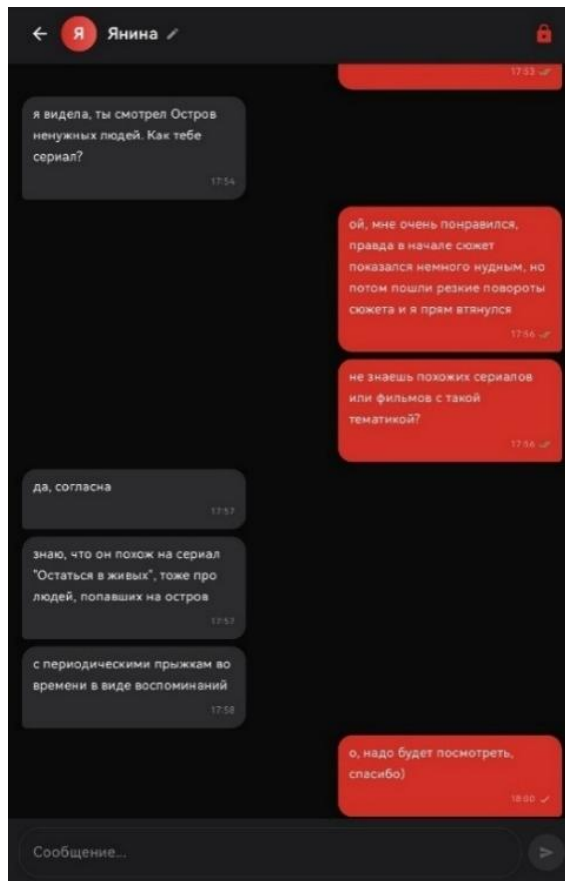


Рис. 3.34. Экран переписки

При отправке сообщения появляется его статус: одна галочка – сообщение доставлено, две галочки – сообщение прочитано (рис. 3.35).

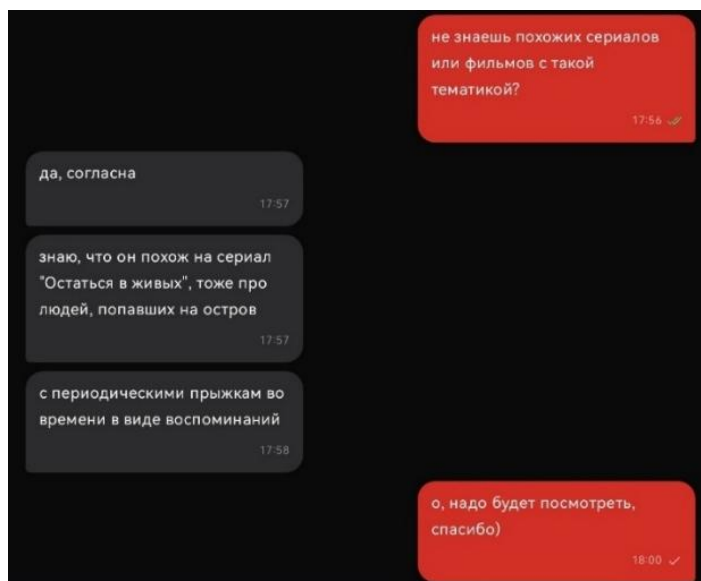


Рис. 3.35. Статусы сообщений

Реализовано долгое нажатие на своем сообщении для удаления (рис. 3.36). Кнопка блокировки позволяет заблокировать собеседника (рис. 3.37). При блокировке:

- поле ввода сообщения скрывается, отображается надпись «Вы заблокировали этого пользователя» (рис. 3.38);
- в списке чатов рядом с именем заблокированного пользователя появляется значок замка, имя становится серым (рис. 3.39);
- если пользователь заблокирован, он не может отправлять сообщения и у него появляется сообщение «Пользователь вас заблокировал» (рис. 3.40).

Разблокировка осуществляется повторным нажатием на кнопку блокировки.

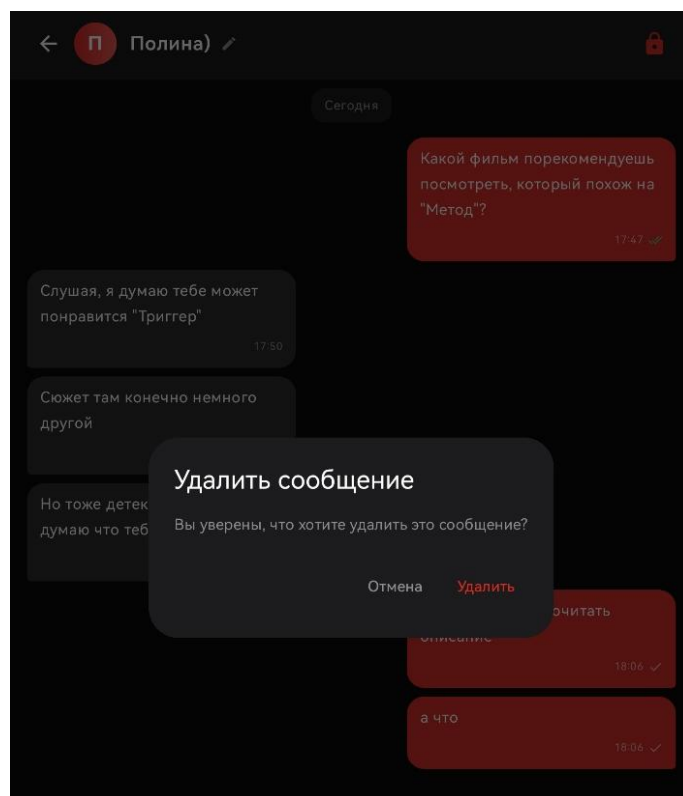


Рис. 3.36. Удаление сообщения



Рис. 3.37. Кнопка блокировки собеседника

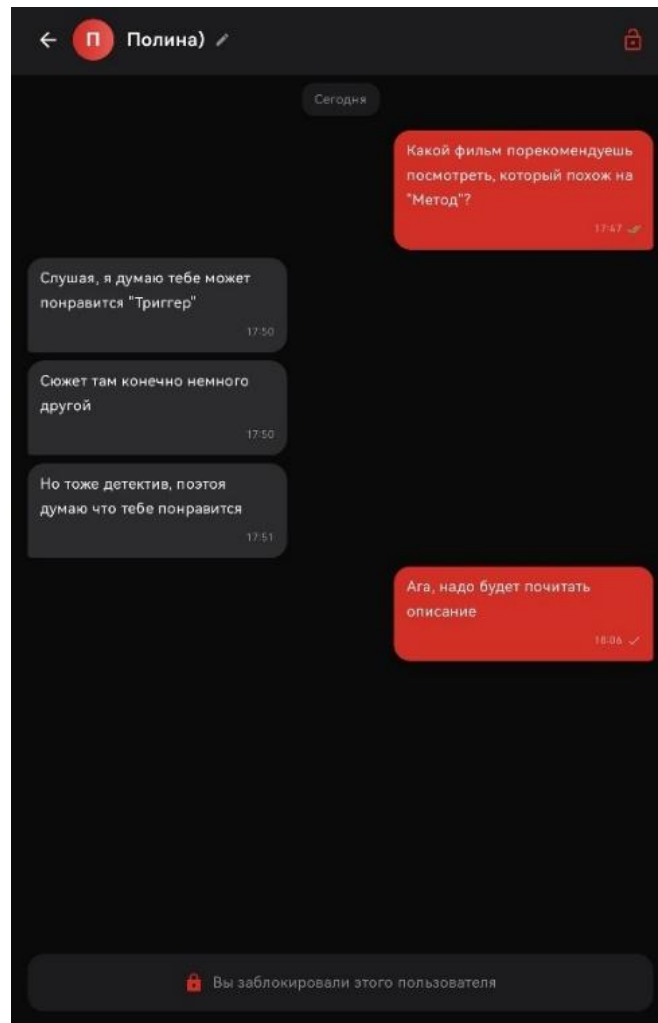


Рис. 3.38. Блокировка пользователя

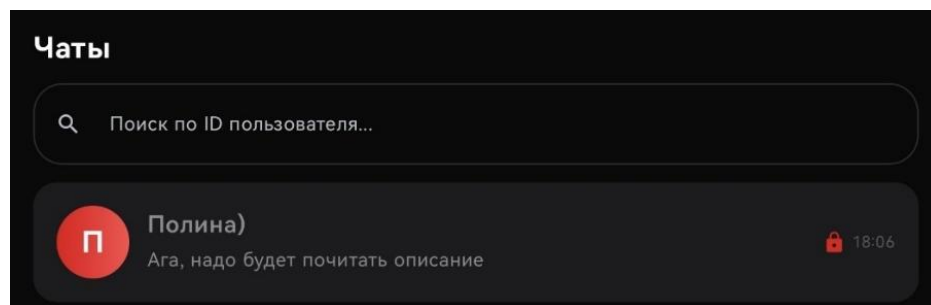


Рис. 3.39. Вид заблокированного пользователя в списке чатов

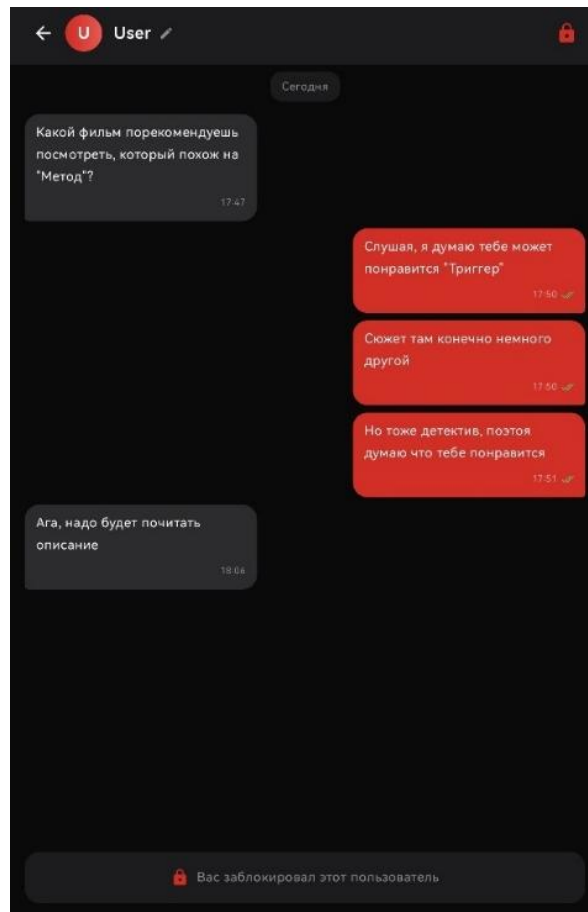


Рис. 3.40. Заблокированный пользователь

В экране переписки предусмотрена возможность изменить отображаемое имя собеседника (рис. 3.41). Измененное имя отображается только для текущего пользователя. Исходное имя собеседника при этом не изменяется. Функция доступна только для текущего пользователя и не влияет на отображение у собеседника.

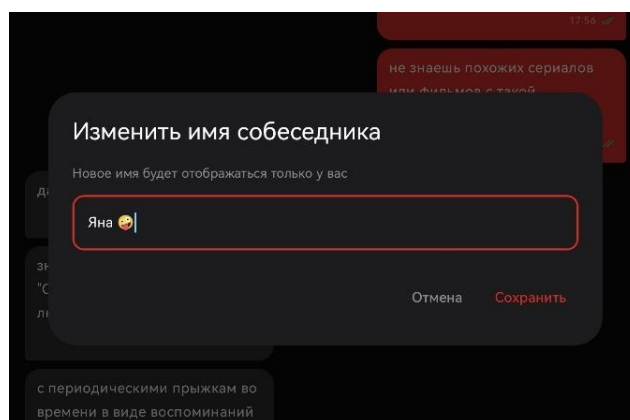


Рис. 3.41. Изменение отображаемого имени собеседника

Выход из аккаунта осуществляется через кнопку «Выйти из аккаунта» в разделе профиля (рис. 3.42). При нажатии открывается диалог подтверждения выхода из аккаунта (рис. 3.43). После подтверждения выполняется переход на экран аутентификации.

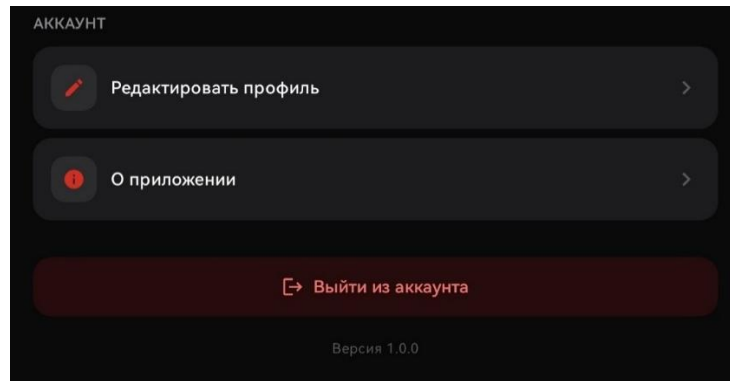


Рис. 3.42. Выход из аккаунта

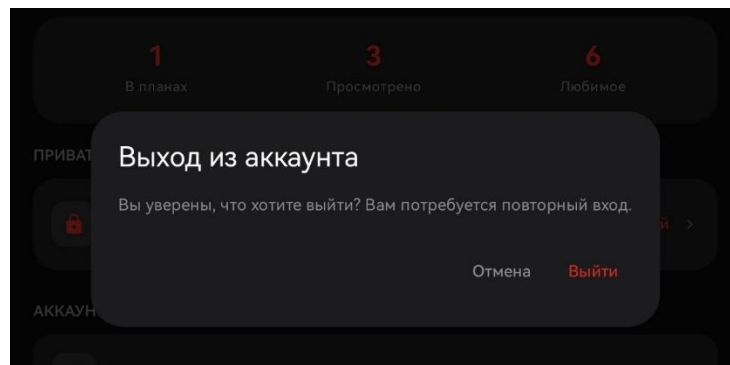


Рис. 3.43. Подтверждение выхода из аккаунта

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы была проведена комплексная разработка информационной системы анализа предпочтений пользователей при выборе кинопродукции для просмотра. Непрерывное расширение каталогов на цифровых платформах и ориентация массовых сервисов на усредненные алгоритмы рекомендаций обусловили актуальность создания специализированного средства, позволяющего пользователям находить единомышленников по кинематографическим вкусам и получать рекомендации на основе реальных предпочтений других людей, а не автоматических подборок.

На первом этапе исследования был проведен анализ предметной области. Изучены ключевые процессы, связанные с выбором кинопродукции, включая поиск информации, формирование интересов, хранение просмотренного контента и обмен рекомендациями. Проанализированы существующие информационные системы – «Иви», «Кинопоиск» и «Letterboxd», выявлены их ограничения и функциональные пробелы. Сопоставление показало, что ни одна из рассматриваемых платформ не объединяет хранение коллекций, автоматическое вычисление процента совпадения интересов и приватную коммуникацию между пользователями, что подтвердило необходимость разработки собственного решения.

На основе проведенного анализа были сформулированы требования к информационной системе, учитывающие особенности выбора кинопродукции, действующие нормативные положения (Федеральные законы №149-ФЗ и №152-ФЗ, ГОСТ Р ИСО/МЭК 12207-2010, ГОСТ Р ИСО/МЭК 25010-2015) и задачи, поставленные перед разработкой. Разработаны функциональные модели в нотации IDEF0, включая контекстную диаграмму и декомпозицию основных процессов: управление доступом пользователей, поиск и получение данных о кинопродукции, управление пользовательскими коллекциями, сравнение коллекций и обеспечение взаимодействия

пользователей. Спроектированы архитектура компонентов системы, структура целевого программного обеспечения на основе паттерна MVVM и модель целевой базы данных в «Cloud Firestore».

С учетом сформированных требований осуществлена программная реализация мобильного приложения для платформы Android. Приложение написано на языке Kotlin с использованием Jetpack Compose, бэкенд построен на «Firebase», внешним источником данных о фильмах и сериалах выступает «Кинопоиск API». Реализованы модули аутентификации, поиска кинопродукции, управления коллекциями, профиля с настройкой приватности, подбора пользователей с близкими вкусами на основе сравнения коллекции «Любимое» и обмена сообщениями.

Поставленные в начале работы цели достигнуты в полном объеме. Разработанное программное средство позволяет пользователям добавлять кинопродукцию в коллекции, находить людей со схожими предпочтениями и обмениваться сообщениями. Полученные результаты могут служить основой для дальнейшего расширения системы, подключения дополнительных источников данных о кинопродукции и внедрения более сложных алгоритмов анализа предпочтений.

СПИСОК ЛИТЕРАТУРЫ

1. Введение в рекомендательные системы // Яндекс Образование: сайт.
URL: <https://education.yandex.ru/handbook/ml/article/intro-recsys> (дата обращения: 29.11.2025 г.). Режим доступа: свободный.
2. Сравнение подходов в рекомендательных системах // Habr, сайт.
URL: <https://habr.com/ru/articles/462615/> (дата обращения: 30.11.2025 г.).
Режим доступа: свободный.
3. Иви (онлайн-кинотеатр) // Википедия : свободная энциклопедия.
URL: [https://ru.wikipedia.org/wiki/Иви_\(онлайн-кинотеатр\)](https://ru.wikipedia.org/wiki/Иви_(онлайн-кинотеатр)) (дата обращения: 29.11.2025 г.). Режим доступа: свободный.
4. Методы машинного обучения в рекомендательных системах онлайн-кинотеатра. URL:
<https://assets.ctfassets.net/oxjq45e8ilak/4jUBgdY0PmUQ24cUQqYaa6/55e886dfd2645f7df4cfb579fde4d600/ML.pdf> (дата обращения: 29.11.2025 г.). Режим доступа: свободный.
5. Кинопоиск // Википедия : свободная энциклопедия. URL:
<https://ru.wikipedia.org/wiki/Кинопоиск> (дата обращения: 30.11.2025 г.). Режим доступа: свободный.
6. Letterboxd // Рувики : интернет-энциклопедия. URL:
<https://ru.ruwiki.ru/wiki/Letterboxd> (дата обращения: 30.11.2025 г.). Режим доступа: свободный.
7. Российская Федерация. Законы. Об информации, информационных технологиях и о защите информации : Федер. закон N 149-ФЗ : принят Государственной Думой 8 июля 2006 г. : одобрен Советом Федерации 14 июля 2006 г. : послед. ред. // КонсультантПлюс : сайт. URL:
https://www.consultant.ru/document/cons_doc_LAW_61798/ (дата обращения: 01.12.2025 г.). Режим доступа: для зарегистрир. пользователей.

8. Российская Федерация. Законы. О персональных данных : Федер. закон N 152-ФЗ : принят Государственной Думой 27 июля 2006 г. : одобрен Советом Федерации 14 июля 2006 г. : послед. ред. // КонсультантПлюс : сайт. URL: https://www.consultant.ru/document/cons_doc_LAW_61801/ (дата обращения: 01.12.2025). Режим доступа: для зарегистрир. пользователей.

9. ГОСТ Р ИСО/МЭК 12207-2010. Информационные технологии. Системная и программная инженерия. Процессы жизненного цикла программных средств : нац. стандарт Российской Федерации : утв. и введен в действие приказом Росстандарта от 30 декабря 2010 г. № 1082-ст : введен впервые : дата введения 2011-07-01. (Система стандартов по информации, библиотечному и издательскому делу) // Кодекс : электрон. фонд правовой и норматив.-техн. информ. URL: <https://docs.cntd.ru/document/1200082859> (дата обращения: 01.12.2025 г.). Режим доступа: свободный.

10. ГОСТ Р ИСО/МЭК 25010-2015. Системы и программная инженерия. Требования к качеству и оценка качества систем и программного обеспечения. Модели качества систем и программного обеспечения : нац. стандарт Российской Федерации : утв. и введен в действие приказом Росстандарта от 11 декабря 2015 г. № 1997-ст : дата введения 2016-07-01 // Кодекс : электрон. фонд правовой и норматив.-техн. информ. URL: <https://docs.cntd.ru/document/1200121069> (дата обращения: 01.12.2025 г.). Режим доступа: свободный.

11. ГОСТ 34.602-2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы : межгосударственный стандарт : издание официальное : принят Межгосударственным советом по стандартизации, метрологии и сертификации (протокол от 30 ноября 2020 г. № 135-П) : введен взамен ГОСТ 34.602-89 : дата введения 2021-07-01.. URL: <https://meganorm.ru/Data/760/76009.pdf> (дата обращения: 01.12.2025 г.). Режим доступа: свободный.

12. Методология IDEF0. Стандарт. Русская версия / перевод с английского фирма «МетаТехнология». – 1993. – 91 с. URL: https://e-learning.bmstu.ru/iu6/pluginfile.php/5193/mod_resource/content/1/Стандарт%20IDEF0.pdf (дата обращения: 10.04.2026 г.). Режим доступа: свободный.

13. Android Studio Documentation: официальная документация среды разработки Android Studio. URL: <https://developer.android.com/studio/intro> (дата обращения: 24.04.2026). Режим доступа: свободный.

14. Kotlin Documentation: официальная документация языка Kotlin. URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 24.04.2026). Режим доступа: свободный.

15. Jetpack Compose Documentation: официальная документация фреймворка Jetpack Compose. URL: <https://developer.android.com/develop/ui/compose/documentation> (дата обращения: 25.04.2026). Режим доступа: свободный.

16. Firebase Documentation: официальная документация платформы Firebase. URL: <https://firebase.google.com/docs> (дата обращения: 27.04.2026). Режим доступа: свободный.

17. Firebase Authentication Documentation: документация по Firebase Authentication. URL: <https://firebase.google.com/docs/auth> (дата обращения: 28.04.2026). Режим доступа: свободный.

18. Cloud Firestore Documentation: документация по Cloud Firestore. URL: <https://firebase.google.com/docs/firestore> (дата обращения: 27.04.2026). Режим доступа: свободный.

19. Kinopoisk.dev API Documentation: неофициальное API Кинопоиска. URL: <https://kinopoisk.dev/documentation> (дата обращения: 29.04.2026). Режим доступа: свободный.

20. Retrofit: репозиторий и документация библиотеки Retrofit. URL: <https://square.github.io/retrofit/> (дата обращения: 29.04.2026). Режим доступа: свободный.

21. Coil: репозиторий и документация библиотеки Coil для загрузки изображений. URL: <https://coil-kt.github.io/coil/> (дата обращения: 30.04.2026).
Режим доступа: свободный.

ПРОГРАММНЫЙ КОД

1. Логика аутентификации.

```

fun loadCurrentUser() {
    val userId = auth.currentUser?.uid ?: return
    viewModelScope.launch {
        try {
            val doc =
db.collection("users").document(userId).get().await()
            _currentUser.value = doc.toObject(User::class.java)
            Log.d("AuthViewModel", "User loaded:
${_currentUser.value?.username}")
        } catch (e: Exception) {
            Log.e("AuthViewModel", "Error loading user", e)
        }
    }
}

fun getCurrentUsername(): String {
    return _currentUser.value?.username ?:
auth.currentUser?.email?.split("@")?.first() ?: "Пользователь"
}

fun login(email: String, password: String) {
    viewModelScope.launch {
        _isLoading.value = true
        try {
            auth.signInWithEmailAndPassword(email,
password).await()
            Log.d("AuthViewModel", "Login successful")
            _isLoggedIn.value = true
            loadCurrentUser()
        } catch (e: Exception) {
            Log.e("AuthViewModel", "Login error", e)
            _authError.value = getRussianErrorMessage€
        } finally {
            _isLoading.value = false
        }
    }
}

fun register(email: String, password: String, username: String) {
    viewModelScope.launch {
        _isLoading.value = true
        try {
            val existingUser = db.collection("users")
                .whereEqualTo("username", username)
                .get()

```



```

        .await()

        if (!existingUser.isEmpty) {
            _authError.value = "Имя пользователя уже занято"
            return@launch
        }

        val authResult =
auth.createUserWithEmailAndPassword(email, password).await()
        val userId = authResult.user?.uid ?: throw
Exception("Не удалось создать пользователя")

        val user = User(
            id = userId,
            username = username,
            email = email,
            userId = username,
            avatarUrl = "",
            bio = "",
            isPrivate = false,
            createdAt = System.currentTimeMillis()
        )

db.collection("users").document(userId).set(user).await()
        Log.d("AuthViewModel", "Registration successful")
        _isLoggedIn.value = true
        loadCurrentUser()

    } catch (e: Exception) {
        Log.e("AuthViewModel", "Registration error", e)
        _authError.value = getRussianErrorMessage€
    } finally {
        _isLoading.value = false
    }
}

private fun getRussianErrorMessage(exception: Exception): String {
    return when {
        exception.message?.contains("The email address is badly
formatted") == true ->
            "Некорректный формат email"
        exception.message?.contains("The supplied auth credential
is incorrect, malformed or has expired") == true ->
            "Неверный email или пароль"
        exception.message?.contains("The supplied auth
credential") == true ->
            "Неверный email или пароль"
        exception.message?.contains("The password is invalid or
the user does not have a password") == true ->

```

```

        "Неверный пароль"
        exception.message?.contains("There is no user record
corresponding to this identifier") == true ->
        "Пользователь с таким email не найден"
        exception.message?.contains("The email address is already
in use by another account") == true ->
        "Пользователь с таким email уже зарегистрирован"
        exception.message?.contains("Password should be at least 6
characters") == true ->
        "Пароль должен содержать не менее 6 символов"
        exception.message?.contains("Network error") == true ->
        "Ошибка сети. Проверьте подключение к интернету"
        exception.message?.contains("A network error (such as
timeout, interrupted connection or unreachable host) has occurred") ==
true ->
        "Ошибка сети. Проверьте подключение к интернету"
        exception.message?.contains("This operation is not
allowed. You must enable this service in the console") == true ->
        "Операция запрещена. Обратитесь к администратору"
        exception.message?.contains("We have blocked all requests
from this device due to unusual activity") == true ->
        "Слишком много попыток. Попробуйте позже"
        else -> exception.message ?: "Произошла неизвестная
ошибка"
    }
}

fun clearError() {
    _authError.value = null
}

fun logout() {
    Log.d("AuthViewModel", "logout() called")
    auth.signOut()
    _isLoggedIn.value = false
    _currentUser.value = null
    Log.d("AuthViewModel", "logout() finished")
}

fun sendPasswordResetEmail(email: String) {
    viewModelScope.launch {
        _isLoading.value = true
        try {
            auth.sendPasswordResetEmail(email).await()
            _authError.value = "✓ Ссылка для сброса пароля
отправлена на вашу почту"
            _isSuccessMessage.value = true
        } catch (e: Exception) {
            _authError.value = e.message ?: "Ошибка отправки
письма"
            _isSuccessMessage.value = false

```

```

        } finally {
            _isLoading.value = false
        }
    }
}

private val _isSuccessMessage = MutableStateFlow(false)
val isSuccessMessage: StateFlow<Boolean> = _isSuccessMessage

fun clearSuccessMessage() {
    _isSuccessMessage.value = false
}

fun setErrorMessage(message: String) {
    _authError.value = message
    _isSuccessMessage.value = false
}

fun updateUser(updatedUser: User) {
    _currentUser.value = updatedUser
}

fun isUserBlocked(otherUserId: String): Boolean {
    return _currentUser.value?.blockedUsers?.contains(otherUserId)
}

fun blockUser(otherUserId: String) {
    val userId = getCurrentUserId() ?: return
    if (userId.isEmpty()) return

    viewModelScope.launch {
        try {
            val currentBlocked = _currentUser.value?.blockedUsers
            ?: emptyList()
            val newBlocked = currentBlocked + otherUserId
            db.collection("users").document(userId)
                .update("blockedUsers", newBlocked)
                .await()
            _currentUser.value =
                _currentUser.value?.copy(blockedUsers = newBlocked)
        } catch (e: Exception) {
            e.printStackTrace()
        }
    }
}

fun unblockUser(otherUserId: String) {
    val userId = getCurrentUserId() ?: return
    if (userId.isEmpty()) return

```

```

viewModelScope.launch {
    try {
        val currentBlocked = _currentUser.value?.blockedUsers
?: emptyList()
        val newBlocked = currentBlocked.filter { it !=
otherUserId }
        db.collection("users").document(userId)
            .update("blockedUsers", newBlocked)
            .await()
        _currentUser.value =
        _currentUser.value?.copy(blockedUsers = newBlocked)
    } catch (e: Exception) {
        e.printStackTrace()
    }
}
}
}
}

```

2. Нижняя панель навигации.

```

@Composable
fun MainScreen(
    navController: NavController,
    authViewModel: AuthViewModel,
    collectionsViewModel: CollectionsViewModel,
    chatsViewModel: ChatsViewModel,
    selectedTab: Int,
    onTabSelected: (Int) -> Unit
) {

    val unreadCount by
chatsViewModel.totalUnreadCount.collectAsState()

    Scaffold(
        containerColor = Color(0xFF0A0A0A),
        bottomBar = {
            NavigationBar(
                containerColor = Color(0xFF141414),
                tonalElevation = 0.dp,
                modifier = Modifier.height(64.dp)
            ) {
                NavigationBarItem(
                    selected = selectedTab == 0,
                    onClick = { onTabSelected(0) },
                    icon = { Icon(Icons.Default.Home,
contentDescription = "Главная") },
                    label = { Text("Главная") },
                    colors = NavigationBarItemDefaults.colors(
                        selectedIconColor = Color(0xFFE50914),
                        selectedTextColor = Color(0xFFE50914),

```

```

        unselectedIconColor = Color(0xFF8E8E93),
        unselectedTextColor = Color(0xFF8E8E93)
    )
)

NavigationBarItem(
    selected = selectedTab == 1,
    onClick = { onTabSelected(1) },
    icon = {
        BadgedBox(
            badge = {
                if (unreadCount > 0) {
                    Badge(
                        containerColor =
Color(0xFFE50914),
                        contentColor = Color.White
                    ) {
                        Text(if (unreadCount > 99)
“99+” else unreadCount.toString())
                    }
                }
            }
        ) {
            Icon(Icons.AutoMirrored.Filled.Chat,
contentDescription = “Чаты”)
        }
    },
    label = { Text(“Чаты”) },
    colors = NavigationBarItemDefaults.colors(
        selectedIconColor = Color(0xFFE50914),
        selectedTextColor = Color(0xFFE50914),
        unselectedIconColor = Color(0xFF8E8E93),
        unselectedTextColor = Color(0xFF8E8E93)
    )
)

NavigationBarItem(
    selected = selectedTab == 2,
    onClick = { onTabSelected(2) },
    icon = { Icon(Icons.Default.People,
contentDescription = “Люди”) },
    label = { Text(“Люди”) },
    colors = NavigationBarItemDefaults.colors(
        selectedIconColor = Color(0xFFE50914),
        selectedTextColor = Color(0xFFE50914),
        unselectedIconColor = Color(0xFF8E8E93),
        unselectedTextColor = Color(0xFF8E8E93)
    )
)
}
}

```

```

    ) { paddingValues ->
        Box(
            modifier = Modifier
                .padding(paddingValues)
                .background(Color(0xFF0A0A0A))
        ){
            when (selectedTab) {
                0 -> HomeScreen(
                    navController = navController,
                    authViewModel = authViewModel,
                    collectionsViewModel = collectionsViewModel
                )
                1 -> ChatsListScreen(
                    navController = navController,
                    chatsViewModel = chatsViewModel
                )
                2 -> PeopleScreen(
                    navController = navController
                )
            }
        }
    }
}

```

3. Навигация между экранами.

```

package com.example.moviematch.navigation

import android.compose.runtime.*
import android.lifecycle.viewmodel.compose.viewModel
import android.navigation.compose.*
import android.compose.ui.Modifier
import android.compose.ui.graphics.Color
import android.compose.foundation.background
import com.example.moviematch.screens.auth.AuthScreen
import com.example.moviematch.screens.main.MainScreen
import com.example.moviematch.screens.profile.ProfileScreen
import com.example.moviematch.screens.chats.ChatScreen
import com.example.moviematch.viewmodel.AuthViewModel
import com.example.moviematch.viewmodel.CollectionsViewModel
import com.example.moviematch.viewmodel.ChatsViewModel
import com.example.moviematch.screens.people.PeopleScreen
import com.example.moviematch.screens.profile.OtherUserProfileScreen

@Composable
fun NavGraph(
    authViewModel: AuthViewModel = viewModel(),
    collectionsViewModel: CollectionsViewModel = viewModel(),
    chatsViewModel: ChatsViewModel = viewModel()
) {

```

```

var selectedTab by remember { mutableStateOf(0) }

val navController = rememberNavController()
val isLoggedIn by authViewModel.isLoggedIn.collectAsState()

LaunchedEffect(isLoggedIn) {
    if (isLoggedIn) {
        chatsViewModel.loadChats()
    }
}

NavHost(
    navController = navController,
    startDestination = if (isLoggedIn) "main" else "auth",
    modifier = Modifier.background(Color(0xFF0A0A0A))
) {
    composable("auth") {
        AuthScreen(navController)
    }

    composable("main") {
        MainScreen(
            navController = navController,
            authViewModel = authViewModel,
            collectionsViewModel = collectionsViewModel,
            chatsViewModel = chatsViewModel,
            selectedTab = selectedTab,
            onTabSelected = { selectedTab = it }
        )
    }

    composable("chat/{chatId}") { backStackEntry ->
        val chatId = backStackEntry.arguments?.getString("chatId")
        ChatScreen(
            chatId = chatId,
            navController = navController,
            chatsViewModel = chatsViewModel
        )
    }

    composable("profile") {
        ProfileScreen(navController)
    }

    composable("people") {
        PeopleScreen(navController = navController)
    }

    composable("user_profile/{userId}") { backStackEntry ->
        val userId = backStackEntry.arguments?.getString("userId")

```

```

?: ""
        OtherUserProfileScreen(
            userId = userId,
            navController = navController,
            authViewModel = authViewModel
        )
    }
}
}

```

4. Логика управления коллекциями.

```

package com.example.moviematch.viewmodel

import android.util.Log
import android.lifecycle.ViewModel
import android.lifecycle.viewModelScope
import com.example.moviematch.models.Movie
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.firestore.FirebaseFirestore
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch
import kotlinx.coroutines.tasks.await

class CollectionsViewModel : ViewModel() {

    private val auth = FirebaseAuth.getInstance()
    private val db = FirebaseFirestore.getInstance()
    private val currentUserId = auth.currentUser?.uid ?: ""

    private val _watched = MutableStateFlow<List<Movie>>(emptyList())
    val watched: StateFlow<List<Movie>> = _watched

    private val _planToWatch =
MutableStateFlow<List<Movie>>(emptyList())
    val planToWatch: StateFlow<List<Movie>> = _planToWatch

    private val _favorites =
MutableStateFlow<List<Movie>>(emptyList())
    val favorites: StateFlow<List<Movie>> = _favorites

    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow<Boolean> = _isLoading

    init {
        loadAllCollections()
    }

    fun loadAllCollections() {
        if (currentUserId.isEmpty()) return
    }
}

```



```

viewModelScope.launch {
    _isLoading.value = true
    try {
        loadWatched()
        loadPlanToWatch()
        loadFavorites()
    } finally {
        _isLoading.value = false
    }
}

private suspend fun loadWatched() {
    val snapshot = db.collection("users")
        .document(currentUserId)
        .collection("watched")
        .get()
        .await()
    _watched.value = snapshot.documents.mapNotNull { doc ->
        doc.toObject(Movie::class.java)
    }
}

private suspend fun loadPlanToWatch() {
    val snapshot = db.collection("users")
        .document(currentUserId)
        .collection("plan_to_watch")
        .get()
        .await()
    _planToWatch.value = snapshot.documents.mapNotNull { doc ->
        doc.toObject(Movie::class.java)
    }
}

private suspend fun loadFavorites() {
    val snapshot = db.collection("users")
        .document(currentUserId)
        .collection("favorites")
        .get()
        .await()
    _favorites.value = snapshot.documents.mapNotNull { doc ->
        doc.toObject(Movie::class.java)
    }
}

fun addToWatched(movie: Movie, userRating: Int = 0, userComment:
String = "") {
    viewModelScope.launch {
        try {
            val movieWithRating = movie.copy(
                userRating = userRating,

```

```

        userComment = userComment,
        addedAt = System.currentTimeMillis()
    )
    db.collection("users")
        .document(currentUserId)
        .collection("watched")
        .document(movie.id.toString())
        .set(movieWithRating)
        .await()
    loadWatched()
} catch (e: Exception) {
    Log.e("CollectionsVM", "Error adding to watched", e)
}
}

fun addToPlanToWatch(movie: Movie) {
    viewModelScope.launch {
        try {
            db.collection("users")
                .document(currentUserId)
                .collection("plan_to_watch")
                .document(movie.id.toString())
                .set(movie)
                .await()
            loadPlanToWatch()
        } catch (e: Exception) {
            Log.e("CollectionsVM", "Error adding to plan", e)
        }
    }
}

fun addToFavorites(movie: Movie) {
    viewModelScope.launch {
        try {
            db.collection("users")
                .document(currentUserId)
                .collection("favorites")
                .document(movie.id.toString())
                .set(movie)
                .await()
            loadFavorites()
        } catch (e: Exception) {
            Log.e("CollectionsVM", "Error adding to favorites", e)
        }
    }
}

fun removeFromWatched(movieId: Int) {
    viewModelScope.launch {
        try {

```

```

        db.collection("users")
            .document(currentUserId)
            .collection("watched")
            .document(movieId.toString())
            .delete()
            .await()
        loadWatched()
        Log.d("CollectionsVM", "Удалено из просмотрено:
$movieId")
    } catch (e: Exception) {
        Log.e("CollectionsVM", "Ошибка удаления из watched",
e)
    }
}

fun removeFromPlanToWatch(movieId: Int) {
    viewModelScope.launch {
        try {
            db.collection("users")
                .document(currentUserId)
                .collection("plan_to_watch")
                .document(movieId.toString())
                .delete()
                .await()
            loadPlanToWatch()
            Log.d("CollectionsVM", "Удалено из планов: $movieId")
        } catch (e: Exception) {
            Log.e("CollectionsVM", "Ошибка удаления из
plan_to_watch", e)
        }
    }
}

fun removeFromFavorites(movieId: Int) {
    viewModelScope.launch {
        try {
            db.collection("users")
                .document(currentUserId)
                .collection("favorites")
                .document(movieId.toString())
                .delete()
                .await()
            loadFavorites()
            Log.d("CollectionsVM", "Удалено из любимого:
$movieId")
        } catch (e: Exception) {
            Log.e("CollectionsVM", "Ошибка удаления из favorites",
e)
        }
    }
}

```

```

    }

    fun isInWatched(movieId: Int): Boolean {
        return _watched.value.any { it.id == movieId }
    }

    fun isInPlanToWatch(movieId: Int): Boolean {
        return _planToWatch.value.any { it.id == movieId }
    }

    fun isInFavorites(movieId: Int): Boolean {
        return _favorites.value.any { it.id == movieId }
    }
}

```

5. Логика профиля.

```

package com.example.moviematch.viewmodel

import android.net.Uri
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.moviematch.models.User
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.firestore.FirebaseFirestore
import com.google.firebase.storage.FirebaseStorage
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch
import kotlinx.coroutines.tasks.await
import java.util.UUID
import android.util.Log

data class UserStats(
    val planToWatch: Int = 0,
    val watched: Int = 0,
    val favorites: Int = 0
)

class ProfileViewModel : ViewModel() {

    private val _errorMessage = MutableStateFlow<String?>(null)
    val errorMessage: StateFlow<String?> = _errorMessage

    fun clearError() {
        _errorMessage.value = null
    }

    fun setErrorMessage(message: String) {
        _errorMessage.value = message
    }
}

```

```

private val auth = FirebaseAuth.getInstance()
private val db = FirebaseFirestore.getInstance()
private val storage = FirebaseStorage.getInstance()

private val _currentUser = MutableStateFlow<User?>(null)
val currentUser: StateFlow<User?> = _currentUser

private val _isLoading = MutableStateFlow(false)
val isLoading: StateFlow<Boolean> = _isLoading

private val _stats = MutableStateFlow<UserStats>()
val stats: StateFlow<UserStats> = _stats

private val _isPrivate = MutableStateFlow(false)
val isPrivate: StateFlow<Boolean> = _isPrivate

private val currentUserId: String
    get() = auth.currentUser?.uid ?: ""

fun loadUserData() {
    val userId = currentUserId
    if (userId.isEmpty()) return

    viewModelScope.launch {
        _isLoading.value = true
        try {
            val doc =
db.collection("users").document(userId).get().await()
            val user = doc.toObject<User>()
            _currentUser.value = user
            _isPrivate.value = user?.isPrivate ?: false
            Log.d("ProfileVM", "isPrivate loaded:
${_isPrivate.value}")
        } catch (e: Exception) {
            e.printStackTrace()
        } finally {
            _isLoading.value = false
        }
    }
}

fun loadPrivacyStatus() {
    val userId = currentUserId
    if (userId.isEmpty()) return

    viewModelScope.launch {
        try {
            val doc =
db.collection("users").document(userId).get().await()
            val isPrivateValue = doc.getBoolean("isPrivate") ?:
false

```

```

        _isPrivate.value = isPrivateValue
        Log.d("ProfileVM", "Privacy status loaded:
$isPrivateValue")
    } catch (e: Exception) {
        e.printStackTrace()
    }
}

fun updateStats(planToWatch: Int, watched: Int, favorites: Int) {
    _stats.value = UserStats(planToWatch, watched, favorites)
}

fun updateUsername(newUsername: String) {
    val userId = currentUserId
    if (userId.isEmpty()) return

    viewModelScope.launch {
        try {
            db.collection("users").document(userId)
                .update("username", newUsername)
                .await()
            val updatedUser = _currentUser.value?.copy(username =
newUsername)
            _currentUser.value = updatedUser
        } catch (e: Exception) {
            e.printStackTrace()
        }
    }
}

fun updateUserId(newUserId: String) {
    val userId = currentUserId
    if (userId.isEmpty()) return

    if (!newUserId.matches(Regex("^[a-zA-Z0-9_-]+$"))) {
        _errorMessage.value = "ID может содержать только латиницу,
цифры, _ и -"
        return
    }

    viewModelScope.launch {
        _isLoading.value = true
        try {
            val existingUser = db.collection("users")
                .whereEqualTo("userId", newUserId)
                .get()
                .await()

            if (!existingUser.isEmpty &&
existingUser.documents.firstOrNull()?.id != userId) {

```

```

        _errorMessage.value = "Пользователь с таким ID уже
существует"
        return@launch
    }

    db.collection("users").document(userId)
        .update("userId", newUserId)
        .await()
    val updatedUser = _currentUser.value?.copy(userId =
newUserId)
        _currentUser.value = updatedUser
        _errorMessage.value = "ID успешно изменен"

    } catch (e: Exception) {
        _errorMessage.value = e.message ?: "Ошибка при смене
ID"
        e.printStackTrace()
    } finally {
        _isLoading.value = false
    }
}

fun updatePrivacy(isPrivate: Boolean) {
    val userId = currentUserId
    if (userId.isEmpty()) return

    viewModelScope.launch {
        try {
            db.collection("users").document(userId)
                .update("isPrivate", isPrivate)
                .await()
            _isPrivate.value = isPrivate
            val updatedUser = _currentUser.value?.copy(isPrivate =
isPrivate)
                _currentUser.value = updatedUser
                Log.d("ProfileVM", "Privacy updated to: $isPrivate")
        } catch (e: Exception) {
            e.printStackTrace()
        }
    }
}

fun updateAvatarUri(imageUriString: String) {
    val userId = currentUserId
    if (userId.isEmpty()) return

    viewModelScope.launch {
        _isLoading.value = true
        try {
            val imageUri = Uri.parse(imageUriString)

```

```

        val fileRef =
storage.reference.child("avatars/${UUID.randomUUID()}.jpg")
        fileRef.putFile(imageUri).await()
        val downloadUrl = fileRef.downloadUrl.await()

        db.collection("users").document(userId)
            .update("avatarUrl", downloadUrl.toString())
            .await()

        val updatedUser = _currentUser.value?.copy(avatarUrl =
downloadUrl.toString())
        _currentUser.value = updatedUser
    } catch (e: Exception) {
        e.printStackTrace()
    } finally {
        _isLoading.value = false
    }
}

fun updateBio(newBio: String) {
    val userId = currentUserId
    if (userId.isEmpty()) return

    viewModelScope.launch {
        try {
            db.collection("users").document(userId)
                .update("bio", newBio)
                .await()
            val updatedUser = _currentUser.value?.copy(bio =
newBio)
            _currentUser.value = updatedUser
        } catch (e: Exception) {
            e.printStackTrace()
        }
    }
}
}

```

6. Логика профиля другого пользователя.

```

package com.example.moviematch.viewmodel

import android.util.Log
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.moviematch.models.Chat
import com.example.moviematch.models.Movie
import com.example.moviematch.models.User
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.firestore.FirebaseFirestore

```



```

import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch
import kotlinx.coroutines.tasks.await

class OtherUserProfileViewModel(
    private val authViewModel: AuthViewModel
) : ViewModel() {

    private val db = FirebaseFirestore.getInstance()
    private val currentUserId =
        FirebaseAuth.getInstance().currentUser?.uid ?: ""

    private val _otherUser = MutableStateFlow<User?>(null)
    val otherUser: StateFlow<User?> = _otherUser

    private val _favorites =
        MutableStateFlow<List<Movie>>(emptyList())
    val favorites: StateFlow<List<Movie>> = _favorites

    private val _planned = MutableStateFlow<List<Movie>>(emptyList())
    val planned: StateFlow<List<Movie>> = _planned

    private val _watched = MutableStateFlow<List<Movie>>(emptyList())
    val watched: StateFlow<List<Movie>> = _watched

    private val _notInMyCollections =
        MutableStateFlow<List<Movie>>(emptyList())
    val notInMyCollections: StateFlow<List<Movie>> =
        _notInMyCollections

    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow<Boolean> = _isLoading

    private val _isPrivate = MutableStateFlow(false)
    val isPrivate: StateFlow<Boolean> = _isPrivate

    fun loadUserProfile(userId: String) {
        viewModelScope.launch {
            _isLoading.value = true
            try {
                val doc =
                    db.collection("users").document(userId).get().await()
                val user = doc.toObject(User::class.java)
                _otherUser.value = user
                _isPrivate.value = user?.isPrivate ?: false

                if (!_isPrivate.value) {
                    loadUserCollections(userId)
                }
            } catch (e: Exception) {

```

```

        Log.e("OtherUserProfileVM", "Error loading user
profile", e)
    } finally {
        _isLoading.value = false
    }
}

private suspend fun loadUserCollections(userId: String) {
    try {
        val plannedSnapshot = db.collection("users")
            .document(userId)
            .collection("plan_to_watch")
            .get()
            .await()
        _planned.value = plannedSnapshot.documents.mapNotNull {
doc ->
            doc.toObject(Movie::class.java)
        }

        val watchedSnapshot = db.collection("users")
            .document(userId)
            .collection("watched")
            .get()
            .await()
        _watched.value = watchedSnapshot.documents.mapNotNull {
doc ->
            doc.toObject(Movie::class.java)
        }

        val favoritesSnapshot = db.collection("users")
            .document(userId)
            .collection("favorites")
            .get()
            .await()
        _favorites.value = favoritesSnapshot.documents.mapNotNull
{ doc ->
            doc.toObject(Movie::class.java)
        }
    } catch (e: Exception) {
        Log.e("OtherUserProfileVM", "Error loading user
collections", e)
    }
}

fun loadNotInMyCollections(userId: String) {
    viewModelScope.launch {
        try {
            val userFavorites = db.collection("users")
                .document(userId)
                .collection("favorites")

```

```

        .get()
        .await()
        .documents.mapNotNull {
it.toObject(Movie::class.java) }

        val userPlanned = db.collection("users")
        .document(userId)
        .collection("plan_to_watch")
        .get()
        .await()
        .documents.mapNotNull {
it.toObject(Movie::class.java) }

        val userWatched = db.collection("users")
        .document(userId)
        .collection("watched")
        .get()
        .await()
        .documents.mapNotNull {
it.toObject(Movie::class.java) }

        val allUserMovies = (userFavorites + userPlanned +
userWatched)
        .distinctBy { it.id }

        val myUserId =
FirebaseAuth.getInstance().currentUser?.uid ?: ""
        val myFavorites = db.collection("users")
        .document(myUserId)
        .collection("favorites")
        .get()
        .await()
        .documents.mapNotNull {
it.toObject(Movie::class.java) }

        val myPlanned = db.collection("users")
        .document(myUserId)
        .collection("plan_to_watch")
        .get()
        .await()
        .documents.mapNotNull {
it.toObject(Movie::class.java) }

        val myWatched = db.collection("users")
        .document(myUserId)
        .collection("watched")
        .get()
        .await()
        .documents.mapNotNull {
it.toObject(Movie::class.java) }

```

```

        val allMyMovies = (myFavorites + myPlanned +
myWatched)
            .distinctBy { it.id }
        val myMovieIds = allMyMovies.map { it.id }.toSet()

        val notInMy = allUserMovies
            .filter { !myMovieIds.contains(it.id) }
            .sortedByDescending { it.rating }

        _notInMyCollections.value = notInMy

    } catch (e: Exception) {
        Log.e("OtherUserProfileVM", "Error loading not in my
collections", e)
    }
}

fun loadUserFavorites(userId: String) {
    if (_isPrivate.value) return
    viewModelScope.launch {
        try {
            val snapshot = db.collection("users")
                .document(userId)
                .collection("favorites")
                .get()
                .await()
            _favorites.value = snapshot.documents.mapNotNull { doc
->
                doc.toObject(Movie::class.java)
            }
        } catch (e: Exception) {
            Log.e("OtherUserProfileVM", "Error loading favorites",
e)
        }
    }
}

suspend fun getOrCreateChat(otherUserId: String): String? {
    val currentUserId =
FirebaseAuth.getInstance().currentUser?.uid ?: return null
    if (otherUserId.isEmpty()) return null

    val querySnapshot = db.collection("chats")
        .whereArrayContains("participants", currentUserId)
        .get()
        .await()

    val existingChat = querySnapshot.documents.firstOrNull { doc -
>
        val participants = doc["participants"] as? List<String>

```

```

        participants?.contains(otherUserId) == true
    }

    if (existingChat != null) {
        return existingChat.id
    }

    val chat = Chat(
        id = "",
        participants = listOf(currentUserId, otherUserId),
        lastMessage = "",
        lastMessageTime = System.currentTimeMillis(),
        participantTyping = emptyMap()
    )

    val chatRef = db.collection("chats").document()
    chatRef.set(chat.copy(id = chatRef.id)).await()

    return chatRef.id
}

suspend fun getMovieStatusForCurrentUser(movieId: Int):
MovieStatus {
    val currentUserId =
    FirebaseAuth.getInstance().currentUser?.uid ?: return MovieStatus.NONE

    return try {
        val watchedDoc = db.collection("users")
            .document(currentUserId)
            .collection("watched")
            .document(movieId.toString())
            .get()
            .await()
        if (watchedDoc.exists()) return MovieStatus.WATCHED

        val plannedDoc = db.collection("users")
            .document(currentUserId)
            .collection("plan_to_watch")
            .document(movieId.toString())
            .get()
            .await()
        if (plannedDoc.exists()) return MovieStatus.PLANNED

        val favoriteDoc = db.collection("users")
            .document(currentUserId)
            .collection("favorites")
            .document(movieId.toString())
            .get()
            .await()
        if (favoriteDoc.exists()) return MovieStatus.FAVORITE
    }
}

```

```

        MovieStatus.NONE
    } catch (e: Exception) {
        MovieStatus.NONE
    }
}

enum class MovieStatus {
    WATCHED, PLANNED, FAVORITE, NONE
}

fun isUserBlocked(otherUserId: String): Boolean {
    return authViewModel.isUserBlocked(otherUserId)
}

fun blockUser(otherUserId: String) {
    authViewModel.blockUser(otherUserId)
}

fun unblockUser(otherUserId: String) {
    authViewModel.unblockUser(otherUserId)
}
}

```

7. Логика поиска по схожести коллекций.

```

class PeopleViewModel : ViewModel() {

    private val auth = FirebaseAuth.getInstance()
    private val db = FirebaseFirestore.getInstance()
    private val currentUserId = auth.currentUser?.uid ?: ""

    private val _users =
MutableStateFlow<List<UserWithMatch>>(emptyList())
    val users: StateFlow<List<UserWithMatch>> = _users

    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow<Boolean> = _isLoading

    private val _searchQuery = MutableStateFlow("")
    val searchQuery: StateFlow<String> = _searchQuery

    fun loadUsersWithMatch() {
        if (currentUserId.isEmpty()) return

        viewModelScope.launch {
            _isLoading.value = true
            try {
                val allUsersSnapshot = db.collection("users")
                    .whereNotEqualTo("id", currentUserId)

```

```

        .get()
        .await()

        val allUsers = allUsersSnapshot.documents.mapNotNull {
doc ->
            doc.toObject(User::class.java)
        }

        val myFavorites = getFavoritesMovies()

        val usersWithMatch = mutableListOf<UserWithMatch>()

        for (user in allUsers) {
            val userFavorites =
getFavoritesMoviesForUser(user.id)
            val (percentage, common) =
calculateMatchPercentage(myFavorites, userFavorites)

            if (percentage >= 50) {
                usersWithMatch.add(UserWithMatch(user,
percentage, common))
            }
        }

        _users.value = usersWithMatch.sortedByDescending {
it.matchPercentage }

    } catch (e: Exception) {
        Log.e("PeopleVM", "Error loading users", e)
    } finally {
        _isLoading.value = false
    }
}

private suspend fun getFavoritesMovies(): List<Movie> {
    return try {
        val snapshot = db.collection("users")
            .document(currentUserId)
            .collection("favorites")
            .get()
            .await()
        snapshot.documents.mapNotNull { doc ->
            doc.toObject(Movie::class.java)
        }
    } catch (e: Exception) {
        emptyList()
    }
}

private suspend fun getFavoritesMoviesForUser(userId: String):

```

```

List<Movie> {
    return try {
        val snapshot = db.collection("users")
            .document(userId)
            .collection("favorites")
            .get()
            .await()
        snapshot.documents.mapNotNull { doc ->
            doc.toObject(Movie::class.java)
        }
    } catch (e: Exception) {
        emptyList()
    }
}

private fun calculateMatchPercentage(
    myMovies: List<Movie>,
    userMovies: List<Movie>
): Pair<Int, List<Movie>> {
    if (myMovies.isEmpty() || userMovies.isEmpty()) return Pair(0,
emptyList())

    val myIds = myMovies.map { it.id }.toSet()
    val userIds = userMovies.map { it.id }.toSet()

    val commonIds = myIds.intersect(userIds)
    val commonMovies = myMovies.filter { commonIds.contains(it.id)
}

    val percentage = (commonIds.size * 100) / maxOf(myIds.size,
userIds.size)

    return Pair(percentage, commonMovies)
}

fun searchUsers(query: String) {
    _searchQuery.value = query
    if (query.isBlank()) {
        loadUsersWithMatch()
        return
    }

    viewModelScope.launch {
        _isLoading.value = true
        try {
            val allUsersSnapshot = db.collection("users")
                .whereNotEqualTo("id", currentUserId)
                .get()
                .await()

            val filteredUsers = allUsersSnapshot.documents

```



```

        .mapNotNull { doc ->
doc.toObject(User::class.java) }
        .filter { user ->
            user.userId.contains(query, ignoreCase = true)
||
            user.username.contains(query,
ignoreCase = true)
        }

        val myFavorites = getFavoritesMovies()
        val usersWithMatch = mutableListOf<UserWithMatch>()

        for (user in filteredUsers) {
            val userFavorites =
getFavoritesMoviesForUser(user.id)
            val (percentage, common) =
calculateMatchPercentage(myFavorites, userFavorites)
            usersWithMatch.add(UserWithMatch(user, percentage,
common))
        }

        _users.value = usersWithMatch.sortedByDescending {
it.matchPercentage }

    } catch (e: Exception) {
        Log.e("PeopleVM", "Error searching users", e)
    } finally {
        _isLoading.value = false
    }
}
}
}

```

8. Логика чатов.

```

fun loadChats() {
    val userId = currentUserId
    if (userId.isEmpty()) return

    db.collection("chats")
        .whereArrayContains("participants", userId)
        .orderBy("lastMessageTime", Query.Direction.DESENDING)
        .addSnapshotListener { snapshot, error ->
            if (error != null) {
                Log.e("ChatsVM", "Error loading chats", error)
                return@addSnapshotListener
            }
            val chatsList = snapshot?.documents?.mapNotNull {
document ->
                val chat = document.toObject(Chat::class.java)
                chat?.copy(id = document.id)
            }
        }
    }
}

```

```

        } ?: emptyList()
        _chats.value = chatsList
        updateTotalUnreadCount()
    }
}

fun loadMessages(chatId: String) {
    db.collection("chats")
        .document(chatId)
        .collection("messages")
        .orderBy("timestamp", Query.Direction.ASCENDING)
        .addSnapshotListener { snapshot, error ->
            if (error != null) {
                Log.e("ChatsVM", "Error loading messages", error)
                return@addSnapshotListener
            }
            val messagesList = snapshot?.documents?.mapNotNull {
document ->
                val message =
document.toObject(Message::class.java)
                message?.copy(id = document.id)
            } ?: emptyList()

            val oldSize = _messages.value.size
            _messages.value = messagesList

            if (messagesList.size > oldSize &&
messagesList.isNotEmpty()) {
                val newMessage = messagesList.last()
                if (newMessage.senderId != currentUserId) {
                    viewModelScope.launch {
                        val senderName =
getSenderName(newMessage.senderId)
                        onNewMessageCallback?.invoke(senderName,
newMessage.text, chatId)
                    }
                }
            }

            markMessagesAsRead(chatId, messagesList)
            updateTotalUnreadCount()
        }
}

fun sendMessage(chatId: String, text: String) {
    if (text.isBlank()) return
    val userId = currentUserId
    if (userId.isEmpty()) return

    val messageData = mapOf(
        "senderId" to userId,

```

```

        "text" to text,
        "timestamp" to System.currentTimeMillis(),
        "isDelivered" to true,
        "isRead" to false
    )

    val messageRef = db.collection("chats")
        .document(chatId)
        .collection("messages")
        .document()

    messageRef.set(messageData)
        .addOnSuccessListener {
            db.collection("chats").document(chatId)
                .update(
                    mapOf(
                        "lastMessage" to text,
                        "lastMessageTime" to
System.currentTimeMillis()
                    )
                )
                .updateTotalUnreadCount()
        }
        .addOnFailureListener { e ->
            Log.e("ChatsVM", "Error sending message", e)
        }
    }

    private fun markMessagesAsRead(chatId: String, messages:
List<Message>) {
        val userId = currentUserId
        if (userId.isEmpty()) return

        val unreadMessages = messages.filter {
            it.senderId != userId && !it.isRead
        }

        Log.d("ChatsVM", "Marking ${unreadMessages.size} messages as
read in chat $chatId")

        if (unreadMessages.isEmpty()) return

        for (message in unreadMessages) {
            db.collection("chats")
                .document(chatId)
                .collection("messages")
                .document(message.id)
                .update("isRead", true)
        }
        updateTotalUnreadCount()
    }
}

```

```

suspend fun getUnreadCount(chatId: String): Int {
    val userId = currentUserId
    if (userId.isEmpty()) return 0

    return try {
        val snapshot = db.collection("chats")
            .document(chatId)
            .collection("messages")
            .whereEqualTo("isRead", false)
            .whereNotEqualTo("senderId", userId)
            .get()
            .await()
        snapshot.size()
    } catch (e: Exception) {
        Log.e("ChatsVM", "Error getting unread count", e)
        0
    }
}

private fun updateTotalUnreadCount() {
    viewModelScope.launch {
        var total = 0
        for (chat in _chats.value) {
            val count = getUnreadCount(chat.id)
            total += count
        }
        Log.d("ChatsVM", "Setting totalUnreadCount to: $total")
        _totalUnreadCount.value = total
    }
}

suspend fun getOtherUserByChatId(chatId: String): User? {
    val userId = currentUserId
    if (userId.isEmpty()) return null

    return try {
        val chatDoc =
db.collection("chats").document(chatId).get().await()
        val participants = chatDoc.get("participants") as?
List<String> ?: emptyList()
        val otherUserId = participants.firstOrNull { it != userId
} ?: return null
        val userDoc =
db.collection("users").document(otherUserId).get().await()
        userDoc.toObject(User::class.java)
    } catch (e: Exception) {
        Log.e("ChatsVM", "Error getting other user", e)
        null
    }
}

```

```

suspend fun getOtherUser(chat: Chat): User? {
    val userId = currentUserId
    if (userId.isEmpty()) return null
    val otherUserId = chat.participants.firstOrNull { it != userId } ?: return null
    return try {
        val doc =
db.collection("users").document(otherUserId).get().await()
        doc.toObject(User::class.java)
    } catch (e: Exception) {
        null
    }
}

suspend fun getOrCreateChat(otherUserId: String): String? {
    val userId = currentUserId
    if (userId.isEmpty()) return null

    val querySnapshot = db.collection("chats")
        .whereArrayContains("participants", userId)
        .get()
        .await()

    val existingChat = querySnapshot.documents.firstOrNull { doc -
>
        val participants = doc["participants"] as? List<String>
        participants?.contains(otherUserId) == true
    }

    if (existingChat != null) {
        return existingChat.id
    }

    val chat = Chat(
        id = "",
        participants = listOf(userId, otherUserId),
        lastMessage = "",
        lastMessageTime = System.currentTimeMillis(),
        participantTyping = emptyMap()
    )

    val chatRef = db.collection("chats").document()
    chatRef.set(chat.copy(id = chatRef.id)).await()
    return chatRef.id
}

private suspend fun getSenderName(senderId: String): String {
    return try {
        val doc =
db.collection("users").document(senderId).get().await()

```



```

        "lastMessageTime" to lastMessageTime
    )
    ).await()
} else {
    chatRef.update(
        mapOf(
            "lastMessage" to "",
            "lastMessageTime" to 0L
        )
    ).await()
}

loadChats()

Log.d("ChatsVM", "Message $messageId deleted, chat
updated")
} catch (e: Exception) {
    Log.e("ChatsVM", "Error deleting message", e)
}
}

fun updateDisplayName(chatId: String, otherUserId: String,
newName: String) {
    val userId = currentUserId
    if (userId.isEmpty()) return

    viewModelScope.launch {
        try {
            db.collection("chats").document(chatId)
                .update("displayNames.${otherUserId}", newName)
                .await()
            Log.d("ChatsVM", "Display name for $otherUserId
updated to $newName")
        } catch (e: Exception) {
            Log.e("ChatsVM", "Error updating display name", e)
        }
    }
}

fun blockUser(chatId: String, otherUserId: String) {
    val userId = currentUserId
    if (userId.isEmpty()) return

    viewModelScope.launch {
        try {
            db.collection("chats").document(chatId)
                .update("blockedBy.${otherUserId}", true)
                .await()
            Log.d("ChatsVM", "User $otherUserId blocked")
        } catch (e: Exception) {

```

```

        Log.e("ChatsVM", "Error blocking user", e)
    }
}

fun unblockUser(chatId: String, otherUserId: String) {
    val userId = currentUserId
    if (userId.isEmpty()) return

    viewModelScope.launch {
        try {
            db.collection("chats").document(chatId)
                .update("blockedBy.${otherUserId}", false)
                .await()
            Log.d("ChatsVM", "User $otherUserId unblocked")
        } catch (e: Exception) {
            Log.e("ChatsVM", "Error unblocking user", e)
        }
    }
}

fun isBlocked(chat: Chat): Boolean {
    val userId = currentUserId
    if (userId.isEmpty()) return false
    val otherUserId = chat.participants.firstOrNull { it != userId }
} ?: return false
    return chat.blockedBy[otherUserId] == true
}

fun isBlockedByOther(chat: Chat): Boolean {
    val userId = currentUserId
    if (userId.isEmpty()) return false
    return chat.blockedBy[userId] == true
}
}

```